Security Incident Detection and Response Management System

Backend – Final Project, Sprint I

Software Tools Lab (430L)

Students: Elie Medawar, Marc Bteich, Zoulfikar Shokor

Date of submission: 20 October 2026

GitHub repository: https://github.com/eliemedawar/435L-Project

Title page and table of contents: (Elie Medawar)

2. Table of Contents (Elie Medawar)

2. Table of Contents (Elie Medawar) 2

3. Introduction (Marc Bteich) 4

4. System Architecture (Marc Bteich) 4

5. Implementation Details (Elie Medawar, Marc Bteich) 5

5.1 Users and Identity Management Service (Elie Medawar) 5

5.2 Asset Management Service (Marc Bteich) 16

5.3 Incident Management Service (Elie Medawar) 21

5.4 Threat Intelligence Service (Marc Bteich) 36

5.5 Security Audit Logging Service (Elie Medawar) 40

5.6 Security Reports Service – Summary, Incident and Activity Reports (Elie Medawar) 52

5.6 Security Reports Service – Asset, Threat and Vulnerability Reports (Marc Bteich) 56

5.7 Vulnerability Management Service (Marc Bteich) 58

5.8 Notification and Alert Management Service (Elie Medawar) 62

5.9 Security Playbook and Response Automation Service (Marc Bteich) 74

6. Database Design (Marc Bteich) 79

7. Error Handling and Validation (Elie Medawar) 88

8. API Documentation (Marc Bteich) 94

Users (Marc Bteich) 95

Assets (Marc Bteich) 100

Incidents (Marc Bteich) 107

Threats (Marc Bteich) 112

Logs (Marc Bteich) 116

Reports (Marc Bteich) 121

Vulnerabilities (Marc Bteich) 125

Notifications (Marc Bteich) 129

Playbooks (Marc Bteich) 134

Person B field rules and responses (Marc Bteich) 139

9. Authentication and Authorization (Elie Medawar) 140

10. GitHub and Version Control (Elie Medawar) 145

11. References (Elie Medawar, Marc Bteich) 147

Use of AI tools 148

Section	Responsible member
1. Title Page	Elie Medawar
2. Table of Contents	Elie Medawar
3. Introduction	Marc Bteich
4. System Architecture	Marc Bteich
5.1 Users and Identity Management Service	Elie Medawar
5.2 Asset Management Service	Marc Bteich
5.3 Incident Management Service	Elie Medawar
5.4 Threat Intelligence Service	Marc Bteich
5.5 Security Audit Logging Service	Elie Medawar
5.6 Security Reports Service	Elie Medawar (summary, incidents, activity); Marc Bteich (assets, threats, vulnerabilities)
5.7 Vulnerability Management Service	Marc Bteich
5.8 Notification and Alert Management Service	Elie Medawar
5.9 Security Playbook and Response Automation Service	Marc Bteich
6. Database Design	Marc Bteich
7. Error Handling and Validation	Elie Medawar
8. API Documentation	Marc Bteich
9. Authentication and Authorization	Elie Medawar
10. GitHub and Version Control	Elie Medawar
11. References	Elie Medawar, Marc Bteich

Evidence note: every API screenshot is a browser capture of a genuine Newman (Postman runtime) execution record from the team's running system, with credentials and tokens redacted. Each caption names the student who developed the API, the API title and what the request does.

3. Introduction (Marc Bteich)

The system provides a backend for recording and investigating security incidents, identifying affected organizational assets, maintaining threat indicators, tracking vulnerabilities and following response procedures. A central record of users, incidents, assets, audit events and notifications supports coordinated security work. This sprint implements REST endpoints, persistent storage, authentication, role-based permissions, validation and reproducible API tests; a frontend is outside the professor's required scope.

The objectives are to separate responsibilities into independent services, preserve relationships through database constraints, restrict operations to authorized roles, record security-relevant changes and produce statistics from service APIs. Marc owns Assets, Threat Intelligence, Vulnerabilities, Playbooks/Actions, three reports, the shared schema/sample data and documentation integration. Elie owns Users, Incidents, Audit Logs, Notifications, the shared foundation and the other three reports.

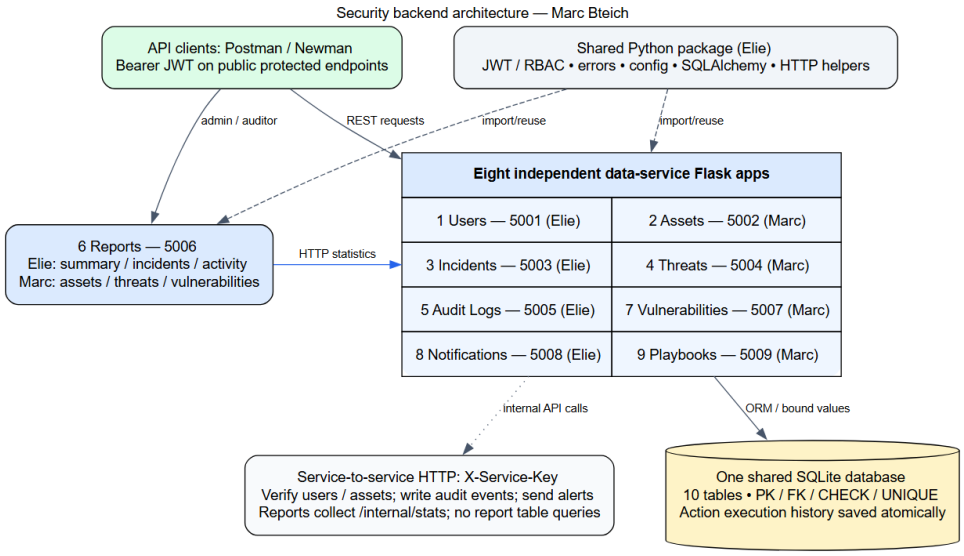
4. System Architecture (Marc Bteich)

Nine Flask applications run independently on ports 5001–5009. They share one SQLite database and common Python infrastructure. Each service uses model classes for persistence, a service/validator layer for business rules and thin routes for authorization and responses. SQLAlchemy binds values rather than concatenating client input into SQL. SQLite development is permitted by the team plan; MySQL execution has not been verified.

Public requests carry Bearer JWTs containing user_id, username, role and expiry. Shared role checks distinguish admin, analyst, user and auditor. Internal HTTP requests carry X-Service-Key. Assets verifies owners through Users; Incidents and Vulnerabilities verify assets through Assets; Playbooks verifies assigned users through Users. Every successful mutation requests an audit event. Critical threat/vulnerability creation identifies all admins through Users and requests notifications through Notifications.

The Reports app registers both owners' blueprints. It calls source /internal/stats endpoints through the existing get_service/ReportBuilder helpers and does not read source tables for report calculations. A missing required source yields 503. Logs and Notifications use the shared best-effort HTTP clients; normal writes can succeed if these recipients are unavailable. Playbook deletion is stricter: execution-history events must be acknowledged by Logs before actions can be cascaded away.

Architecture



5. Implementation Details (Elie Medawar, Marc Bteich)

5.1 Users and Identity Management Service (Elie Medawar)

The Users service (port 5001) owns user accounts, passwords, roles and account status, and it issues the login tokens every other service checks. Code: `services/users/` (routes, `UserService`, `PasswordValidator`, `UserValidator`) and `models/users.py`.

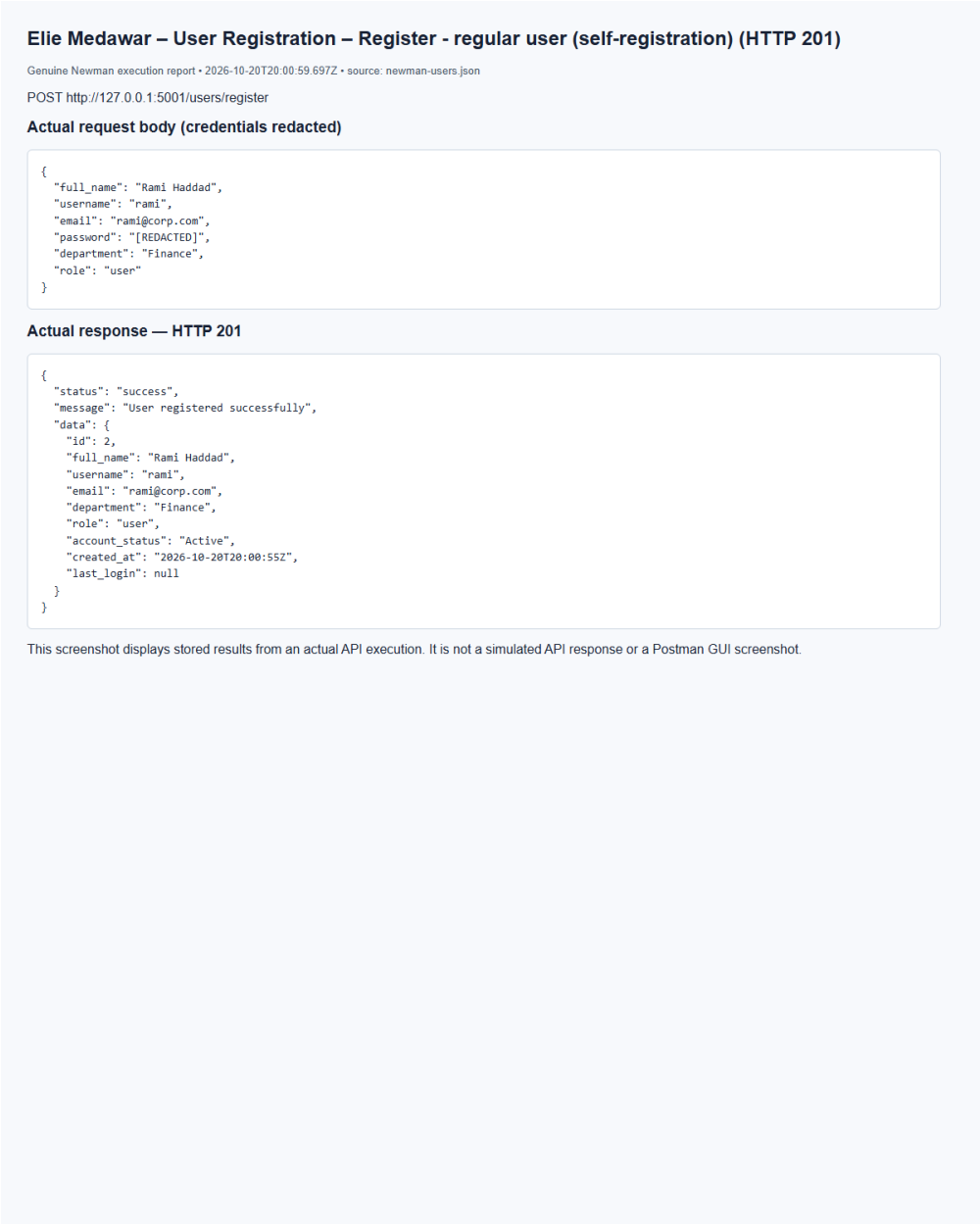
API	Method	Endpoint	Allowed roles	Success
User Registration	POST	/users/register	Public (role user); admin for any role	201
User Login	POST	/users/login	Public	200
Update User Information	PUT	/users/{id}	admin (any user); user (self only)	200
Delete User	DELETE	/users/{id}	admin	200
Retrieve All Users	GET	/users (optional ?role=)	admin, auditor	200
Retrieve User By Username	GET	/users/{username}	admin, auditor; user (self only)	200
Internal: user by id, users by role, statistics	GET	/internal/users/{id}, /internal/users, /internal/stats	Other services (X-Service-Key)	200

Validation and duplicate usernames. Registration checks the required fields, the email format, the username format (3–30 letters, digits, _ or .) and the password rules (8–64 characters with an uppercase letter, a lowercase letter, a digit and a symbol). Before inserting, `UserService` looks up the username case-insensitively and the email, and returns 409 Conflict if either is taken. The UNIQUE constraints on users.username and users.email are a second safety net: if

two requests race, the database rejects the second insert and the service still answers 409. Without an admin token, registration can only create the user role; asking for a privileged role returns 403 and is logged.

Database interaction. One row per account in users: id (PK), full_name, username (unique), email (unique), password_hash (bcrypt), department, role (CHECK: admin, analyst, user, auditor), account_status (CHECK: Active, Inactive, Suspended), created_at, last_login. Login updates last_login. Inactive and Suspended accounts cannot log in. Deleting a user who reported incidents is refused with 409, and the message suggests setting the account to Inactive instead.

Elie Medawar – User Registration – Register - regular user (self-registration) (HTTP 201)



Elie Medawar – User Registration – Register - regular user (self-registration) (HTTP 201)

Genuine Newman execution report • 2026-10-20T20:00:59.697Z • source: newman-users.json

POST http://127.0.0.1:5001/users/register

Actual request body (credentials redacted)


```
{
  "full_name": "Rami Haddad",
  "username": "rami",
  "email": "rami@corp.com",
  "password": "[REDACTED]",
  "department": "Finance",
  "role": "user"
}
```

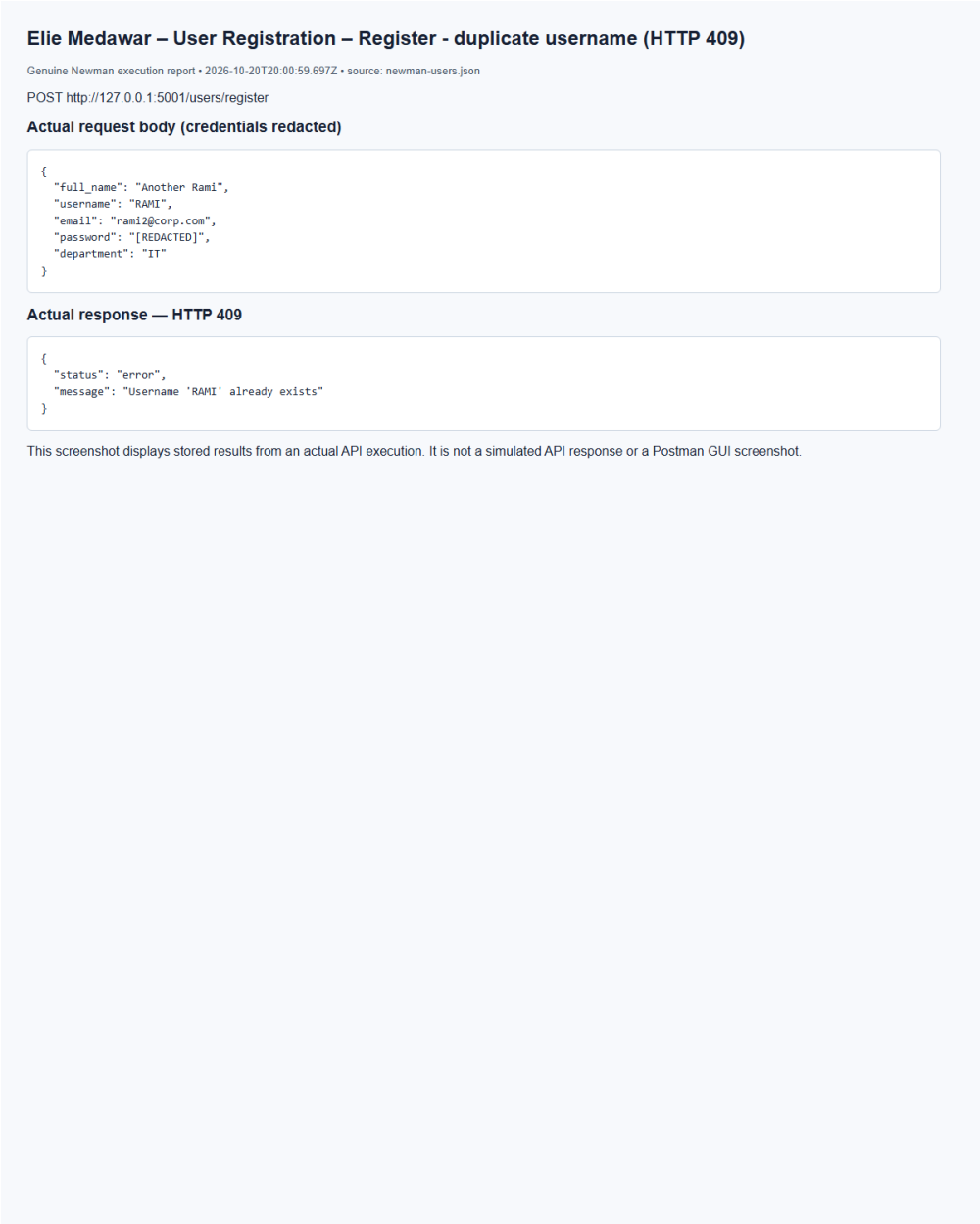

Actual response — HTTP 201


```
{
  "status": "success",
  "message": "User registered successfully",
  "data": {
    "id": 2,
    "full_name": "Rami Haddad",
    "username": "rami",
    "email": "rami@corp.com",
    "department": "Finance",
    "role": "user",
    "account_status": "Active",
    "created_at": "2026-10-20T20:00:55Z",
    "last_login": null
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

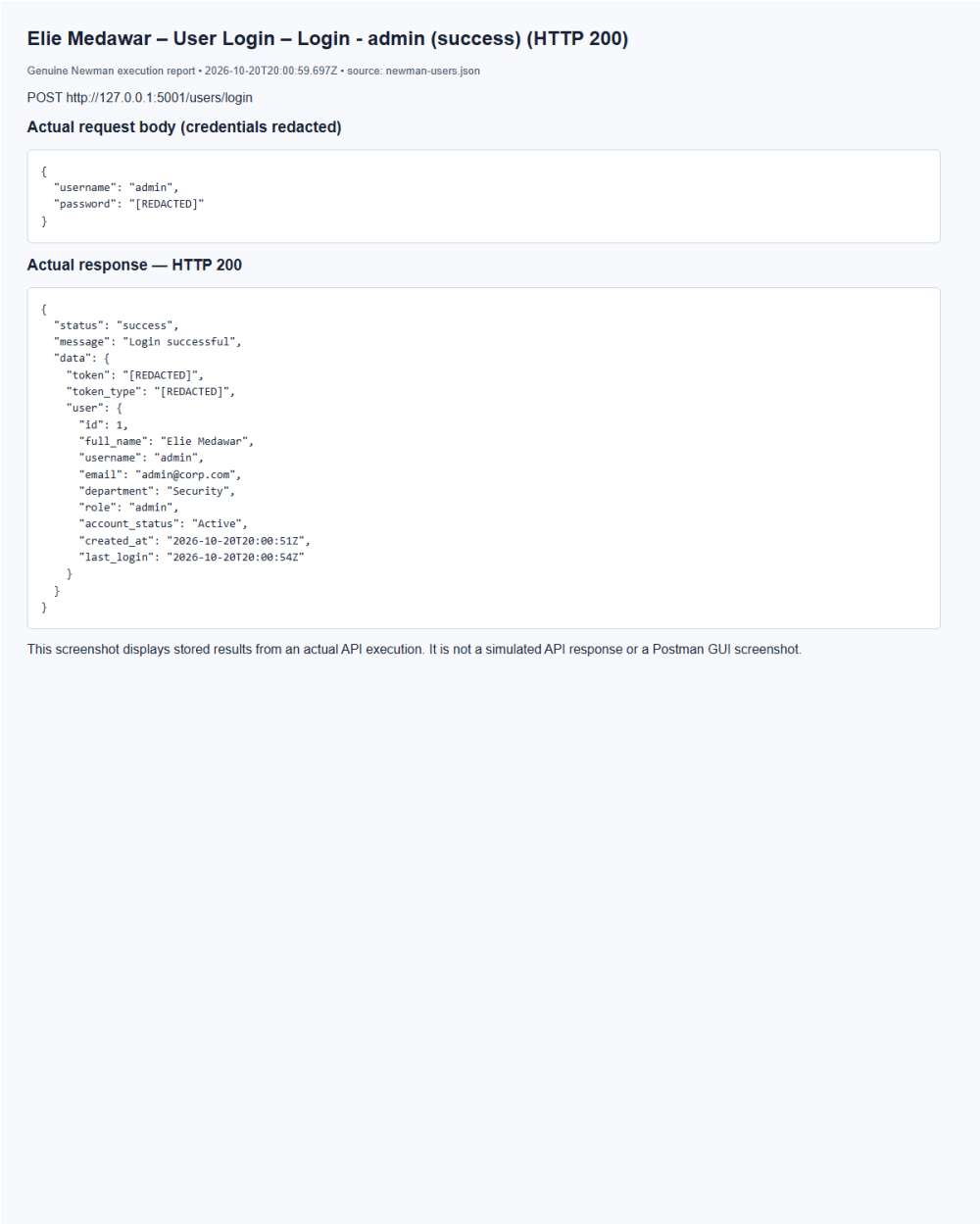
(Output shortened; the full capture is report/screenshots/elie/newman-users-04.png in the submission.)

Elie Medawar – User Registration – Register - duplicate username (HTTP 409)



(Output shortened; the full capture is report/screenshots/elie/newman-users-08.png in the submission.)

Elie Medawar – User Login – Login - admin (success) (HTTP 200)



Elie Medawar – User Login – Login - admin (success) (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:00:59.697Z • source: newman-users.json

POST http://127.0.0.1:5001/users/login

Actual request body (credentials redacted)


```
{
  "username": "admin",
  "password": "[REDACTED]"
}
```

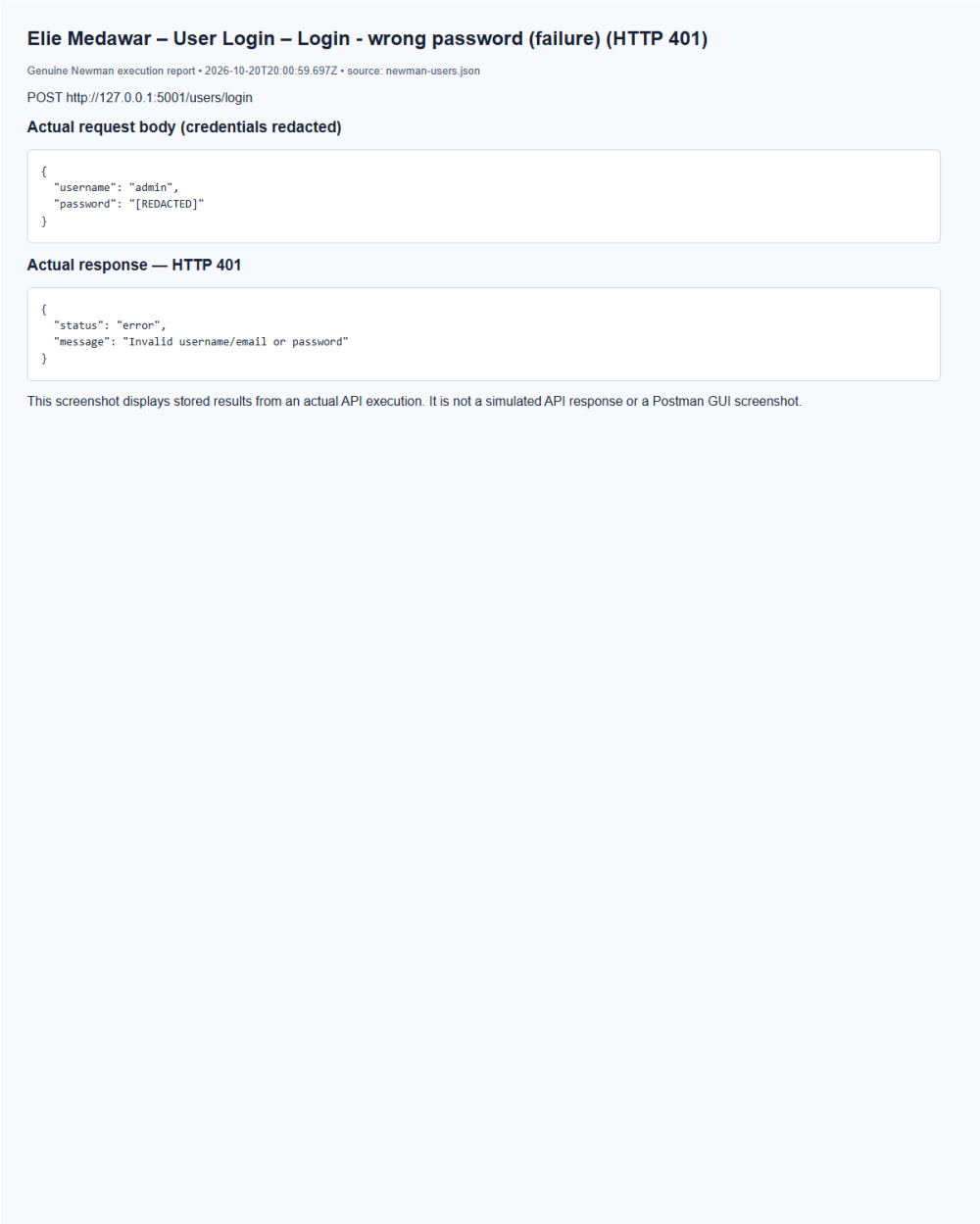

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Login successful",
  "data": {
    "token": "[REDACTED]",
    "token_type": "[REDACTED]",
    "user": {
      "id": 1,
      "full_name": "Elie Medawar",
      "username": "admin",
      "email": "admin@corp.com",
      "department": "Security",
      "role": "admin",
      "account_status": "Active",
      "created_at": "2026-10-20T20:00:51Z",
      "last_login": "2026-10-20T20:00:54Z"
    }
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

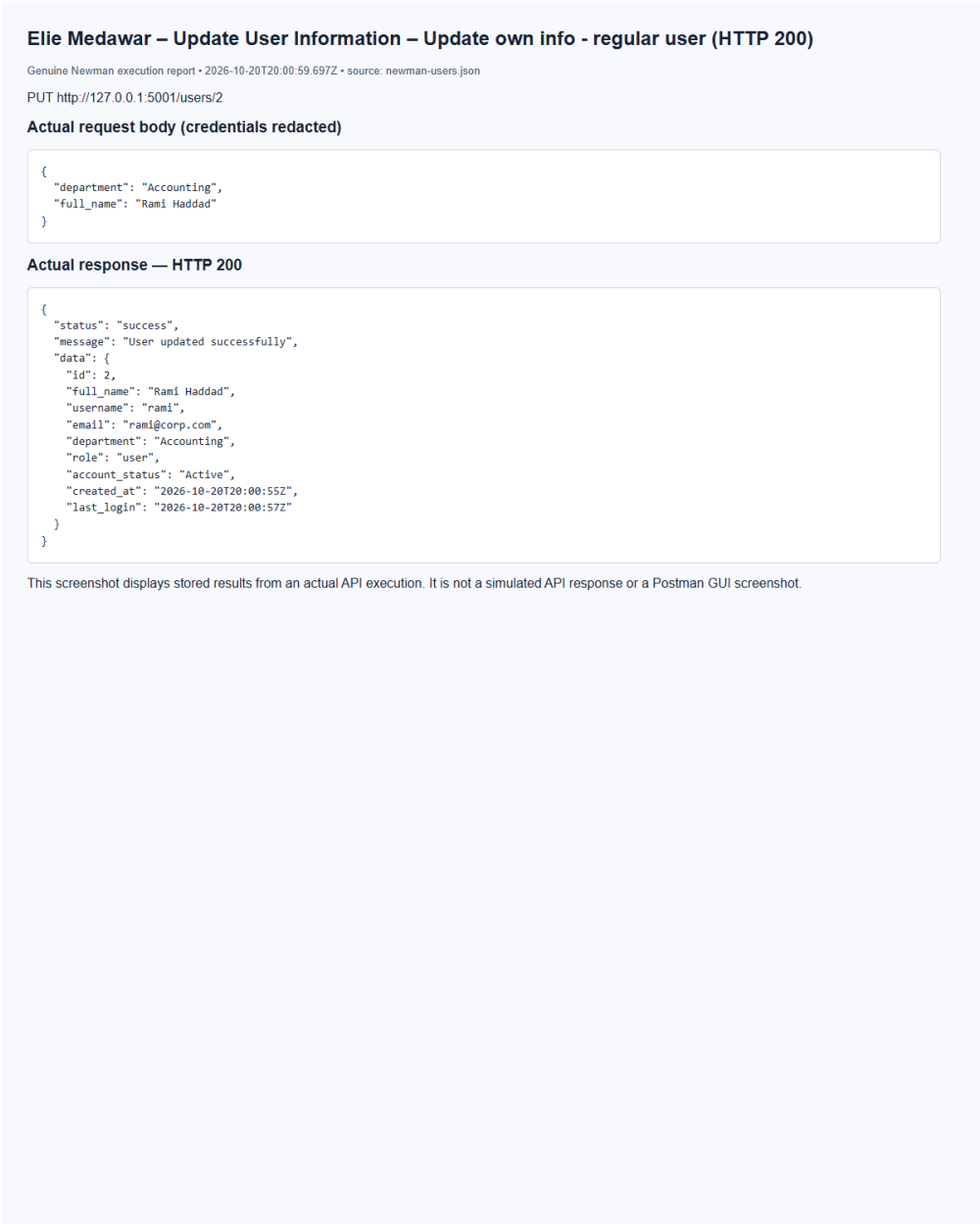
(Output shortened; the full capture is report/screenshots/elie/newman-users-01.png in the submission.)

Elie Medawar – User Login – Login - wrong password (failure) (HTTP 401)



(Output shortened; the full capture is report/screenshots/elie/newman-users-02.png in the submission.)

Elie Medawar – Update User Information – Update own info - regular user (HTTP 200)



Elie Medawar – Update User Information – Update own info - regular user (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:00:59.697Z • source: newman-users.json

PUT http://127.0.0.1:5001/users/2

Actual request body (credentials redacted)


```
{
  "department": "Accounting",
  "full_name": "Rami Haddad"
}
```

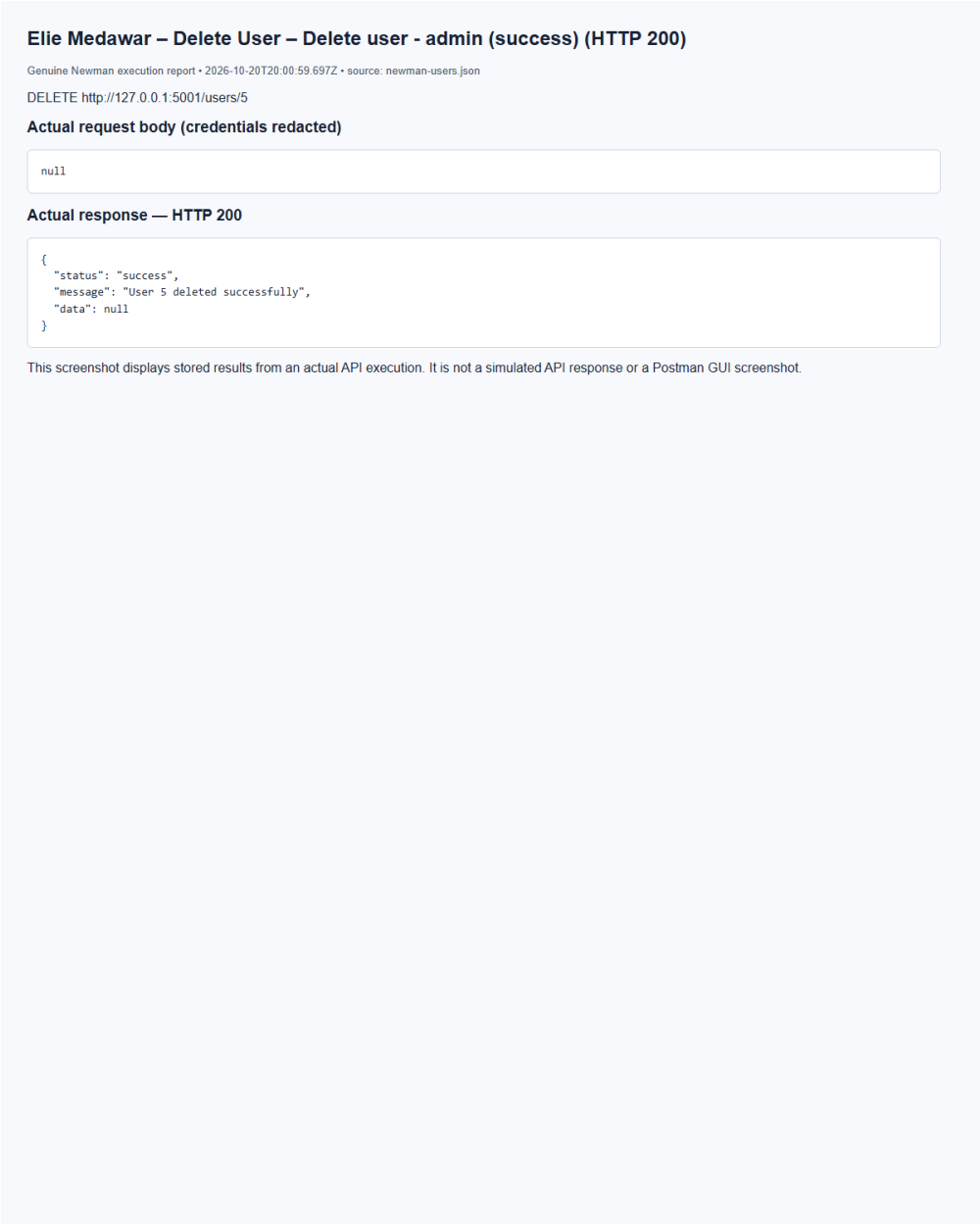

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "User updated successfully",
  "data": {
    "id": 2,
    "full_name": "Rami Haddad",
    "username": "rami",
    "email": "rami@corp.com",
    "department": "Accounting",
    "role": "user",
    "account_status": "Active",
    "created_at": "2026-10-20T20:00:55Z",
    "last_login": "2026-10-20T20:00:57Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

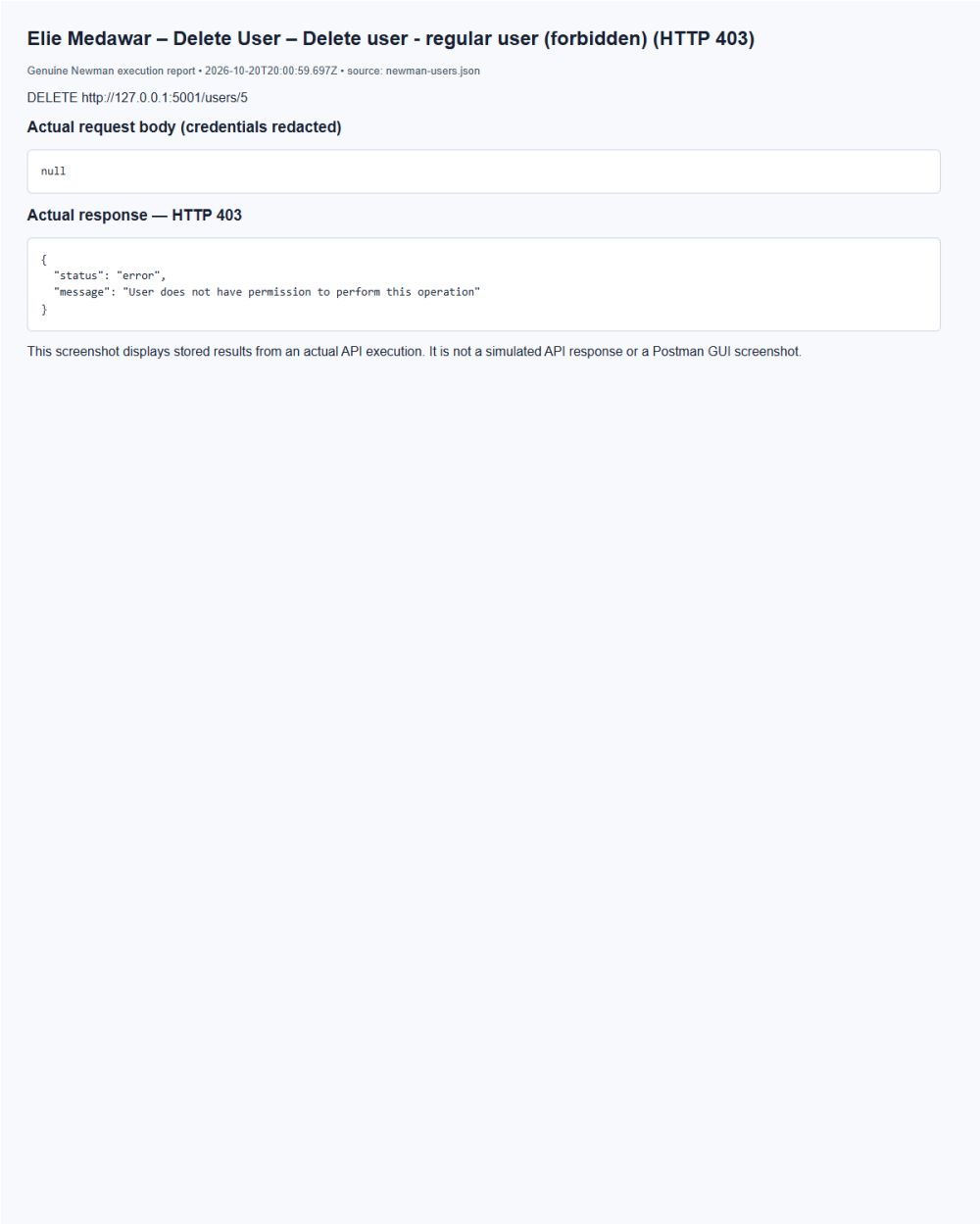
(Output shortened; the full capture is report/screenshots/elie/newman-users-27.png in the submission.)

Elie Medawar – Delete User – Delete user - admin (success) (HTTP 200)



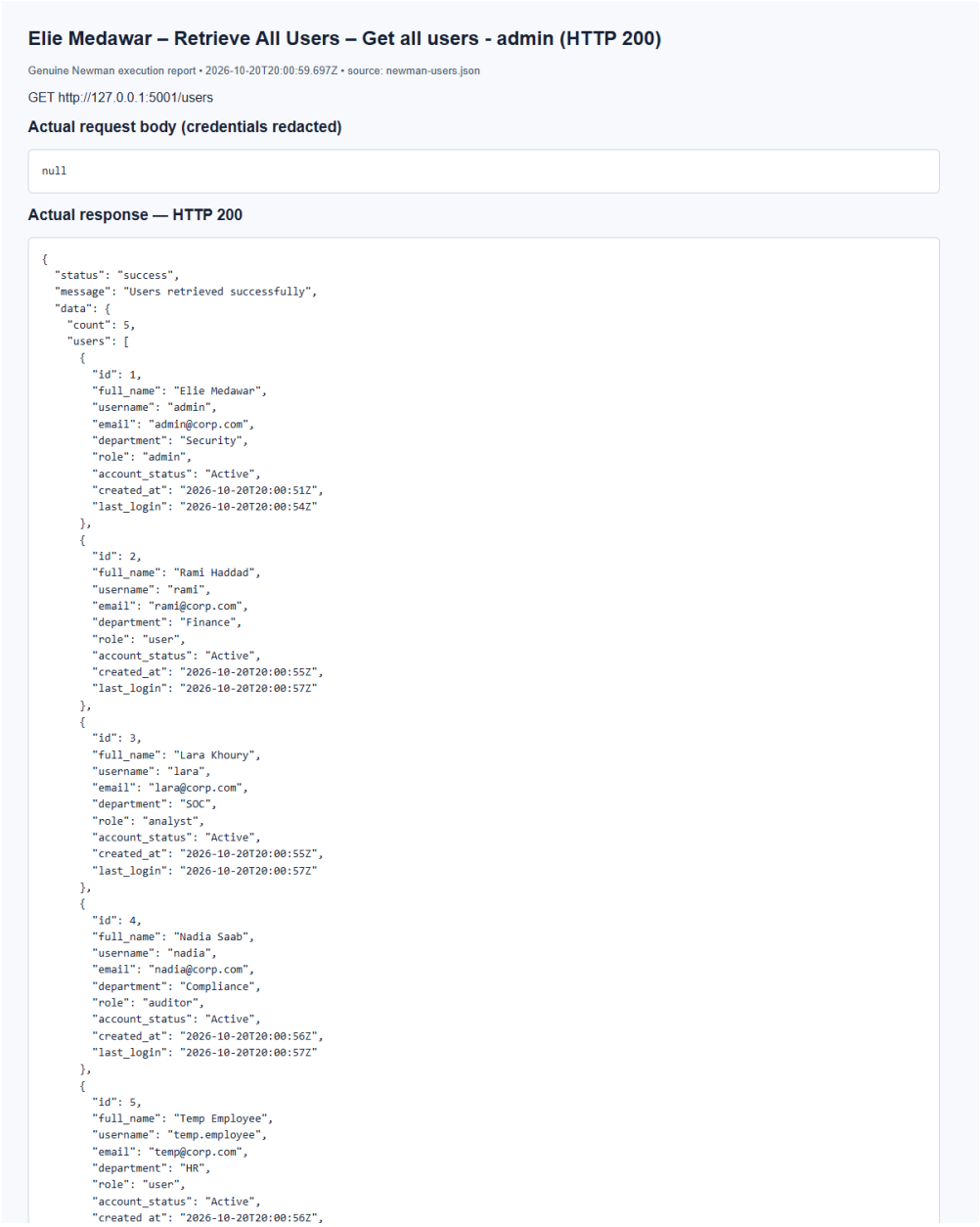
(Output shortened; the full capture is report/screenshots/elie/newman-users-34.png in the submission.)

Elie Medawar – Delete User – Delete user - regular user (forbidden) (HTTP 403)



(Output shortened; the full capture is report/screenshots/elie/newman-users-33.png in the submission.)

Elie Medawar – Retrieve All Users – Get all users - admin (HTTP 200)



Elie Medawar – Retrieve All Users – Get all users - admin (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:00:59.697Z • source: newman-users.json

GET http://127.0.0.1:5001/users

Actual request body (credentials redacted)

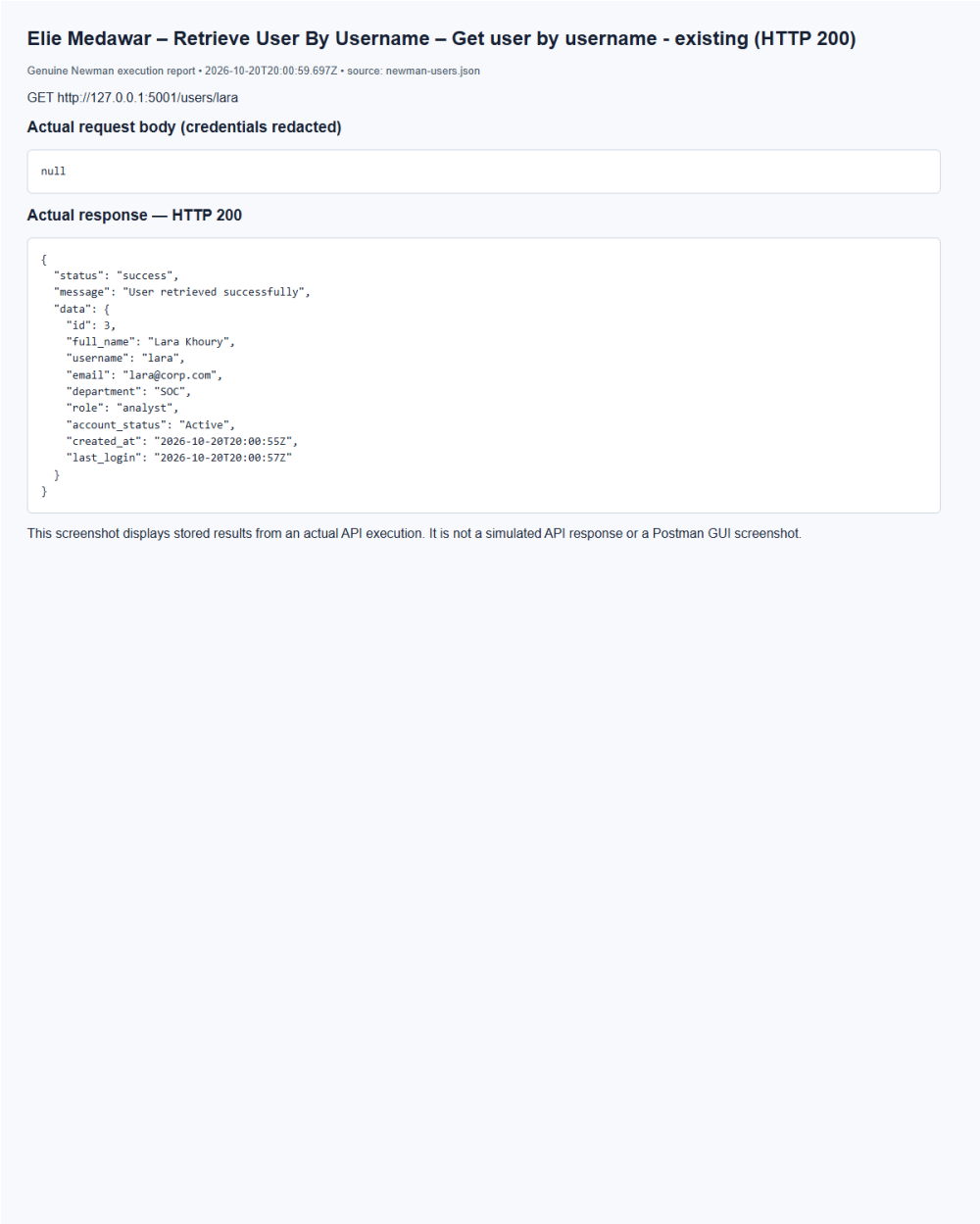
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Users retrieved successfully",
  "data": {
    "count": 5,
    "users": [
      {
        "id": 1,
        "full_name": "Elie Medawar",
        "username": "admin",
        "email": "admin@corp.com",
        "department": "Security",
        "role": "admin",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:51Z",
        "last_login": "2026-10-20T20:00:54Z"
      },
      {
        "id": 2,
        "full_name": "Rami Haddad",
        "username": "rami",
        "email": "rami@corp.com",
        "department": "Finance",
        "role": "user",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:55Z",
        "last_login": "2026-10-20T20:00:57Z"
      },
      {
        "id": 3,
        "full_name": "Lara Khoury",
        "username": "lara",
        "email": "lara@corp.com",
        "department": "SOC",
        "role": "analyst",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:55Z",
        "last_login": "2026-10-20T20:00:57Z"
      },
      {
        "id": 4,
        "full_name": "Nadia Saab",
        "username": "nadia",
        "email": "nadia@corp.com",
        "department": "Compliance",
        "role": "auditor",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:56Z",
        "last_login": "2026-10-20T20:00:57Z"
      },
      {
        "id": 5,
        "full_name": "Temp Employee",
        "username": "temp.employee",
        "email": "temp@corp.com",
        "department": "HR",
        "role": "user",
        "account_status": "Active",
        "created at": "2026-10-20T20:00:56Z",
```


(Output shortened; the full capture is report/screenshots/elie/newman-users-17.png in the submission.)

Elie Medawar – Retrieve User By Username – Get user by username - existing (HTTP 200)



Elie Medawar – Retrieve User By Username – Get user by username - existing (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:00:59.697Z • source: newman-users.json

GET http://127.0.0.1:5001/users/lara

Actual request body (credentials redacted)

```
null
```

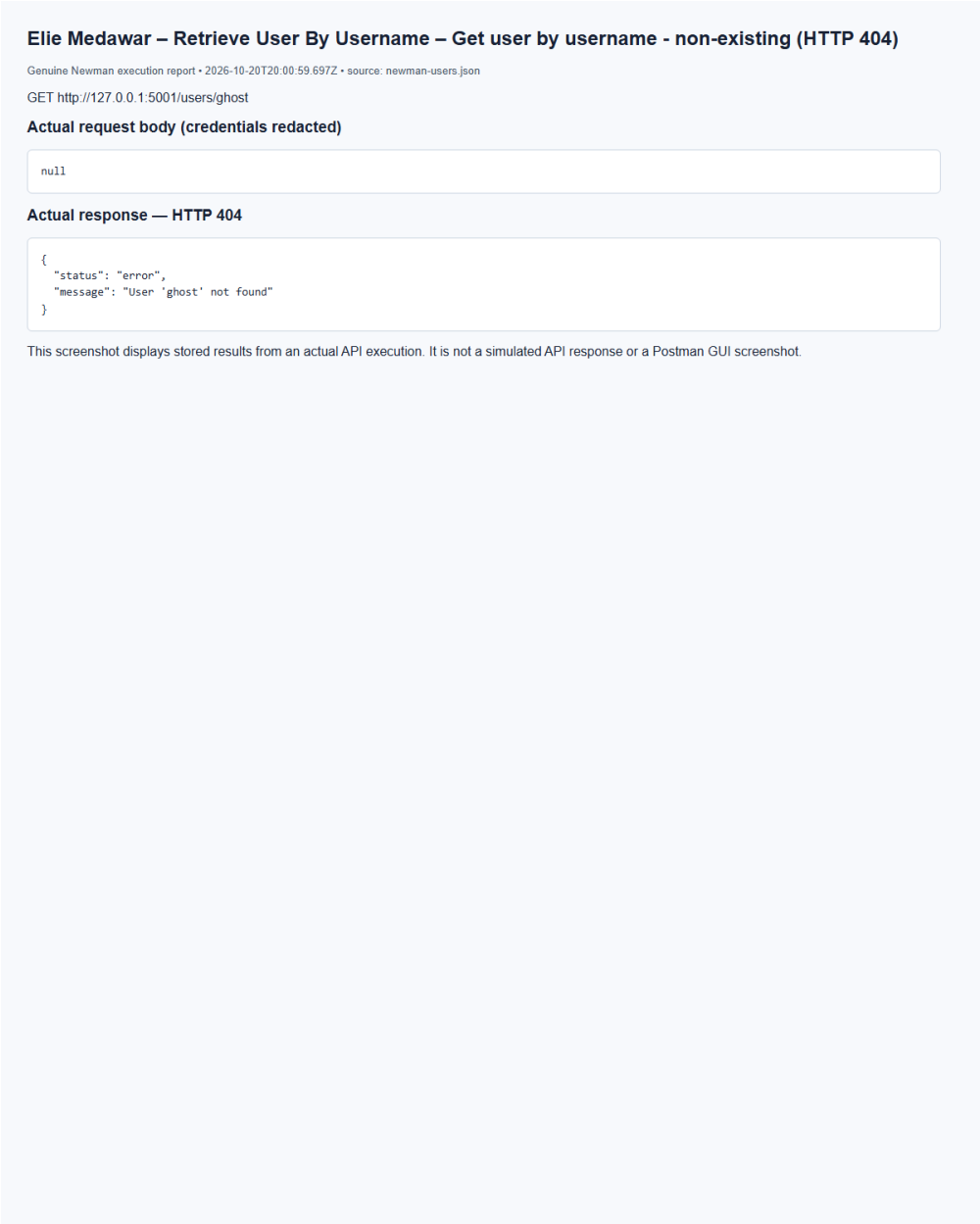
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "User retrieved successfully",
  "data": {
    "id": 3,
    "full_name": "Lara Khoury",
    "username": "lara",
    "email": "lara@corp.com",
    "department": "SOC",
    "role": "analyst",
    "account_status": "Active",
    "created_at": "2026-10-20T20:00:55Z",
    "last_login": "2026-10-20T20:00:57Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-users-23.png in the submission.)

Elie Medawar – Retrieve User By Username – Get user by username - non-existing (HTTP 404)



(Output shortened; the full capture is report/screenshots/elie/newman-users-24.png in the submission.)

5.2 Asset Management Service (Marc Bteich)

AssetValidator validates required strings, type, IP, owner ID, criticality and status. AssetService normalizes IPv4/IPv6, verifies the owner over HTTP and performs ORM writes and queries. Admins write; admins/auditors list; analysts can retrieve/search. Search filters combine with AND and escape LIKE wildcards. Incidents survive asset deletion with a null asset link; existing vulnerabilities block deletion with 409. The original Assets behavior is retained.

Marc Bteich – Create asset – POST /assets, HTTP 201



Marc Bteich – Create asset – POST /assets, HTTP 201

Genuine Newman execution report • 2026-10-20T12:07:40.066Z • source: newman-assets.json

POST http://127.0.0.1:5002/assets

Actual request body (credentials redacted)

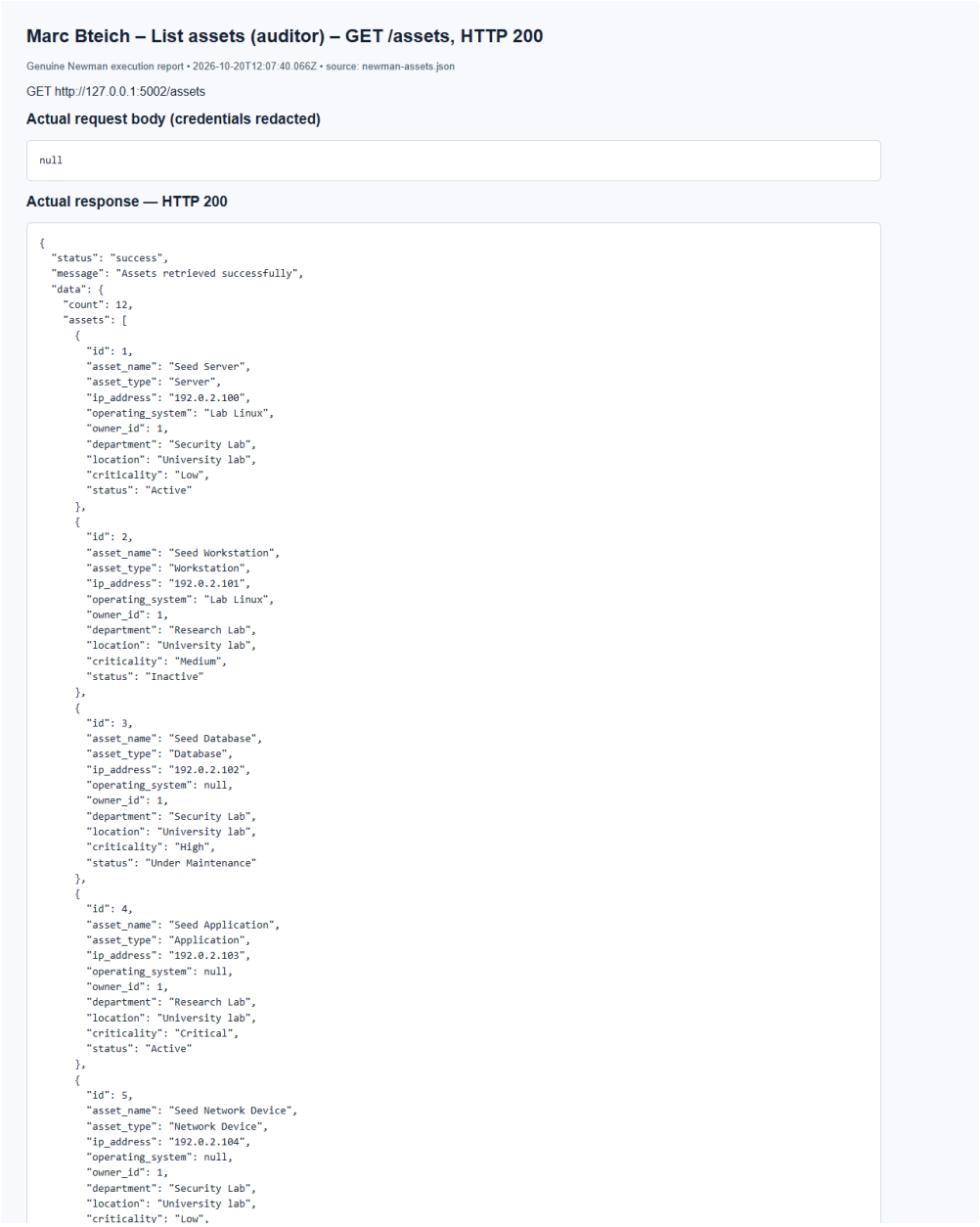

```
{
  "asset_name": "Postman Lab Server",
  "asset_type": "Server",
  "ip_address": "192.0.2.10",
  "operating_system": "Linux",
  "department": "Security",
  "owner_id": 1,
  "location": "Lab",
  "criticality": "Critical",
  "status": "Active"
}
```


Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Asset created successfully",
  "data": {
    "id": 7,
    "asset_name": "Postman Lab Server",
    "asset_type": "Server",
    "ip_address": "192.0.2.10",
    "operating_system": "Linux",
    "owner_id": 1,
    "department": "Security",
    "location": "Lab",
    "criticality": "Critical",
    "status": "Active"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – List assets (auditor) – GET /assets, HTTP 200



Marc Bteich – List assets (auditor) – GET /assets, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:40.066Z • source: newman-assets.json

GET http://127.0.0.1:5002/assets

Actual request body (credentials redacted)

```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Assets retrieved successfully",
  "data": {
    "count": 12,
    "assets": [
      {
        "id": 1,
        "asset_name": "Seed Server",
        "asset_type": "Server",
        "ip_address": "192.0.2.100",
        "operating_system": "Lab Linux",
        "owner_id": 1,
        "department": "Security Lab",
        "location": "University lab",
        "criticality": "Low",
        "status": "Active"
      },
      {
        "id": 2,
        "asset_name": "Seed Workstation",
        "asset_type": "Workstation",
        "ip_address": "192.0.2.101",
        "operating_system": "Lab Linux",
        "owner_id": 1,
        "department": "Research Lab",
        "location": "University lab",
        "criticality": "Medium",
        "status": "Inactive"
      },
      {
        "id": 3,
        "asset_name": "Seed Database",
        "asset_type": "Database",
        "ip_address": "192.0.2.102",
        "operating_system": null,
        "owner_id": 1,
        "department": "Security Lab",
        "location": "University lab",
        "criticality": "High",
        "status": "Under Maintenance"
      },
      {
        "id": 4,
        "asset_name": "Seed Application",
        "asset_type": "Application",
        "ip_address": "192.0.2.103",
        "operating_system": null,
        "owner_id": 1,
        "department": "Research Lab",
        "location": "University lab",
        "criticality": "Critical",
        "status": "Active"
      },
      {
        "id": 5,
        "asset_name": "Seed Network Device",
        "asset_type": "Network Device",
        "ip_address": "192.0.2.104",
        "operating_system": null,
        "owner_id": 1,
        "department": "Security Lab",
        "location": "University lab",
        "criticality": "Low",
```


(Output shortened; the full capture is report/screenshots/marc/newman-assets-11.png in the submission.)

Marc Bteich – Update asset (admin) – PUT /assets/7, HTTP 200

Marc Bteich – Update asset (admin) – PUT /assets/7, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:40.066Z • source: newman-assets.json

PUT http://127.0.0.1:5002/assets/7

Actual request body (credentials redacted)


```
{
  "asset_name": "Updated Postman Server",
  "department": "IT",
  "criticality": "High",
  "status": "Under Maintenance"
}
```


Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Asset updated successfully",
  "data": {
    "id": 7,
    "asset_name": "Updated Postman Server",
    "asset_type": "Server",
    "ip_address": "192.0.2.10",
    "operating_system": "Linux",
    "owner_id": 1,
    "department": "IT",
    "location": "Lab",
    "criticality": "High",
    "status": "Under Maintenance"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Missing token – GET /assets, HTTP 401

Marc Bteich – Missing token – GET /assets, HTTP 401

Genuine Newman execution report • 2026-10-20T12:07:40.066Z • source: newman-assets.json

GET http://127.0.0.1:5002/assets

Actual request body (credentials redacted)

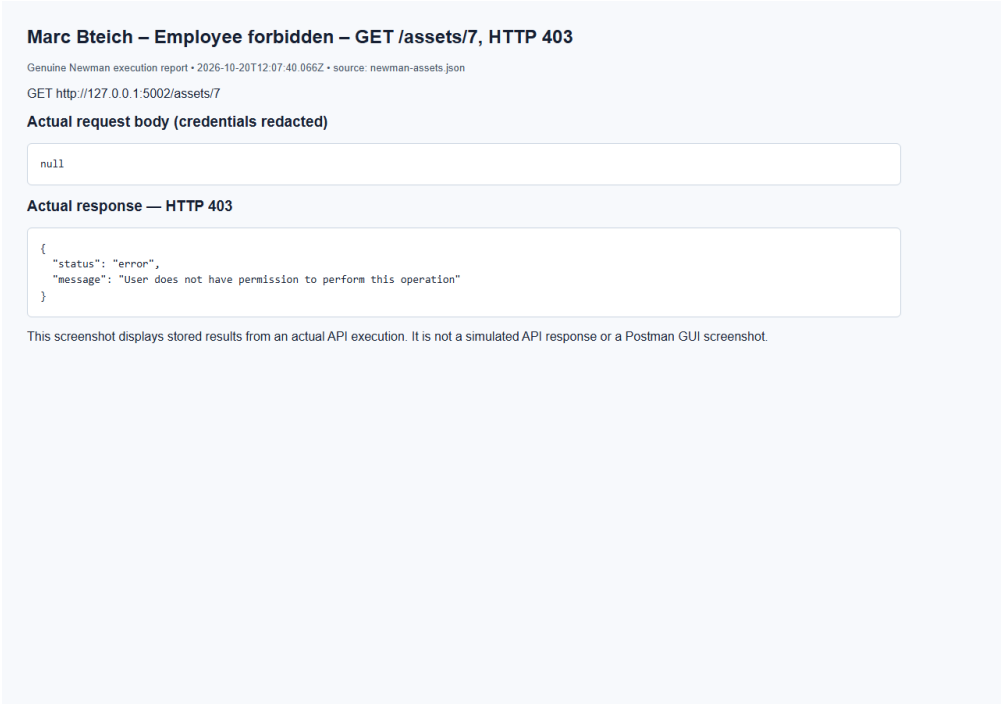

```
null
```


Actual response — HTTP 401


```
{
  "status": "error",
  "message": "Missing authentication token (use 'Authorization: Bearer <token>')"
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Employee forbidden – GET /assets/7, HTTP 403



Marc Bteich – Employee forbidden – GET /assets/7, HTTP 403

Genuine Newman execution report • 2026-10-20T12:07:40.066Z • source: newman-assets.json

GET http://127.0.0.1:5002/assets/7

Actual request body (credentials redacted)

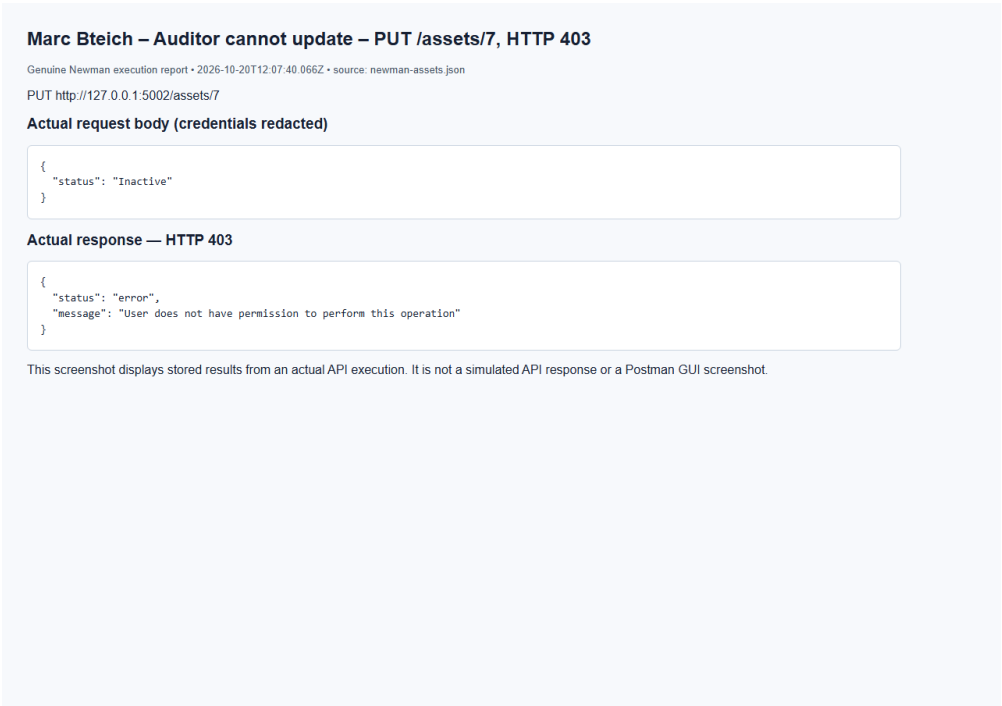
```
null
```

Actual response — HTTP 403

```
{
  "status": "error",
  "message": "User does not have permission to perform this operation"
}
```

This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Auditor cannot update – PUT /assets/7, HTTP 403



Marc Bteich – Auditor cannot update – PUT /assets/7, HTTP 403

Genuine Newman execution report • 2026-10-20T12:07:40.066Z • source: newman-assets.json

PUT http://127.0.0.1:5002/assets/7

Actual request body (credentials redacted)

```
{
  "status": "Inactive"
}
```

Actual response — HTTP 403

```
{
  "status": "error",
  "message": "User does not have permission to perform this operation"
}
```

This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

5.3 Incident Management Service (Elie Medawar)

The Incidents service (port 5003) manages an incident from the moment it is reported until it is closed: assignment to an analyst, the enforced status lifecycle, and investigation notes. Code: `services/incidents/` (`IncidentService`, `IncidentWorkflow`) and `models/incidents.py`.

API	Method	Endpoint	Allowed roles	Success
Create Incident	POST	/incidents	admin, user	201
Retrieve All Incidents	GET	/incidents (optional ?status=, ?severity=)	admin, auditor	200
Retrieve Specific Incident	GET	/incidents/{id}	admin, auditor; analyst (assigned only); user (own reports only)	200
Assign Incident to Analyst	PUT	/incidents/{id}/assign	admin	200
Update Incident Status	PUT	/incidents/{id}/status	admin; analyst (assigned only)	200
Add Investigation Notes	POST	/incidents/{id}/notes	admin; analyst (assigned only)	201

The reporter is always the logged-in user taken from the token; a reporter_id in the body is rejected. Allowed categories: Malware, Unauthorized Access, Phishing, Data Leakage, Account Compromise, Denial of Service, Other.

Severity	Example incident in the evidence	Why this level
Low	Port scan detected and blocked by the firewall	No impact, logged for awareness
Medium	Employee clicked a phishing link	Limited impact but needs investigation
High	Brute force on the HR database	Targets an important system holding personal data
Critical	Ransomware on the web server	Service down, immediate response required

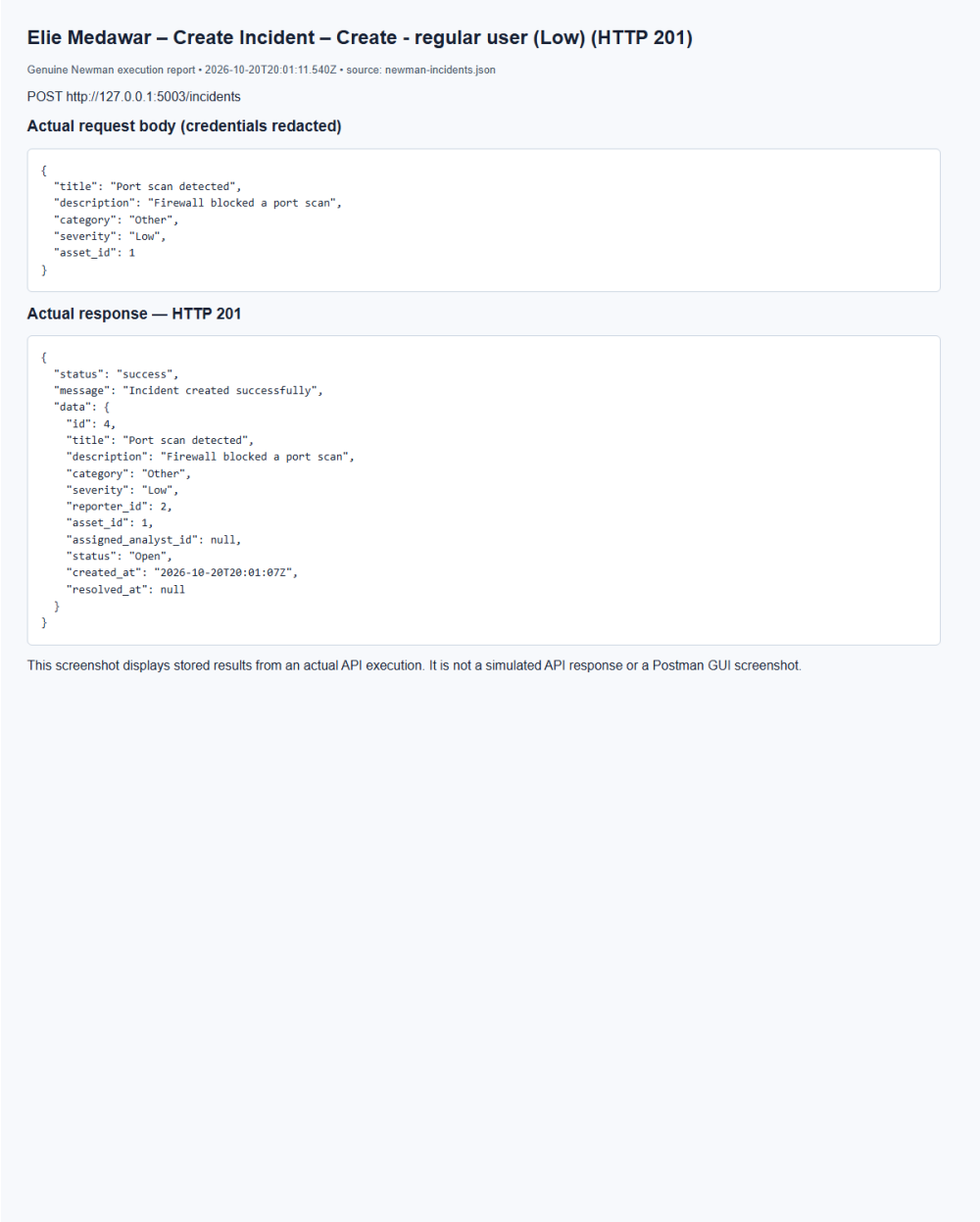
Lifecycle (IncidentWorkflow). Each status may only move to the next one: Open → Under Investigation (only once an analyst is assigned) → Mitigated → Resolved (sets resolved_at) → Closed (final: no status change, reassignment or new note). Skipping a step, going backwards or an unknown status returns 400 with the next allowed status in the message.

Communication with other services. Creating an incident asks the Asset Management Service whether the asset exists (GET /internal/assets/{id}); an unknown asset returns 400 and an unreachable service 503. Assigning asks the Users service for the user and checks the role is analyst and the account Active. Every create, assignment, status change and note is written to the Audit Logging Service. The analyst receives an Incident Assignment notification and the reporter receives a notification at each status change.

Database interaction. incidents: id (PK), title, description, category, severity, reporter_id (FK → users), asset_id (FK → assets, ON DELETE SET NULL), assigned_analyst_id (FK → users, ON

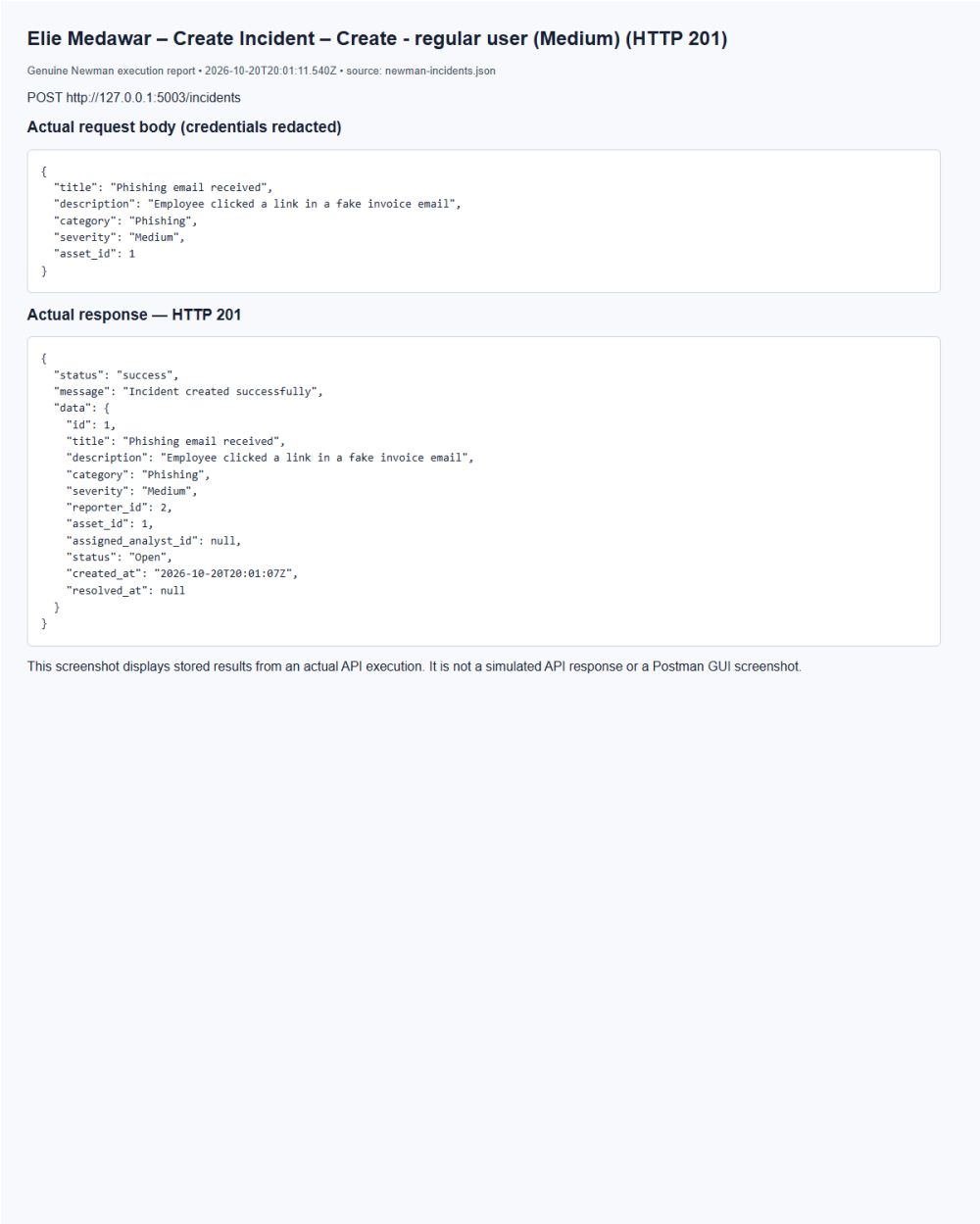
DELETE SET NULL), status, created_at, resolved_at. investigation_notes: id (PK), incident_id (FK → incidents, ON DELETE CASCADE), analyst_id (FK → users), content, created_at. One incident has many notes; each note belongs to one incident and is returned, oldest first, inside GET /incidents/{id}.

Elie Medawar – Create Incident – Create - regular user (Low) (HTTP 201)



(Output shortened; the full capture is report/screenshots/elie/newman-incidents-08.png in the submission.)

Elie Medawar – Create Incident – Create - regular user (Medium) (HTTP 201)



Elie Medawar – Create Incident – Create - regular user (Medium) (HTTP 201)

Genuine Newman execution report • 2026-10-20T20:01:11.540Z • source: newman-incidents.json

POST http://127.0.0.1:5003/incidents

Actual request body (credentials redacted)


```
{
  "title": "Phishing email received",
  "description": "Employee clicked a link in a fake invoice email",
  "category": "Phishing",
  "severity": "Medium",
  "asset_id": 1
}
```

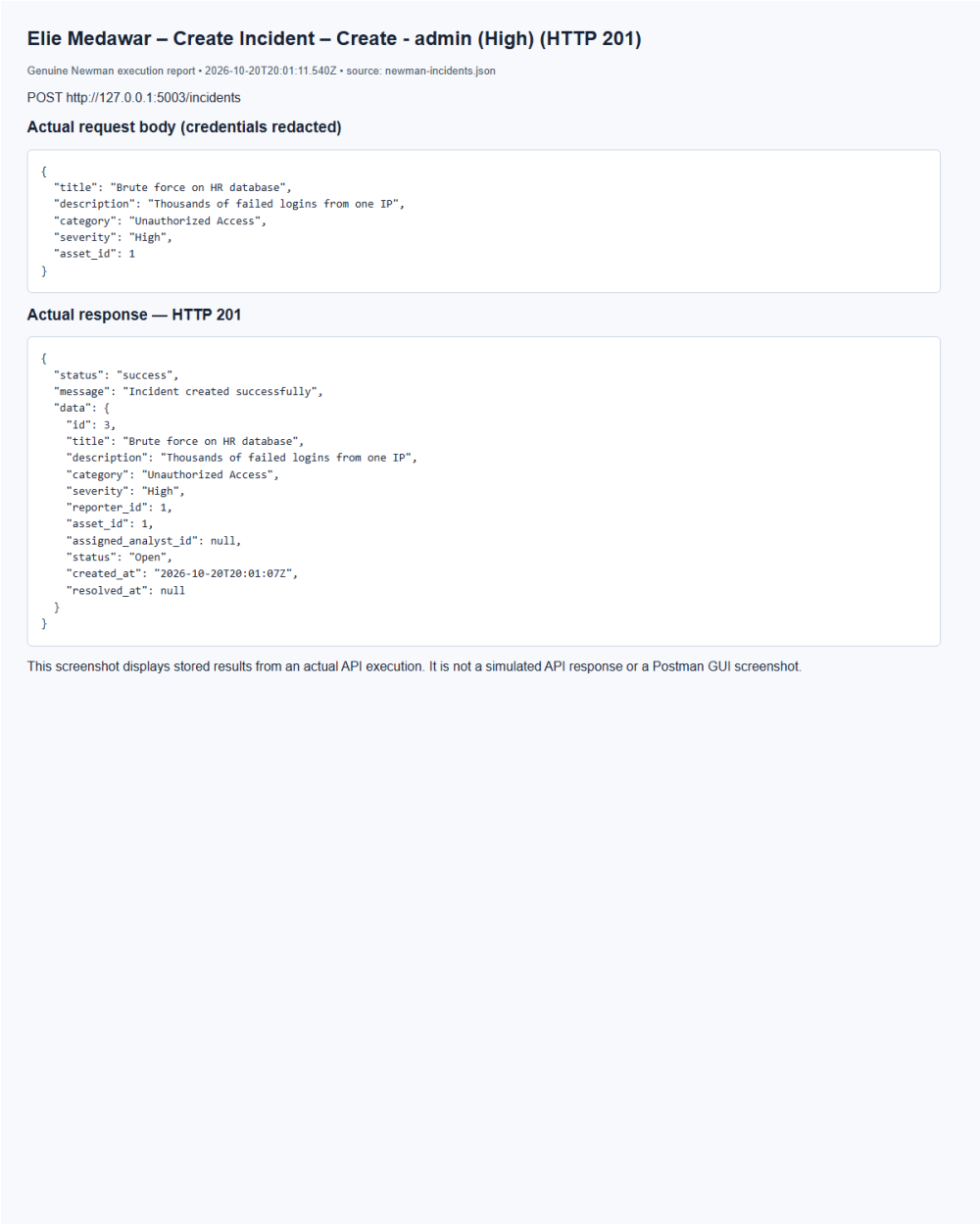

Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Incident created successfully",
  "data": {
    "id": 1,
    "title": "Phishing email received",
    "description": "Employee clicked a link in a fake invoice email",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 1,
    "assigned_analyst_id": null,
    "status": "Open",
    "created_at": "2026-10-20T20:01:07Z",
    "resolved_at": null
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

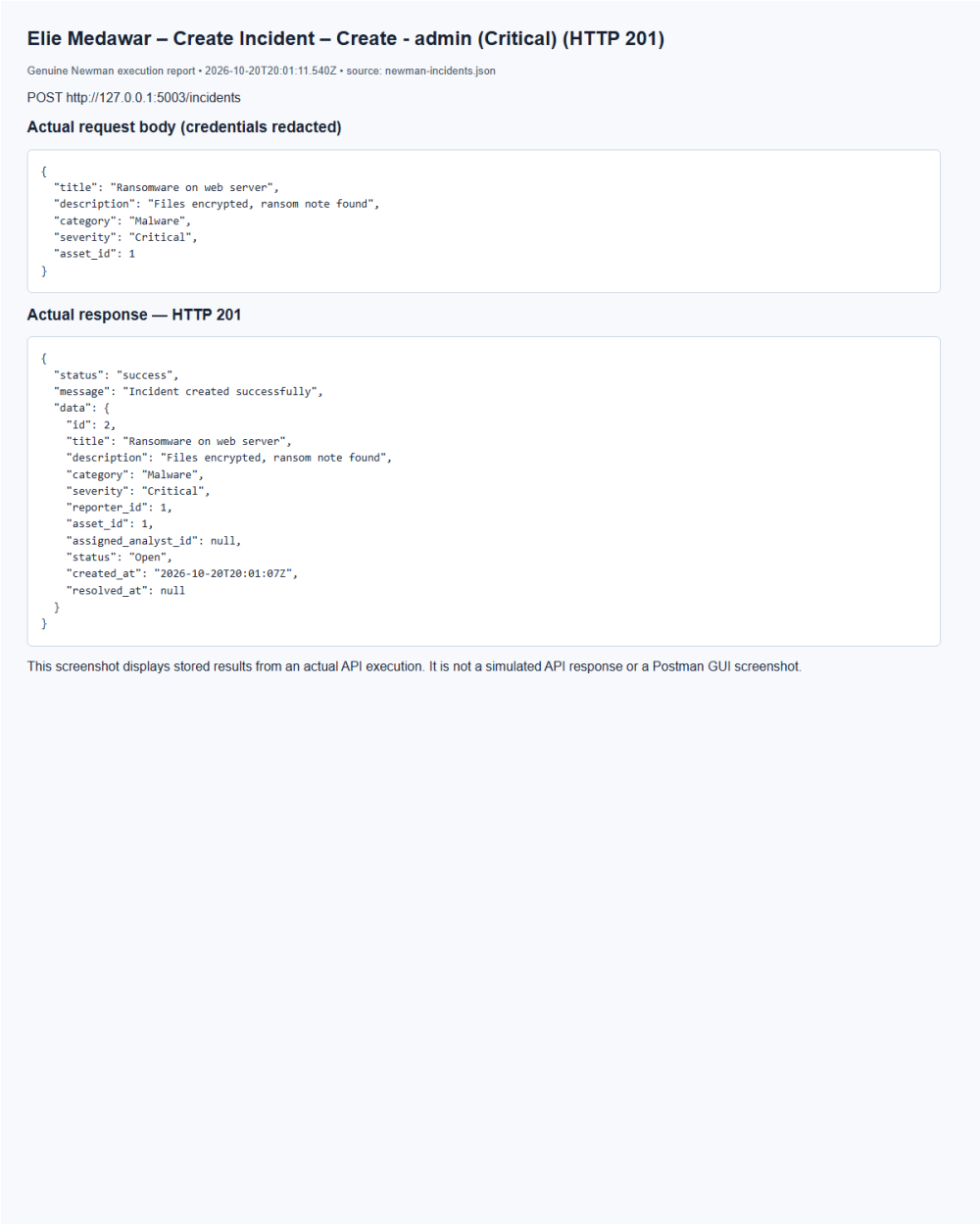
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-05.png in the submission.)

Elie Medawar – Create Incident – Create - admin (High) (HTTP 201)



(Output shortened; the full capture is report/screenshots/elie/newman-incidents-07.png in the submission.)

Elie Medawar – Create Incident – Create - admin (Critical) (HTTP 201)



Elie Medawar – Create Incident – Create - admin (Critical) (HTTP 201)

Genuine Newman execution report • 2026-10-20T20:01:11.540Z • source: newman-incidents.json

POST http://127.0.0.1:5003/incidents

Actual request body (credentials redacted)


```
{
  "title": "Ransomware on web server",
  "description": "Files encrypted, ransom note found",
  "category": "Malware",
  "severity": "Critical",
  "asset_id": 1
}
```

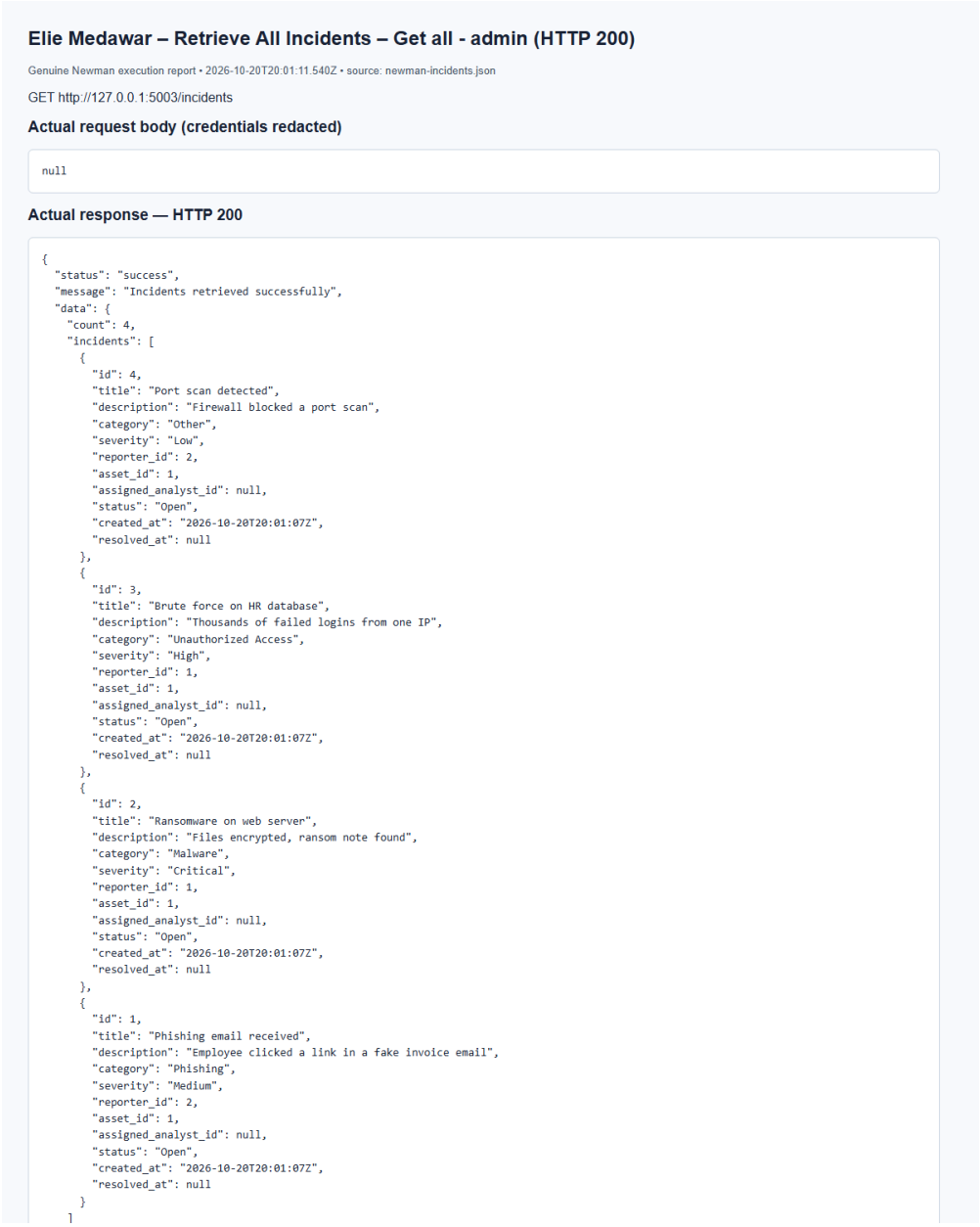

Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Incident created successfully",
  "data": {
    "id": 2,
    "title": "Ransomware on web server",
    "description": "Files encrypted, ransom note found",
    "category": "Malware",
    "severity": "Critical",
    "reporter_id": 1,
    "asset_id": 1,
    "assigned_analyst_id": null,
    "status": "Open",
    "created_at": "2026-10-20T20:01:07Z",
    "resolved_at": null
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-incidents-06.png in the submission.)

Elie Medawar – Retrieve All Incidents – Get all - admin (HTTP 200)



Elie Medawar – Retrieve All Incidents – Get all - admin (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:11.540Z • source: newman-incidents.json

GET http://127.0.0.1:5003/incidents

Actual request body (credentials redacted)

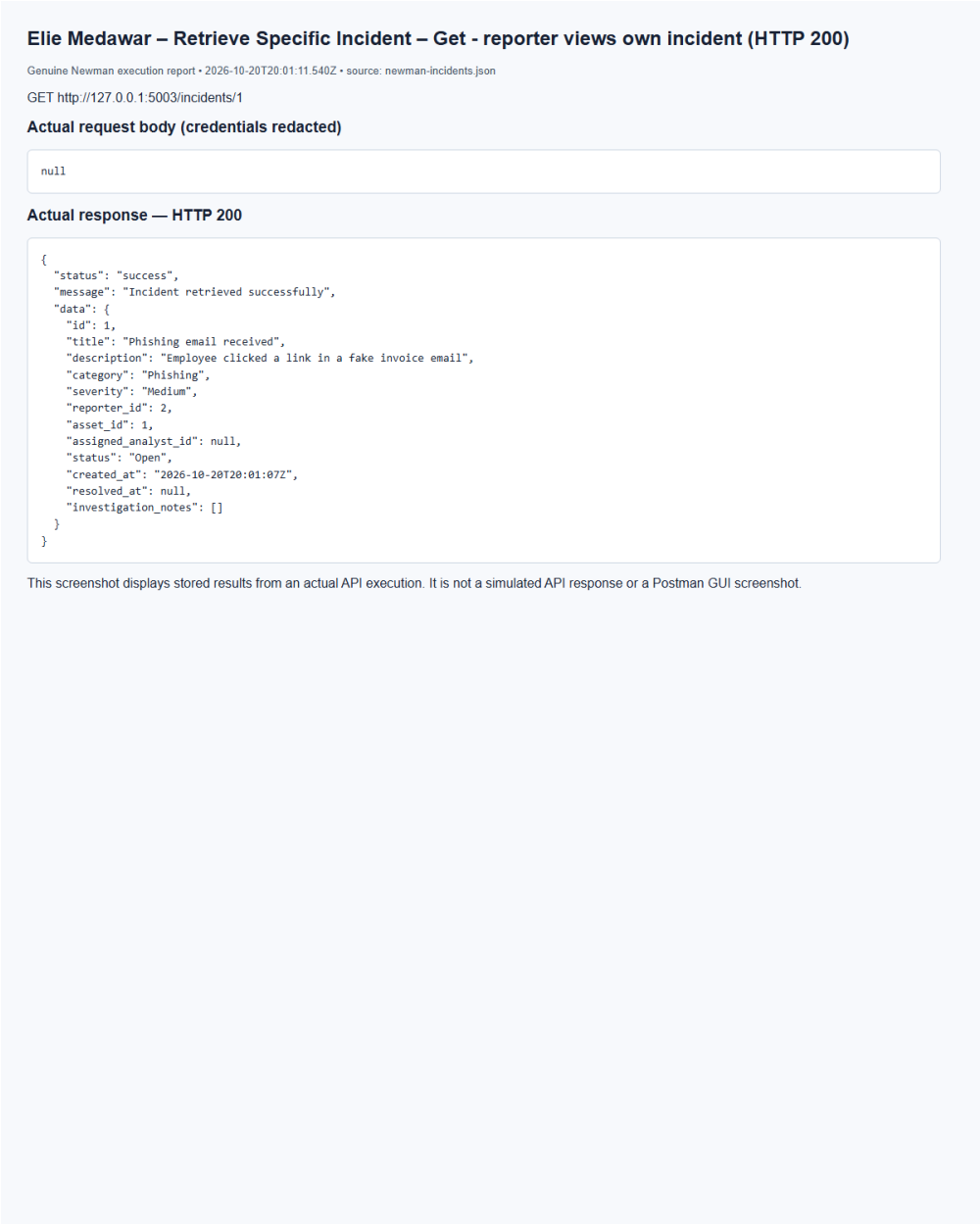
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Incidents retrieved successfully",
  "data": {
    "count": 4,
    "incidents": [
      {
        "id": 4,
        "title": "Port scan detected",
        "description": "Firewall blocked a port scan",
        "category": "Other",
        "severity": "Low",
        "reporter_id": 2,
        "asset_id": 1,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T20:01:07Z",
        "resolved_at": null
      },
      {
        "id": 3,
        "title": "Brute force on HR database",
        "description": "Thousands of failed logins from one IP",
        "category": "Unauthorized Access",
        "severity": "High",
        "reporter_id": 1,
        "asset_id": 1,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T20:01:07Z",
        "resolved_at": null
      },
      {
        "id": 2,
        "title": "Ransomware on web server",
        "description": "Files encrypted, ransom note found",
        "category": "Malware",
        "severity": "Critical",
        "reporter_id": 1,
        "asset_id": 1,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T20:01:07Z",
        "resolved_at": null
      },
      {
        "id": 1,
        "title": "Phishing email received",
        "description": "Employee clicked a link in a fake invoice email",
        "category": "Phishing",
        "severity": "Medium",
        "reporter_id": 2,
        "asset_id": 1,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T20:01:07Z",
        "resolved_at": null
      }
    ]
```


(Output shortened; the full capture is report/screenshots/elie/newman-incidents-15.png in the submission.)

Elie Medawar – Retrieve Specific Incident – Get - reporter views own incident (HTTP 200)



Elie Medawar – Retrieve Specific Incident – Get - reporter views own incident (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:11.540Z • source: newman-incidents.json

GET http://127.0.0.1:5003/incidents/1

Actual request body (credentials redacted)

```
null
```

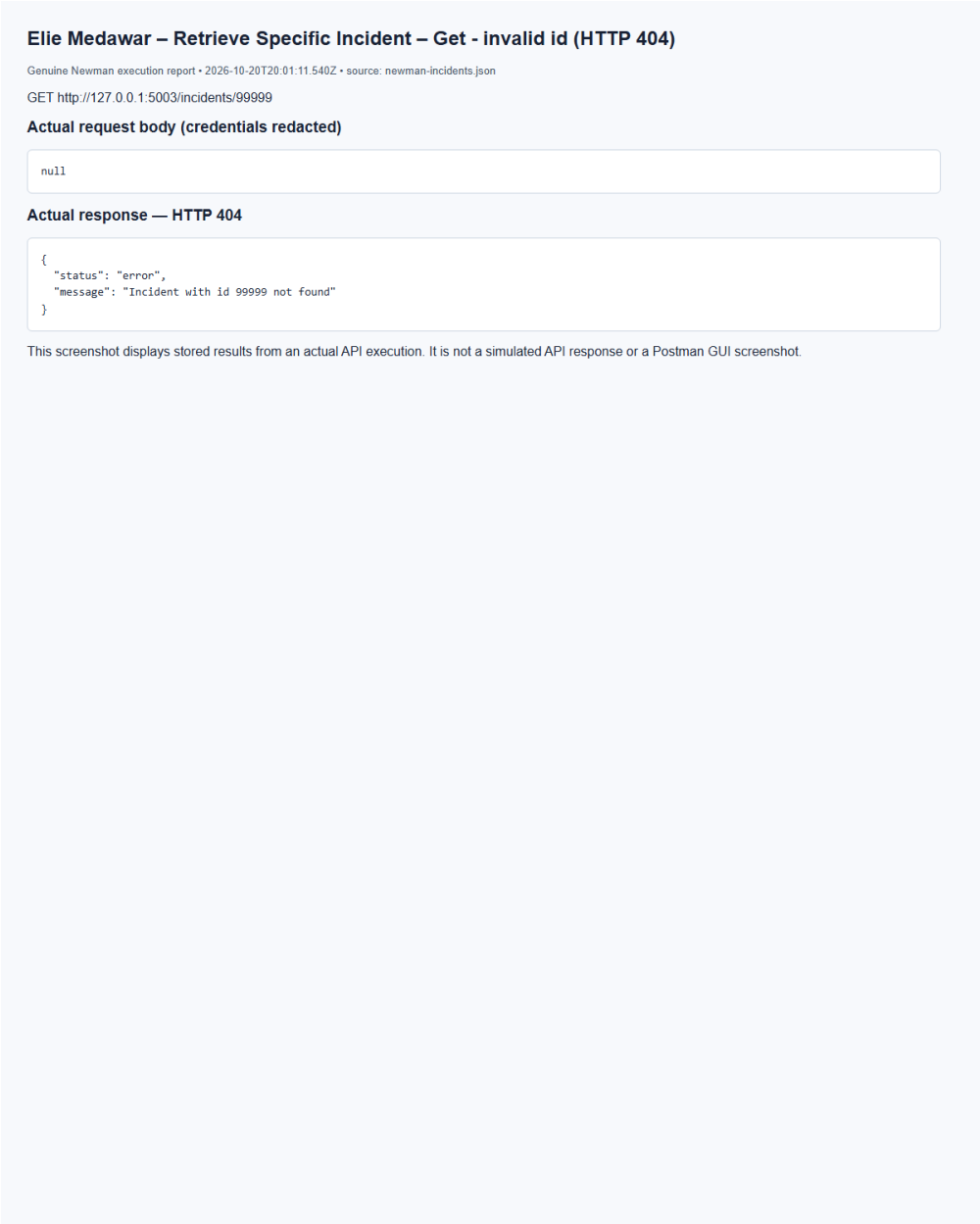
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Incident retrieved successfully",
  "data": {
    "id": 1,
    "title": "Phishing email received",
    "description": "Employee clicked a link in a fake invoice email",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 1,
    "assigned_analyst_id": null,
    "status": "Open",
    "created_at": "2026-10-20T20:01:07Z",
    "resolved_at": null,
    "investigation_notes": []
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

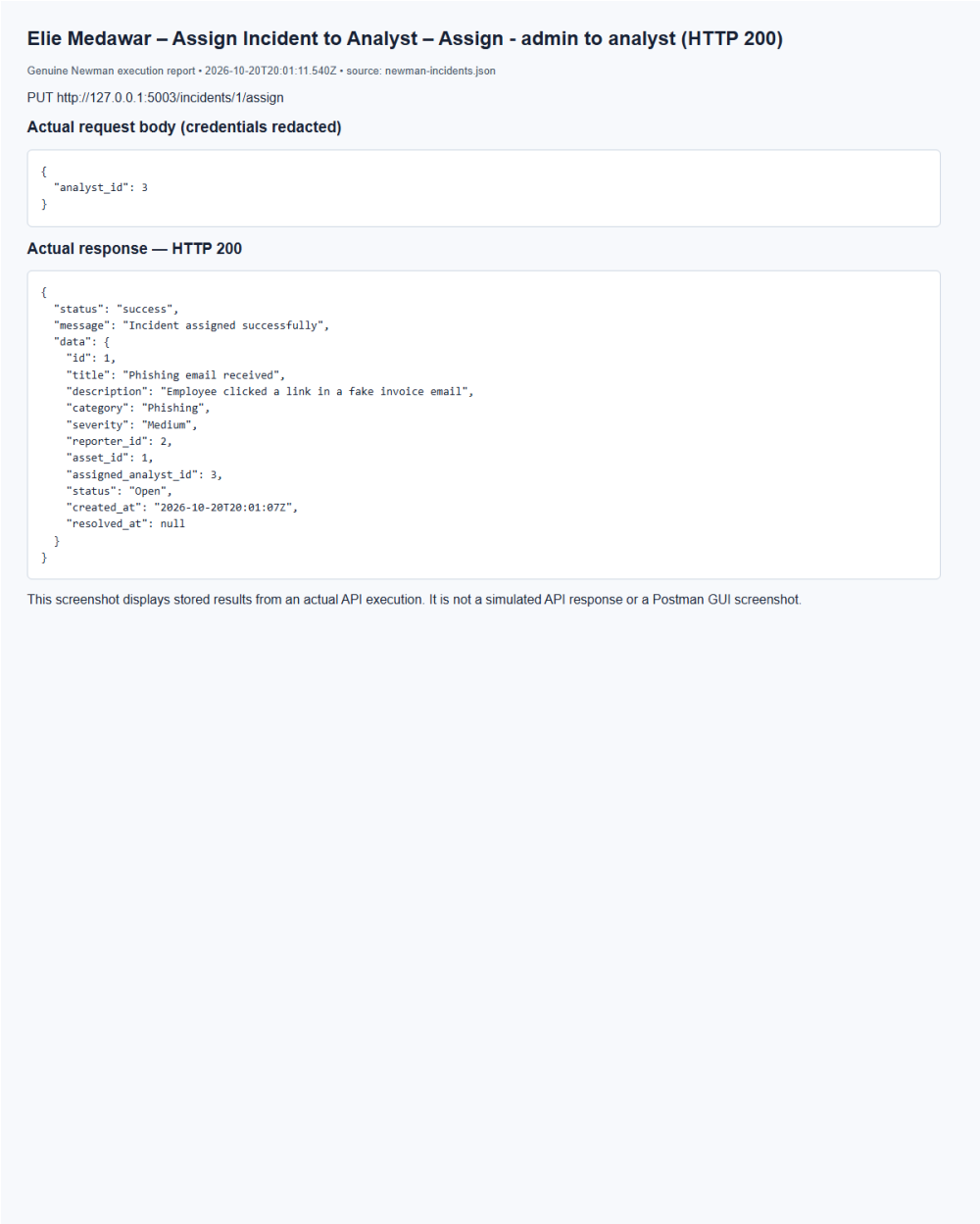
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-19.png in the submission.)

Elie Medawar – Retrieve Specific Incident – Get - invalid id (HTTP 404)



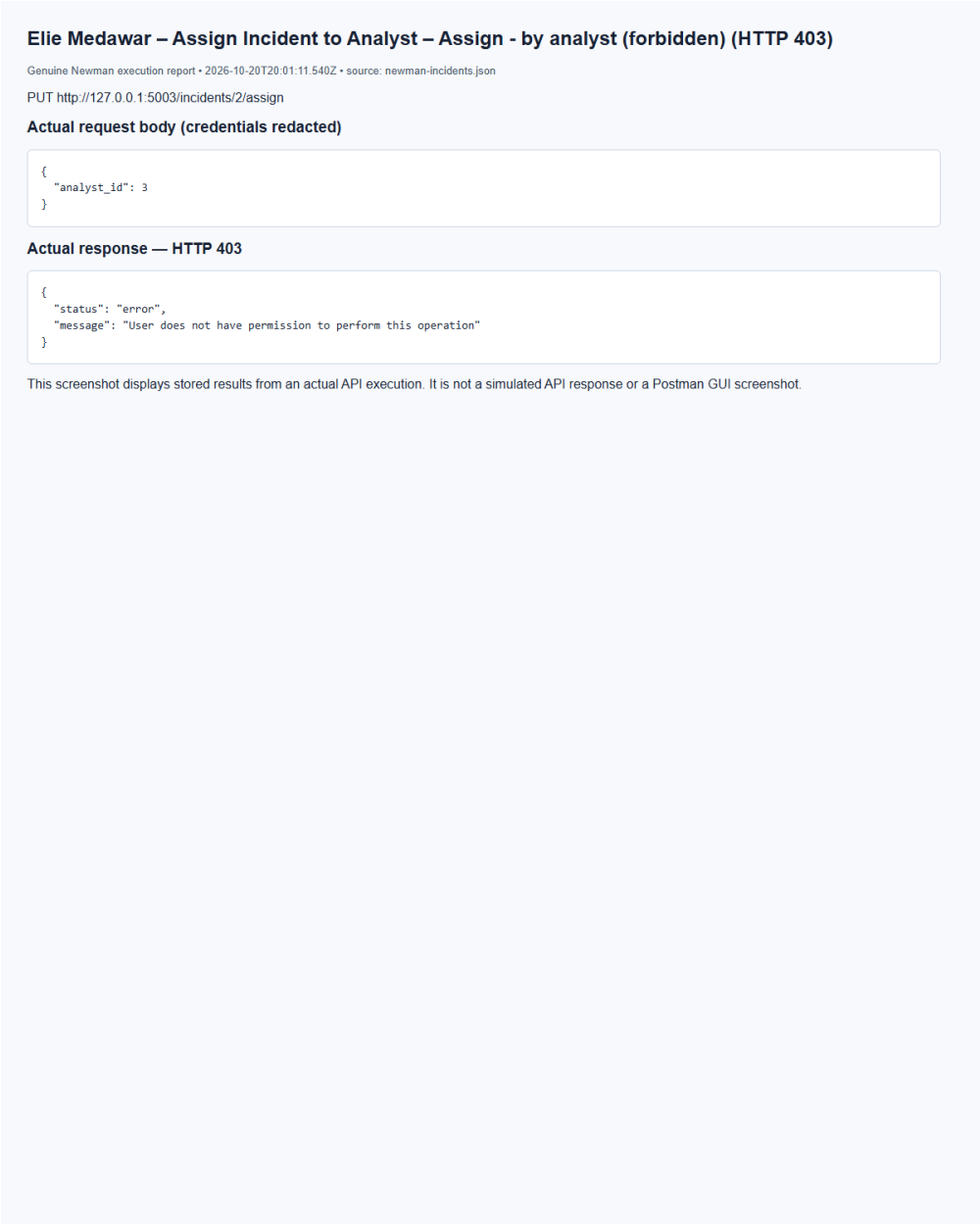
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-21.png in the submission.)

Elie Medawar – Assign Incident to Analyst – Assign - admin to analyst (HTTP 200)



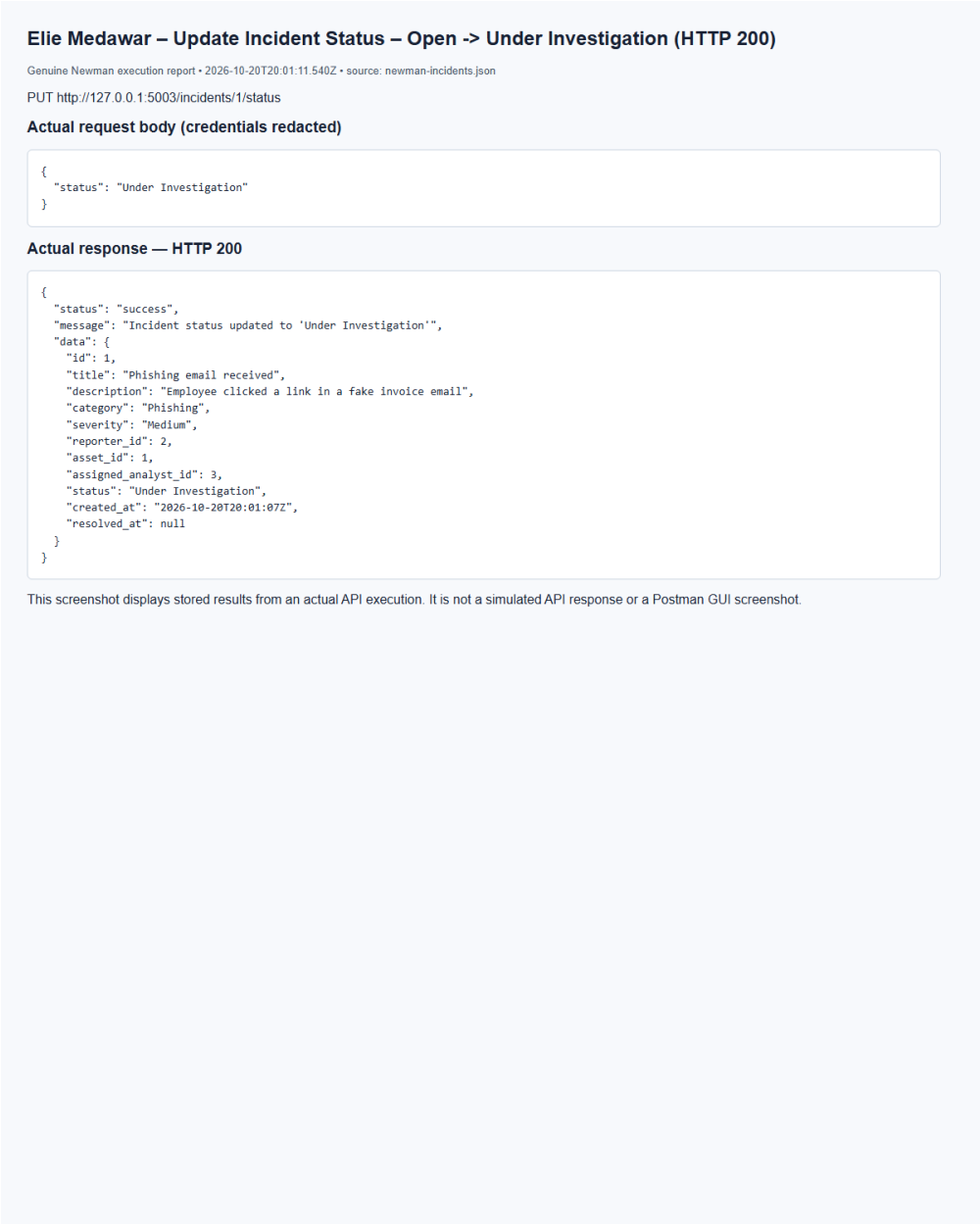
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-24.png in the submission.)

Elie Medawar – Assign Incident to Analyst – Assign - by analyst (forbidden) (HTTP 403)



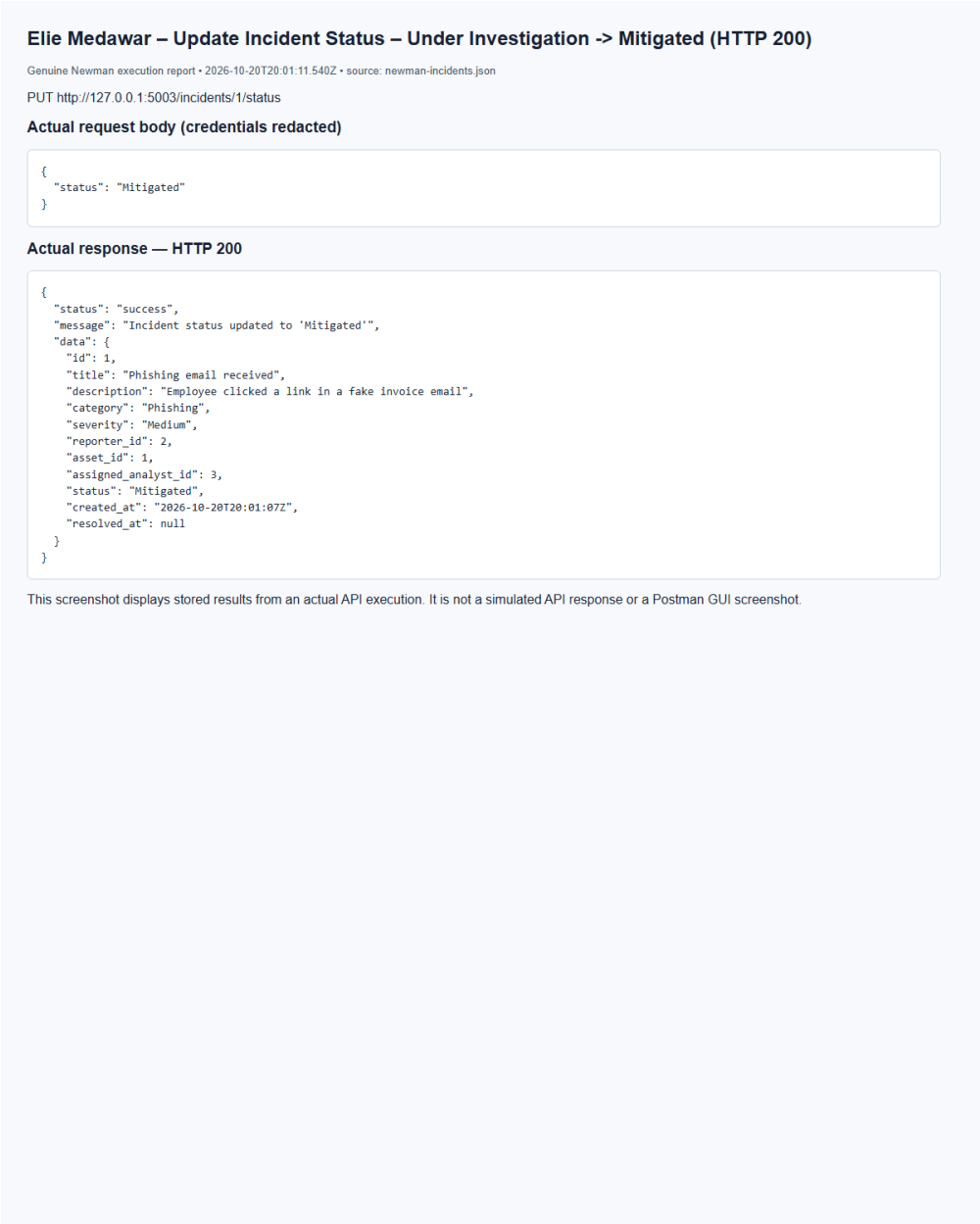
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-27.png in the submission.)

Elie Medawar – Update Incident Status – Open -> Under Investigation (HTTP 200)



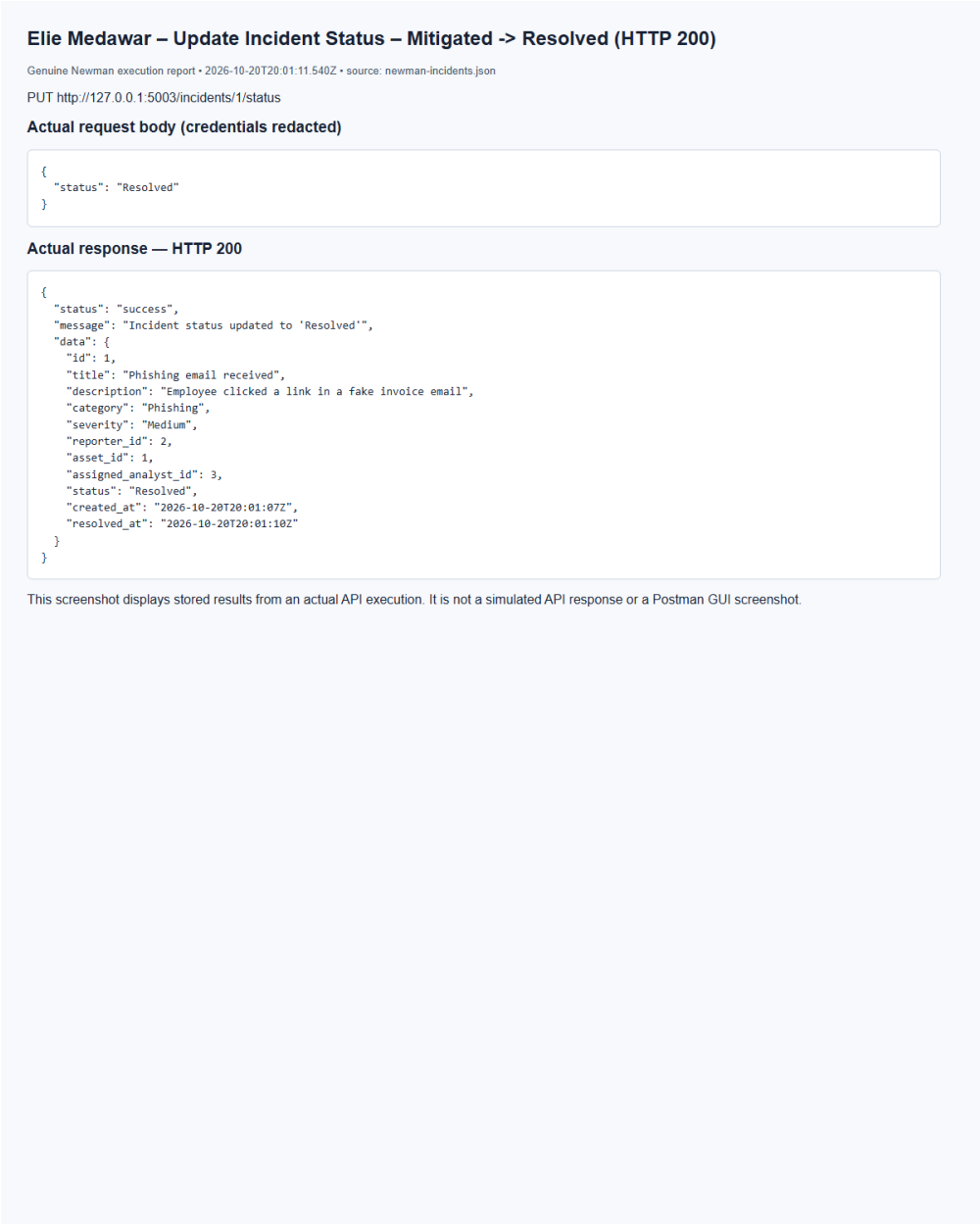
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-30.png in the submission.)

Elie Medawar – Update Incident Status – Under Investigation -> Mitigated (HTTP 200)



(Output shortened; the full capture is report/screenshots/elie/newman-incidents-31.png in the submission.)

Elie Medawar – Update Incident Status – Mitigated -> Resolved (HTTP 200)



Elie Medawar – Update Incident Status – Mitigated -> Resolved (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:11.540Z • source: newman-incidents.json

PUT http://127.0.0.1:5003/incidents/1/status

Actual request body (credentials redacted)


```
{
  "status": "Resolved"
}
```

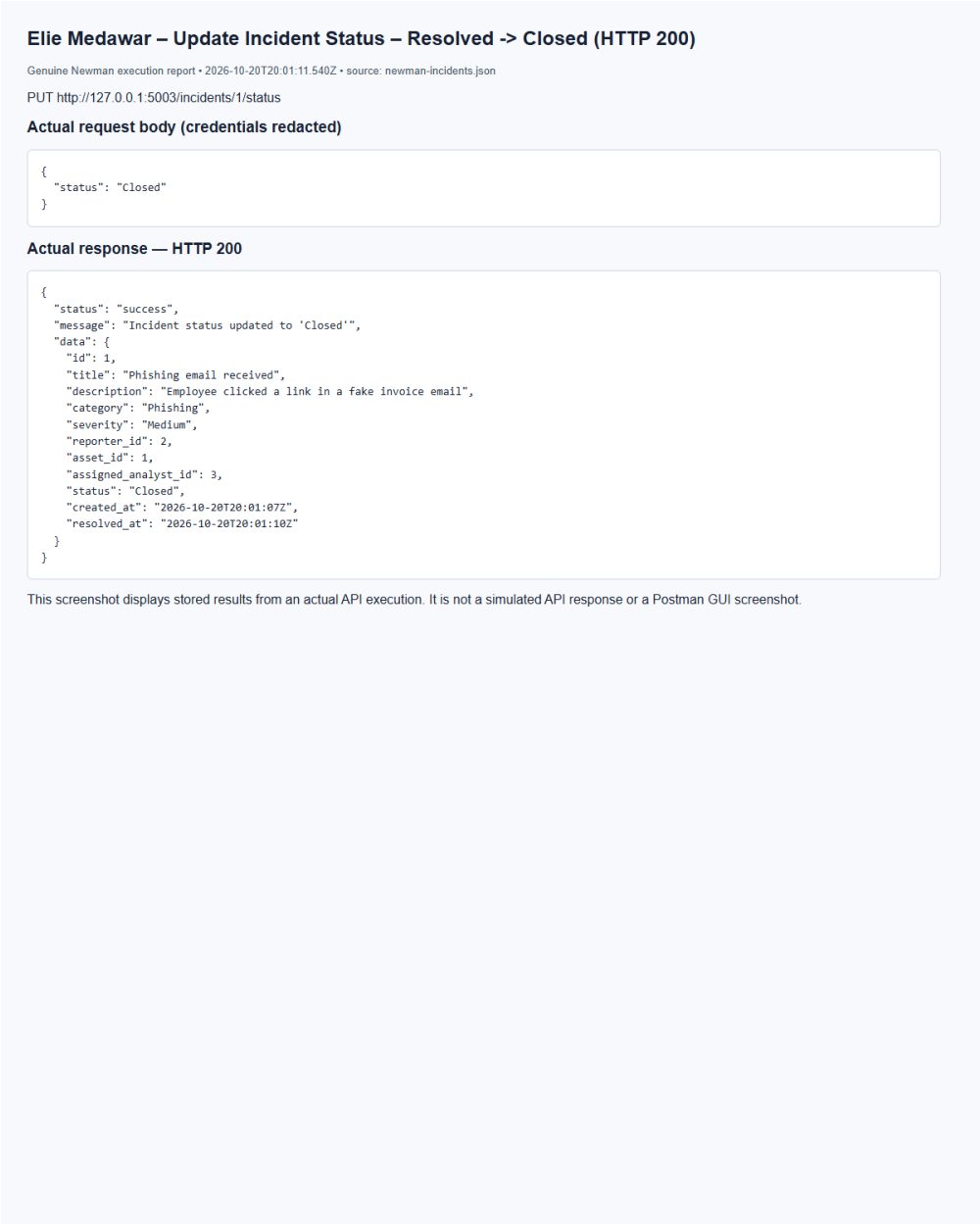

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Incident status updated to 'Resolved'",
  "data": {
    "id": 1,
    "title": "Phishing email received",
    "description": "Employee clicked a link in a fake invoice email",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 1,
    "assigned_analyst_id": 3,
    "status": "Resolved",
    "created_at": "2026-10-20T20:01:07Z",
    "resolved_at": "2026-10-20T20:01:10Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

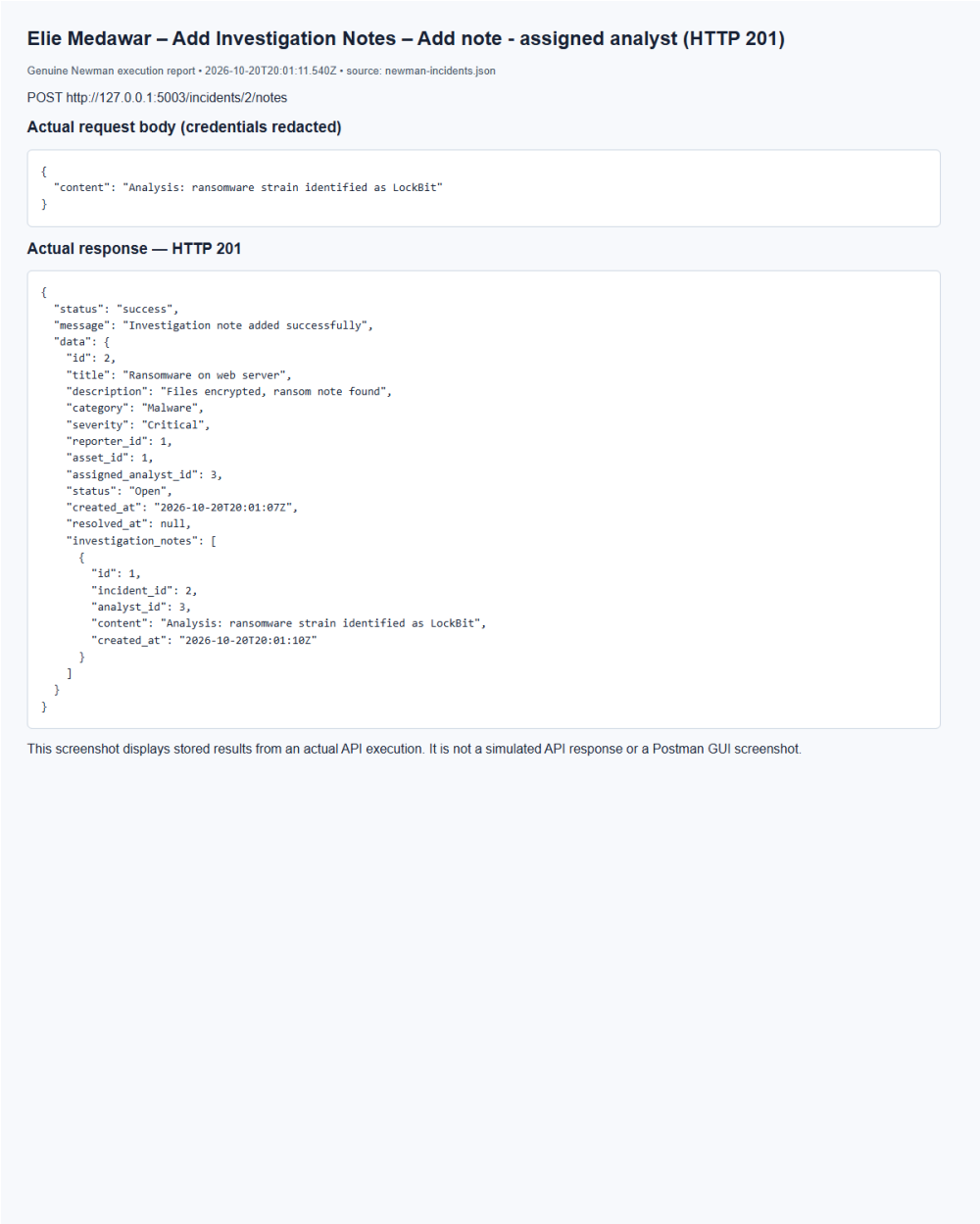
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-32.png in the submission.)

Elie Medawar – Update Incident Status – Resolved -> Closed (HTTP 200)



(Output shortened; the full capture is report/screenshots/elie/newman-incidents-34.png in the submission.)

Elie Medawar – Add Investigation Notes – Add note - assigned analyst (HTTP 201)



Elie Medawar – Add Investigation Notes – Add note - assigned analyst (HTTP 201)

Genuine Newman execution report • 2026-10-20T20:01:11.540Z • source: newman-incidents.json

POST http://127.0.0.1:5003/incidents/2/notes

Actual request body (credentials redacted)


```
{
  "content": "Analysis: ransomware strain identified as LockBit"
}
```

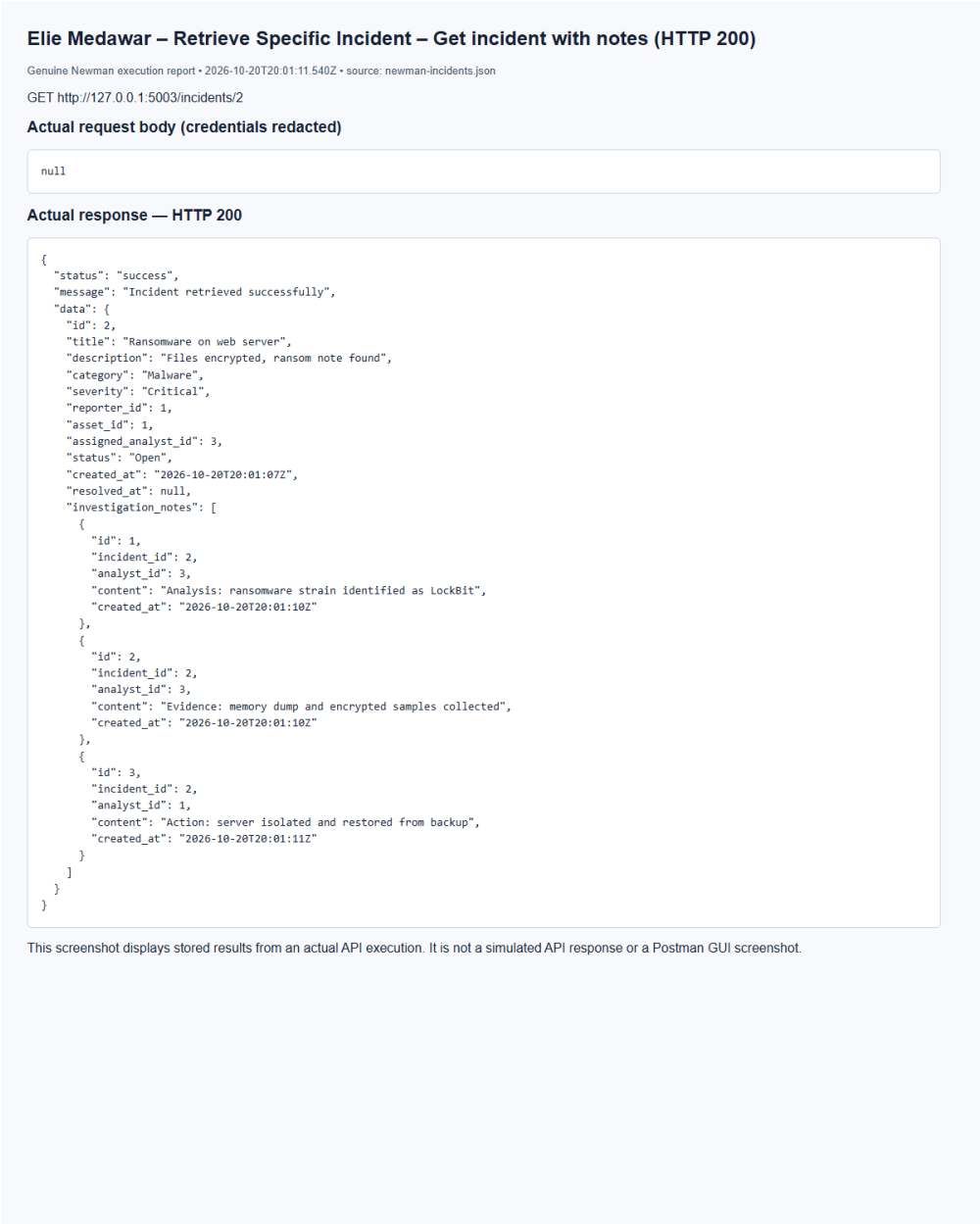

Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Investigation note added successfully",
  "data": {
    "id": 2,
    "title": "Ransomware on web server",
    "description": "Files encrypted, ransom note found",
    "category": "Malware",
    "severity": "Critical",
    "reporter_id": 1,
    "asset_id": 1,
    "assigned_analyst_id": 3,
    "status": "Open",
    "created_at": "2026-10-20T20:01:07Z",
    "resolved_at": null,
    "investigation_notes": [
      {
        "id": 1,
        "incident_id": 2,
        "analyst_id": 3,
        "content": "Analysis: ransomware strain identified as LockBit",
        "created_at": "2026-10-20T20:01:10Z"
      }
    ]
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-incidents-37.png in the submission.)

Elie Medawar – Retrieve Specific Incident – Get incident with notes (HTTP 200)



Elie Medawar – Retrieve Specific Incident – Get incident with notes (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:11.540Z • source: newman-incidents.json

GET http://127.0.0.1:5003/incidents/2

Actual request body (credentials redacted)

```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Incident retrieved successfully",
  "data": {
    "id": 2,
    "title": "Ransomware on web server",
    "description": "Files encrypted, ransom note found",
    "category": "Malware",
    "severity": "Critical",
    "reporter_id": 1,
    "asset_id": 1,
    "assigned_analyst_id": 3,
    "status": "Open",
    "created_at": "2026-10-20T20:01:07Z",
    "resolved_at": null,
    "investigation_notes": [
      {
        "id": 1,
        "incident_id": 2,
        "analyst_id": 3,
        "content": "Analysis: ransomware strain identified as LockBit",
        "created_at": "2026-10-20T20:01:10Z"
      },
      {
        "id": 2,
        "incident_id": 2,
        "analyst_id": 3,
        "content": "Evidence: memory dump and encrypted samples collected",
        "created_at": "2026-10-20T20:01:10Z"
      },
      {
        "id": 3,
        "incident_id": 2,
        "analyst_id": 1,
        "content": "Action: server isolated and restored from backup",
        "created_at": "2026-10-20T20:01:11Z"
      }
    ]
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-incidents-43.png in the submission.)

5.4 Threat Intelligence Service (Marc Bteich)

ThreatValidator canonicalizes IP addresses, domains (lowercase and no terminal dot), and hexadecimal hashes. Hash lengths identify MD5/SHA1/SHA256 formats; malware-signature/Other values must be nonempty strings. A unique constraint on threat_type/threat_value protects against duplicates including concurrent inserts. Admins write; analysts read/search; auditors read lists/details. The detailed Service 4 matrix is followed for auditor search, and the existing shared enum uses Other.

Marc Bteich – Create Critical threat – POST /threats, HTTP 201

Marc Bteich – Create Critical threat – POST /threats, HTTP 201

Genuine Newman execution report • 2026-10-20T12:07:44.605Z • source: newman-threats.json

POST http://127.0.0.1:5004/threats

Actual request body (credentials redacted)


```
{
  "threat_type": "Domain",
  "threat_value": "postman.example.test",
  "description": "Synthetic Postman lab indicator",
  "severity": "Critical",
  "source": "Postman lab"
}
```


Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Threat created",
  "data": {
    "id": 9,
    "threat_type": "Domain",
    "threat_value": "postman.example.test",
    "description": "Synthetic Postman lab indicator",
    "severity": "Critical",
    "source": "Postman lab",
    "date_added": "2026-10-20T12:07:42Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Duplicate normalized threat – POST /threats, HTTP 409

Marc Bteich – Duplicate normalized threat – POST /threats, HTTP 409

Genuine Newman execution report • 2026-10-20T12:07:44.605Z • source: newman-threats.json

POST http://127.0.0.1:5004/threats

Actual request body (credentials redacted)

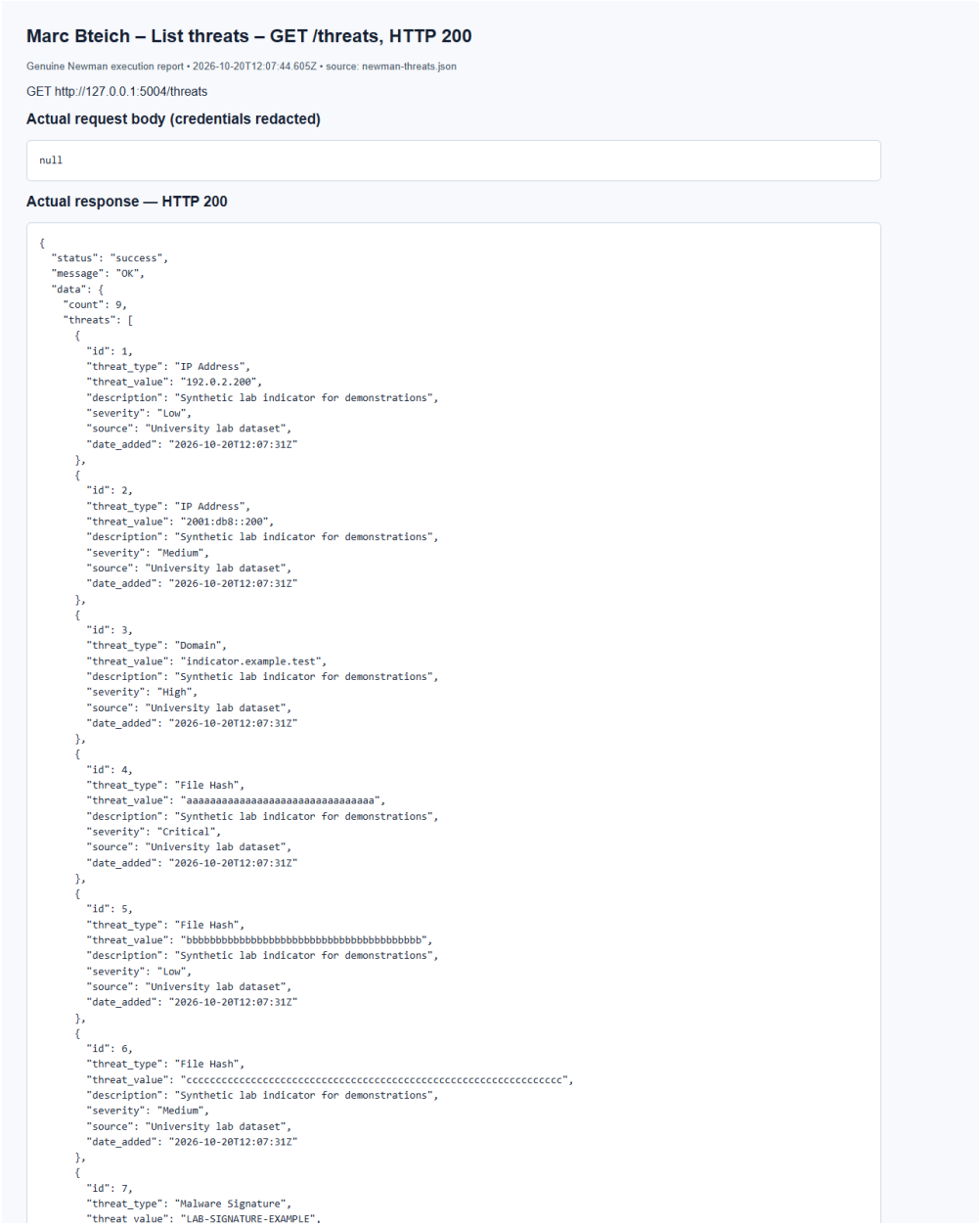

```
{
  "threat_type": "Domain",
  "threat_value": "postman.example.test",
  "description": "Synthetic Postman lab indicator",
  "severity": "Critical",
  "source": "Postman lab"
}
```


Actual response — HTTP 409


```
{
  "status": "error",
  "message": "Threat type/value pair already exists"
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – List threats – GET /threats, HTTP 200



Marc Bteich – List threats – GET /threats, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:44.605Z • source: newman-threats.json

GET http://127.0.0.1:5004/threats

Actual request body (credentials redacted)

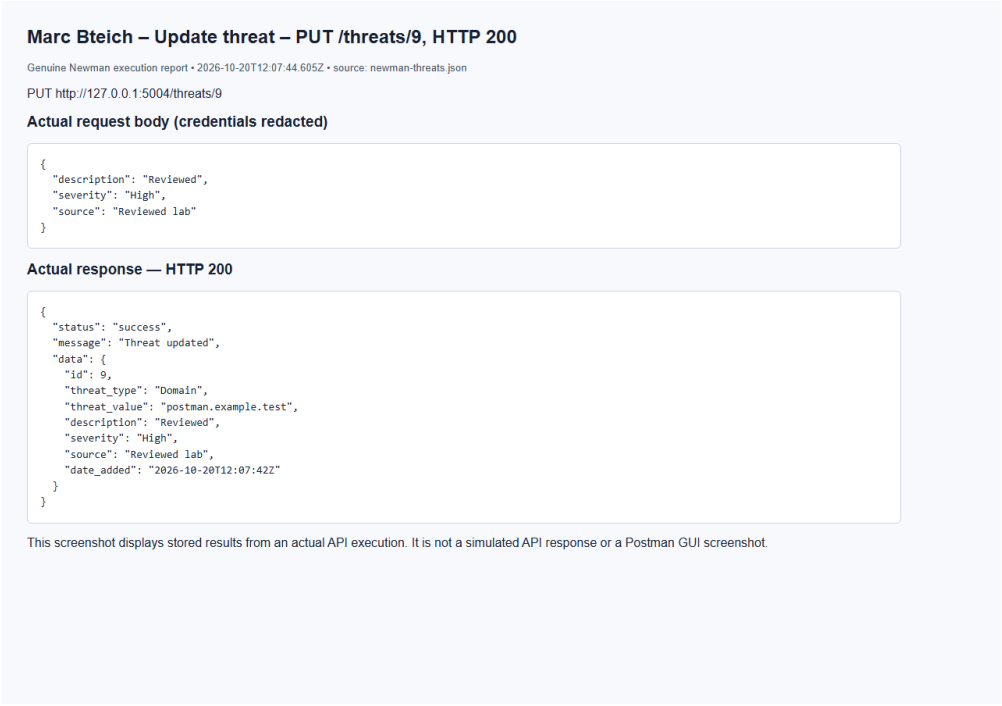
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 9,
    "threats": [
      {
        "id": 1,
        "threat_type": "IP Address",
        "threat_value": "192.0.2.200",
        "description": "Synthetic lab indicator for demonstrations",
        "severity": "Low",
        "source": "University lab dataset",
        "date_added": "2026-10-20T12:07:31Z"
      },
      {
        "id": 2,
        "threat_type": "IP Address",
        "threat_value": "2001:db8::200",
        "description": "Synthetic lab indicator for demonstrations",
        "severity": "Medium",
        "source": "University lab dataset",
        "date_added": "2026-10-20T12:07:31Z"
      },
      {
        "id": 3,
        "threat_type": "Domain",
        "threat_value": "indicator.example.test",
        "description": "Synthetic lab indicator for demonstrations",
        "severity": "High",
        "source": "University lab dataset",
        "date_added": "2026-10-20T12:07:31Z"
      },
      {
        "id": 4,
        "threat_type": "File Hash",
        "threat_value": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
        "description": "Synthetic lab indicator for demonstrations",
        "severity": "Critical",
        "source": "University lab dataset",
        "date_added": "2026-10-20T12:07:31Z"
      },
      {
        "id": 5,
        "threat_type": "File Hash",
        "threat_value": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
        "description": "Synthetic lab indicator for demonstrations",
        "severity": "Low",
        "source": "University lab dataset",
        "date_added": "2026-10-20T12:07:31Z"
      },
      {
        "id": 6,
        "threat_type": "File Hash",
        "threat_value": "ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc",
        "description": "Synthetic lab indicator for demonstrations",
        "severity": "Medium",
        "source": "University lab dataset",
        "date_added": "2026-10-20T12:07:31Z"
      },
      {
        "id": 7,
        "threat_type": "Malware Signature",
        "threat_value": "LAB-SIGNATURE-EXAMPLE",
```


(Output shortened; the full capture is report/screenshots/marc/newman-threats-07.png in the submission.)

Marc Bteich – Update threat – PUT /threats/9, HTTP 200



Marc Bteich – Update threat – PUT /threats/9, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:44.605Z • source: newman-threats.json

PUT http://127.0.0.1:5004/threats/9

Actual request body (credentials redacted)

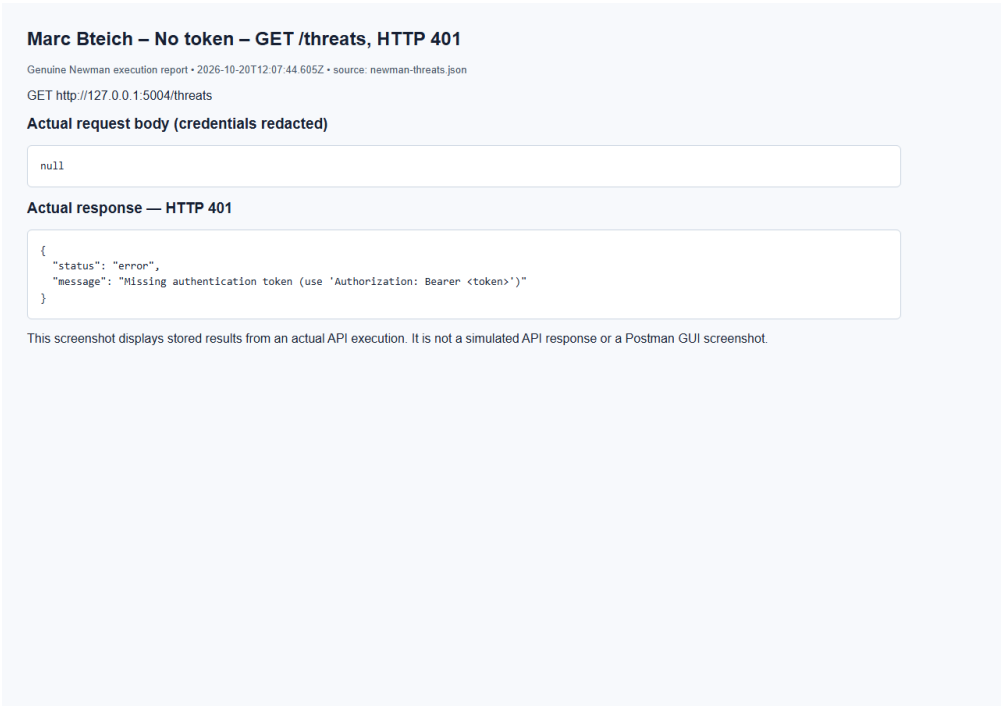

```
{
  "description": "Reviewed",
  "severity": "High",
  "source": "Reviewed lab"
}
```


Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Threat updated",
  "data": {
    "id": 9,
    "threat_type": "Domain",
    "threat_value": "postman.example.test",
    "description": "Reviewed",
    "severity": "High",
    "source": "Reviewed lab",
    "date_added": "2026-10-20T12:07:42Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – No token – GET /threats, HTTP 401



Marc Bteich – No token – GET /threats, HTTP 401

Genuine Newman execution report • 2026-10-20T12:07:44.605Z • source: newman-threats.json

GET http://127.0.0.1:5004/threats

Actual request body (credentials redacted)

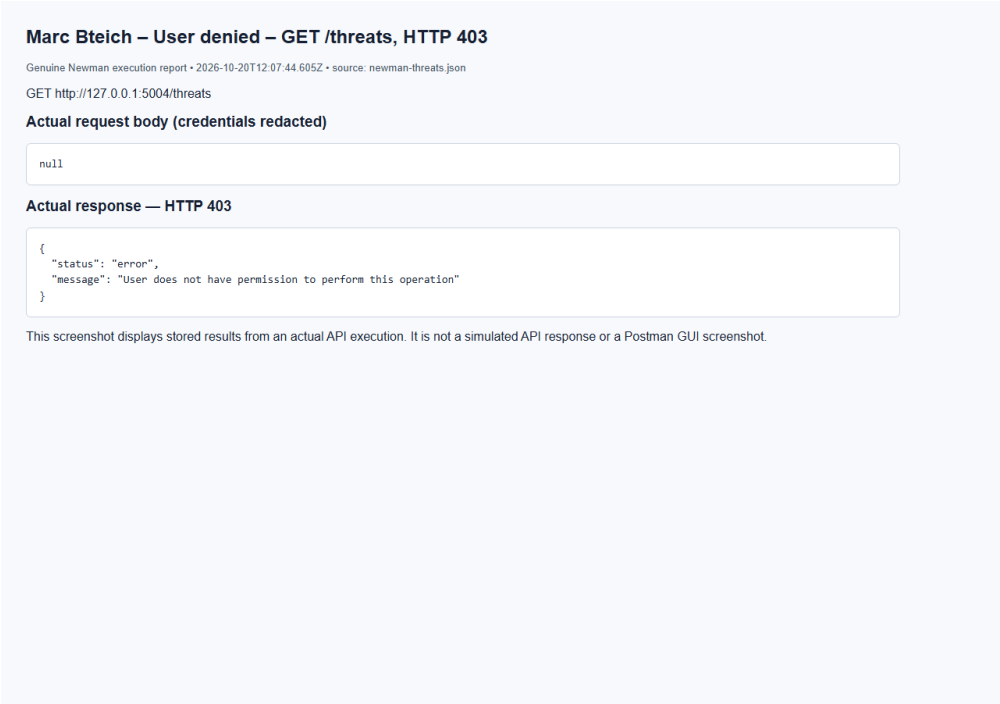

```
null
```


Actual response — HTTP 401


```
{
  "status": "error",
  "message": "Missing authentication token (use 'Authorization: Bearer <token>')"
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – User denied – GET /threats, HTTP 403



5.5 Security Audit Logging Service (Elie Medawar)

The Logs service (port 5005) is the system's audit trail: every service sends it a record of security-relevant activity, and admins and auditors review that record. Code: `services/logs/` (`LogService`, `LogValidator`) and `models/logs.py`.

API	Method	Endpoint	Allowed roles	Success
Create Log Record	POST	/logs	admin; other services (X-Service-Key)	201
Retrieve All Logs	GET	/logs	admin, auditor	200
Retrieve Specific Log	GET	/logs/{id}	admin, auditor, analyst	200
Search Logs	GET	/logs/search	admin, auditor, analyst	200
Delete Logs	DELETE	/logs/{id}	admin	200
Retrieve Logs By User	GET	/logs/user/{user_id}	admin, auditor	200

There is deliberately no PUT or PATCH endpoint: a modification attempt returns 405.

Action	Generated by	Severity
LOGIN_SUCCESS, LOGIN_FAILED, LOGIN_BLOCKED	Users service, on every login attempt	Information / Warning
USER_CREATED, USER_UPDATED, USER_DELETED	Users service	Information / Warning
ROLE_CHANGED	Users service, when an admin changes a role	Warning

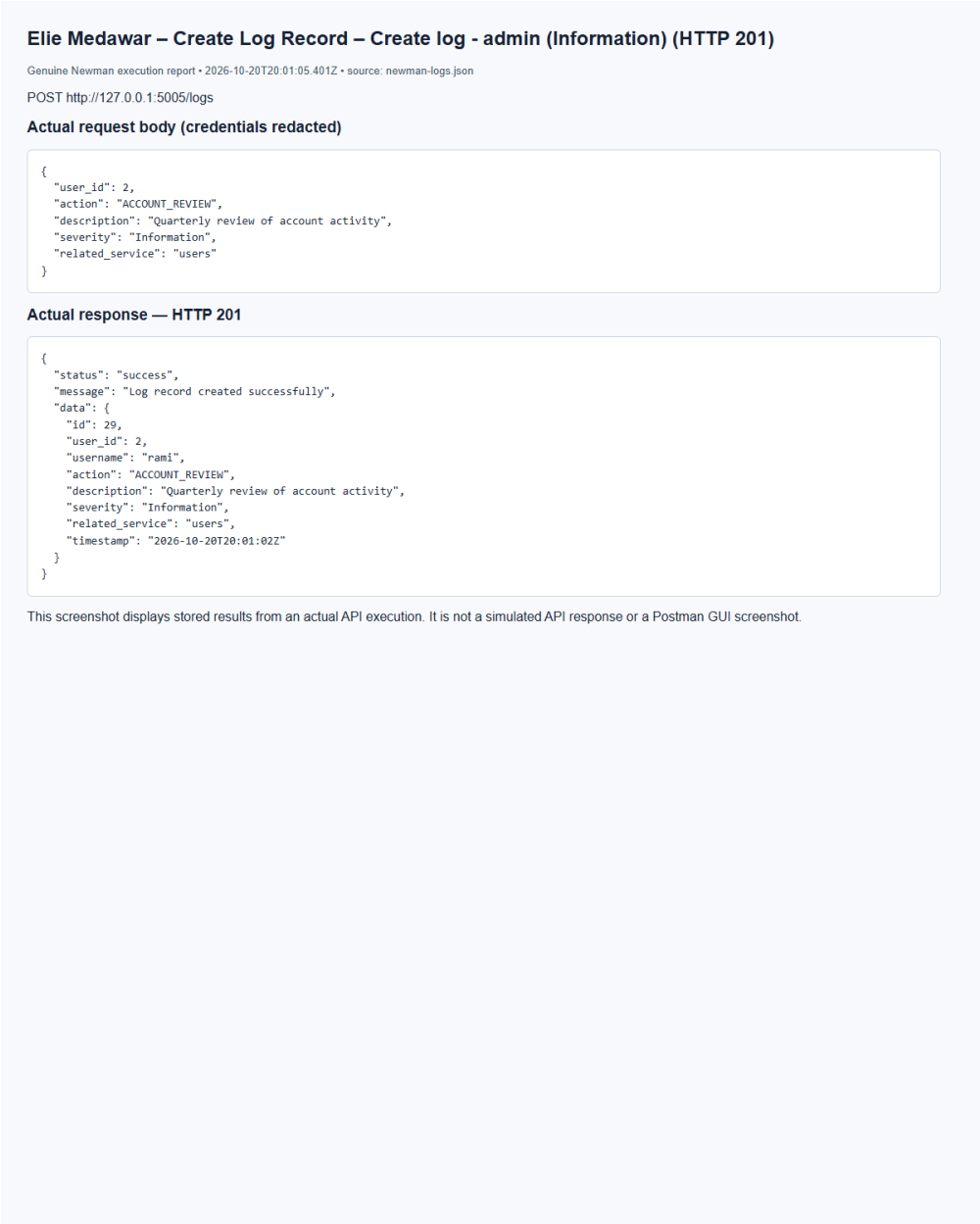
UNAUTHORIZED_ACCESS	@require_auth in every service, on each 403	Warning
INCIDENT_CREATED, INCIDENT_ASSIGNED, INCIDENT_STATUS_CHANGED, INVESTIGATION_NOTE_ADDED	Incidents service	Information (Critical for a Critical incident)
NOTIFICATION_CREATED, NOTIFICATION_READ, NOTIFICATION_DELETED	Notifications service	Information / Warning
LOG_DELETED	Logs service, when an admin deletes a log	Warning
Asset, threat, vulnerability and playbook changes	Marc Bteich's services, through the same log_event() helper	per action

How filtering works. /logs/search accepts user (an id or a username), action, severity, service, date (one day) and date_from / date_to (inclusive range). Every filter given is combined with AND, and results are newest first. At least one filter is required; an unknown parameter or a badly formatted date returns 400. /logs/user/{user_id} filters on user_id and returns an empty list with the message "No logs found for user <id>" when there are none.

Protection of audit records. Records are never modified: there is no update endpoint, and a SQLAlchemy before_update hook raises an error if code ever tries. Only an admin can delete a record, and the deletion itself is logged as LOG_DELETED. logs.user_id uses ON DELETE SET NULL and each record keeps a copy of the username, so deleting an account never deletes or blocks its history.

Database interaction. logs: id (PK), user_id (FK → users), username, action, description, severity (CHECK: Information, Warning, Error, Critical), related_service, timestamp, with indexes on user_id, action and timestamp for the searches.

Elie Medawar – Create Log Record – Create log - admin (Information) (HTTP 201)



Elie Medawar – Create Log Record – Create log - admin (Information) (HTTP 201)

Genuine Newman execution report • 2026-10-20T20:01:05.401Z • source: newman-logs.json

POST http://127.0.0.1:5005/logs

Actual request body (credentials redacted)


```
{
  "user_id": 2,
  "action": "ACCOUNT_REVIEW",
  "description": "Quarterly review of account activity",
  "severity": "Information",
  "related_service": "users"
}
```

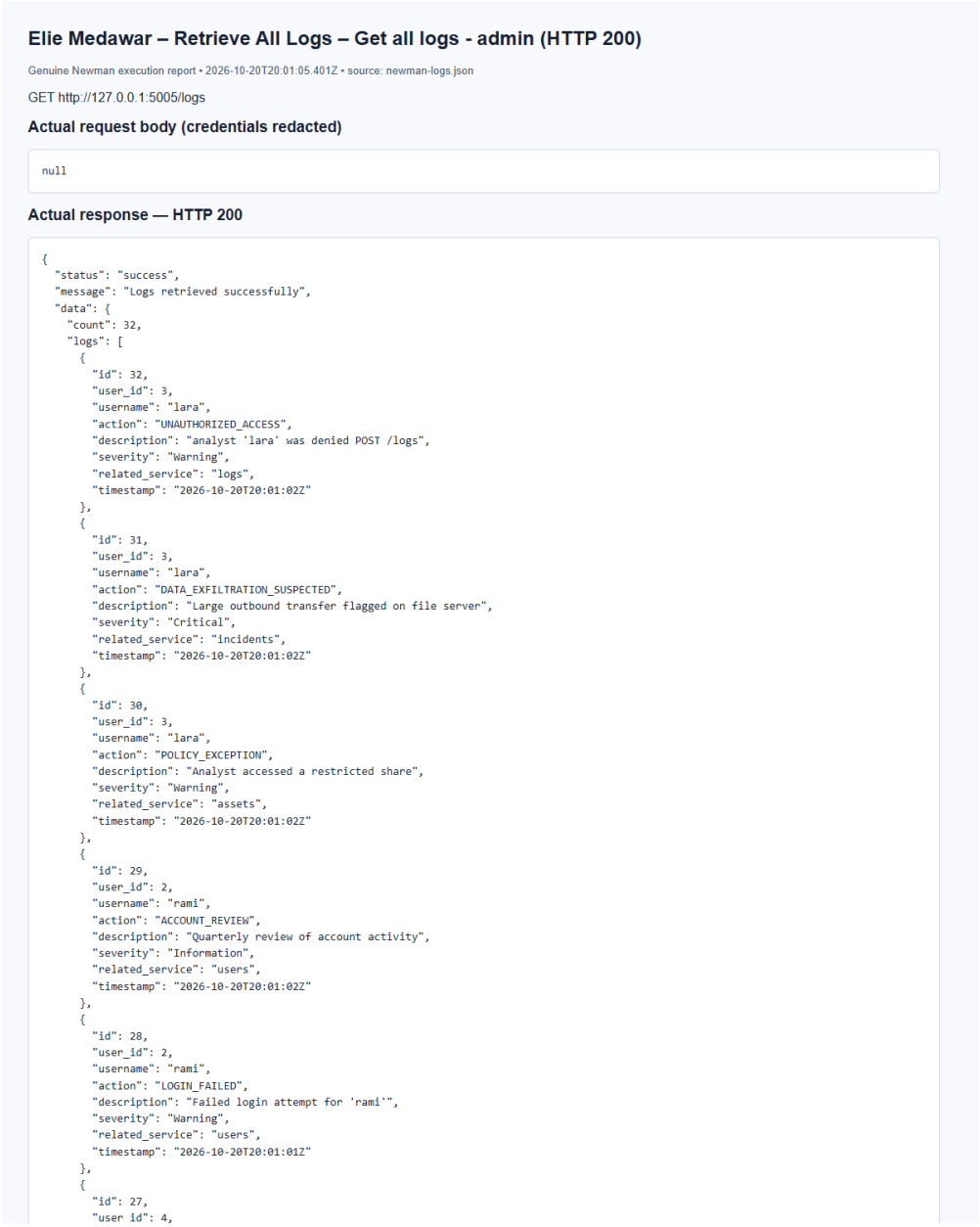

Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Log record created successfully",
  "data": {
    "id": 29,
    "user_id": 2,
    "username": "rami",
    "action": "ACCOUNT_REVIEW",
    "description": "Quarterly review of account activity",
    "severity": "Information",
    "related_service": "users",
    "timestamp": "2026-10-20T20:01:02Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-logs-06.png in the submission.)

Elie Medawar – Retrieve All Logs – Get all logs - admin (HTTP 200)



Elie Medawar – Retrieve All Logs – Get all logs - admin (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:05.401Z • source: newman-logs.json

GET http://127.0.0.1:5005/logs

Actual request body (credentials redacted)

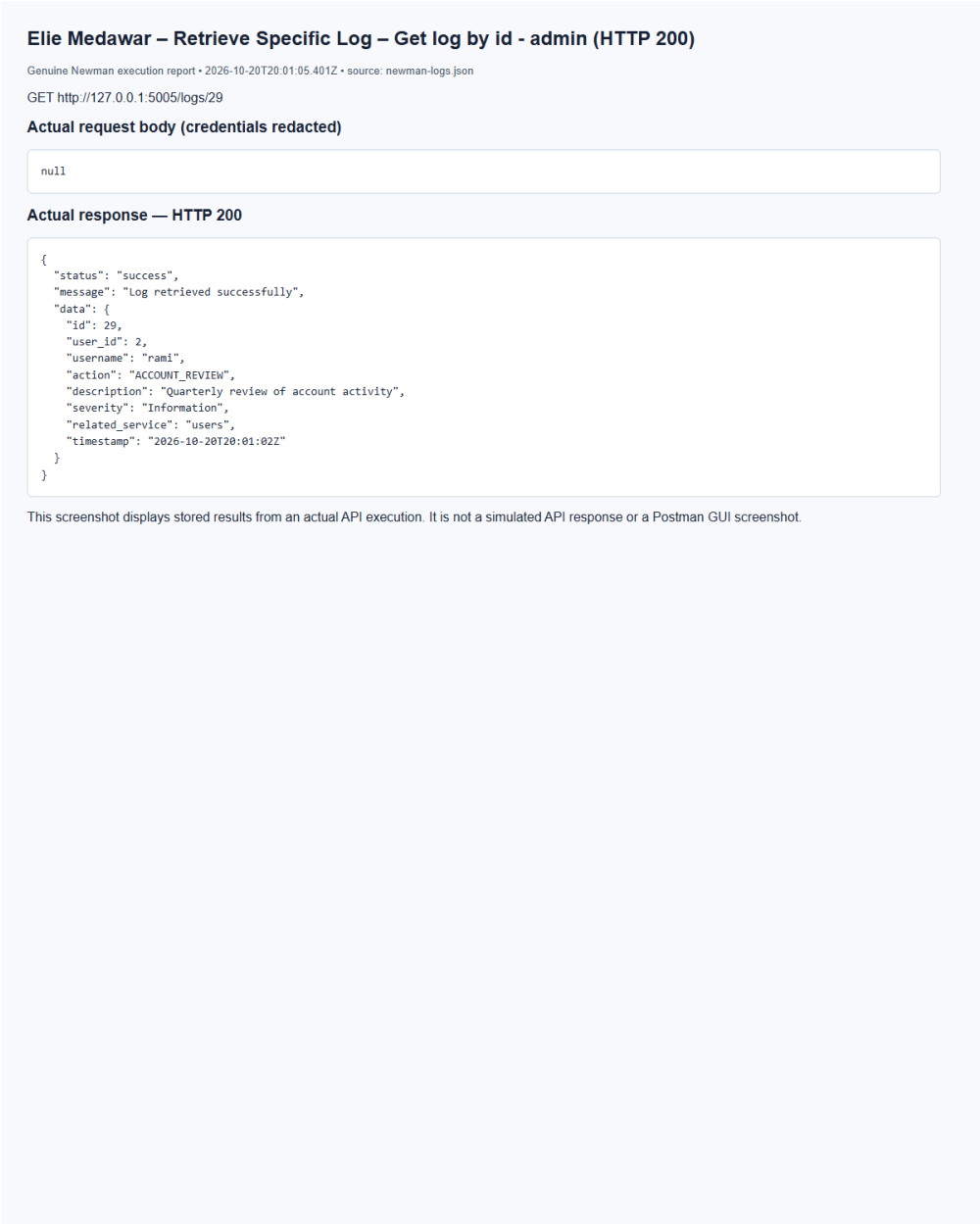
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Logs retrieved successfully",
  "data": {
    "count": 32,
    "logs": [
      {
        "id": 32,
        "user_id": 3,
        "username": "lara",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "analyst 'lara' was denied POST /logs",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 31,
        "user_id": 3,
        "username": "lara",
        "action": "DATA_EXFILTRATION_SUSPECTED",
        "description": "Large outbound transfer flagged on file server",
        "severity": "Critical",
        "related_service": "incidents",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 30,
        "user_id": 3,
        "username": "lara",
        "action": "POLICY_EXCEPTION",
        "description": "Analyst accessed a restricted share",
        "severity": "Warning",
        "related_service": "assets",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 29,
        "user_id": 2,
        "username": "rami",
        "action": "ACCOUNT_REVIEW",
        "description": "Quarterly review of account activity",
        "severity": "Information",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 28,
        "user_id": 2,
        "username": "rami",
        "action": "LOGIN_FAILED",
        "description": "Failed login attempt for 'rami'",
        "severity": "Warning",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:01Z"
      },
      {
        "id": 27,
        "user_id": 4,
```

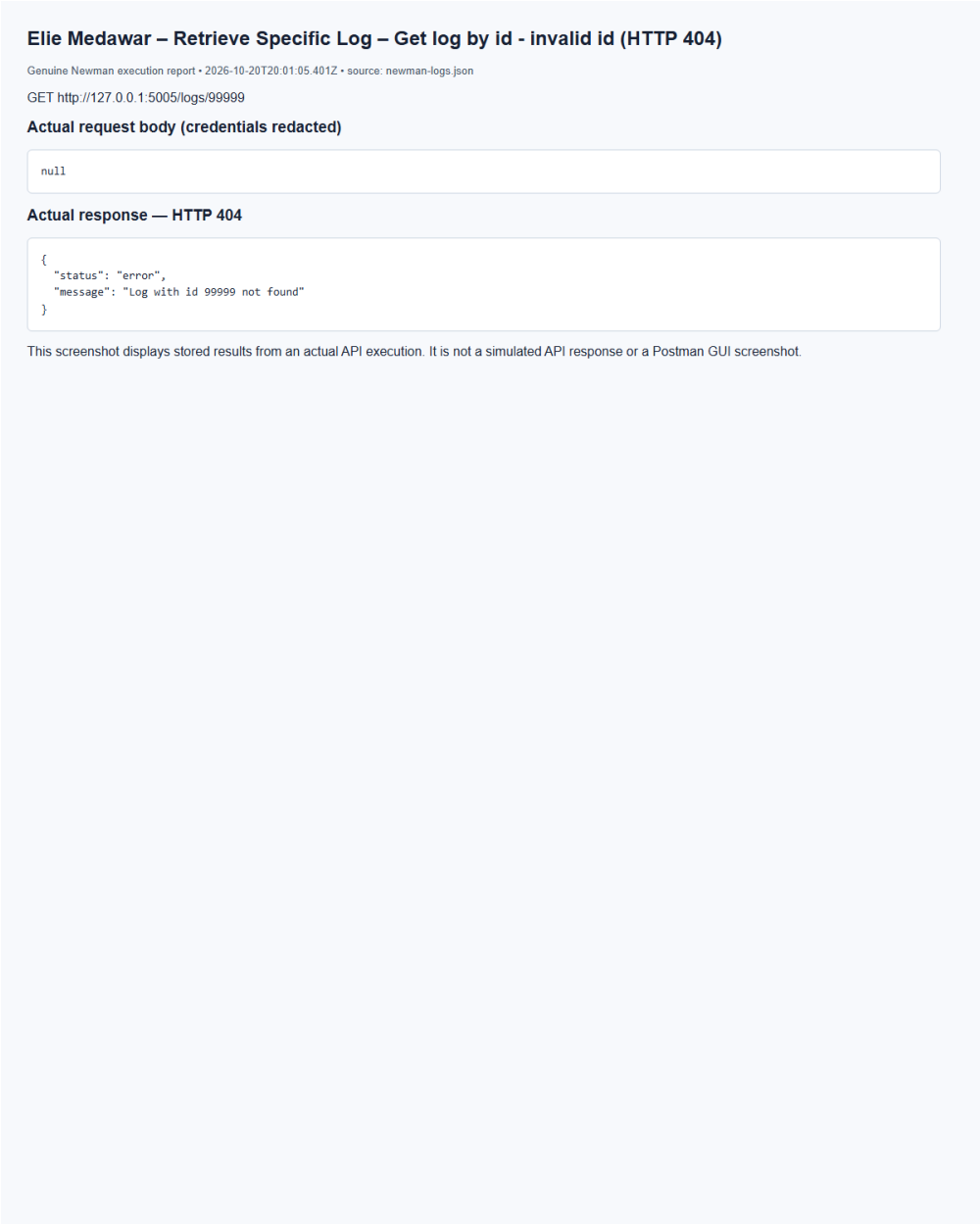

(Output shortened; the full capture is report/screenshots/elie/newman-logs-14.png in the submission.)

Elie Medawar – Retrieve Specific Log – Get log by id - admin (HTTP 200)



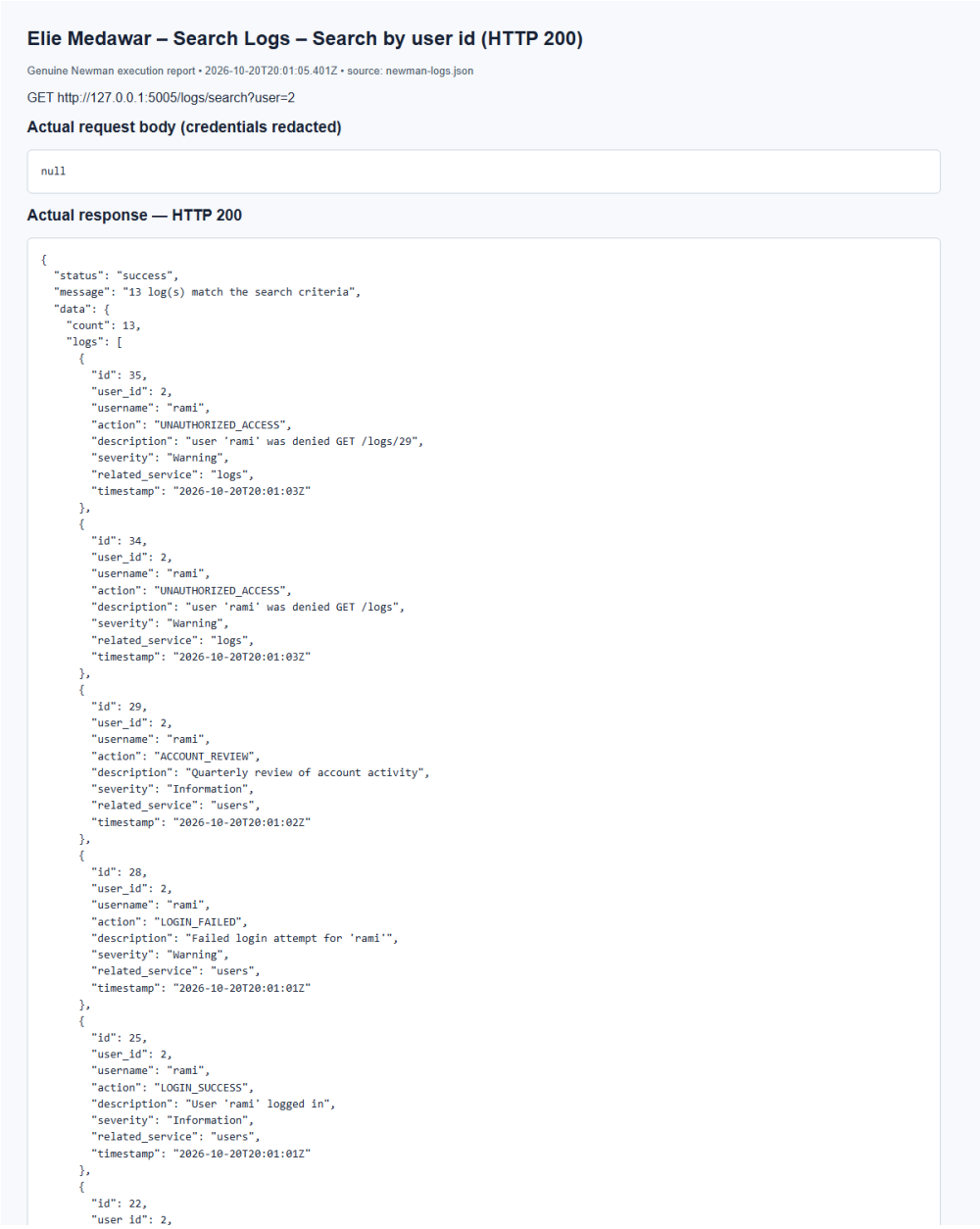
(Output shortened; the full capture is report/screenshots/elie/newman-logs-19.png in the submission.)

Elie Medawar – Retrieve Specific Log – Get log by id - invalid id (HTTP 404)



(Output shortened; the full capture is report/screenshots/elie/newman-logs-21.png in the submission.)

Elie Medawar – Search Logs – Search by user id (HTTP 200)



Elie Medawar – Search Logs – Search by user id (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:05.401Z • source: newman-logs.json

GET http://127.0.0.1:5005/logs/search?user=2

Actual request body (credentials redacted)

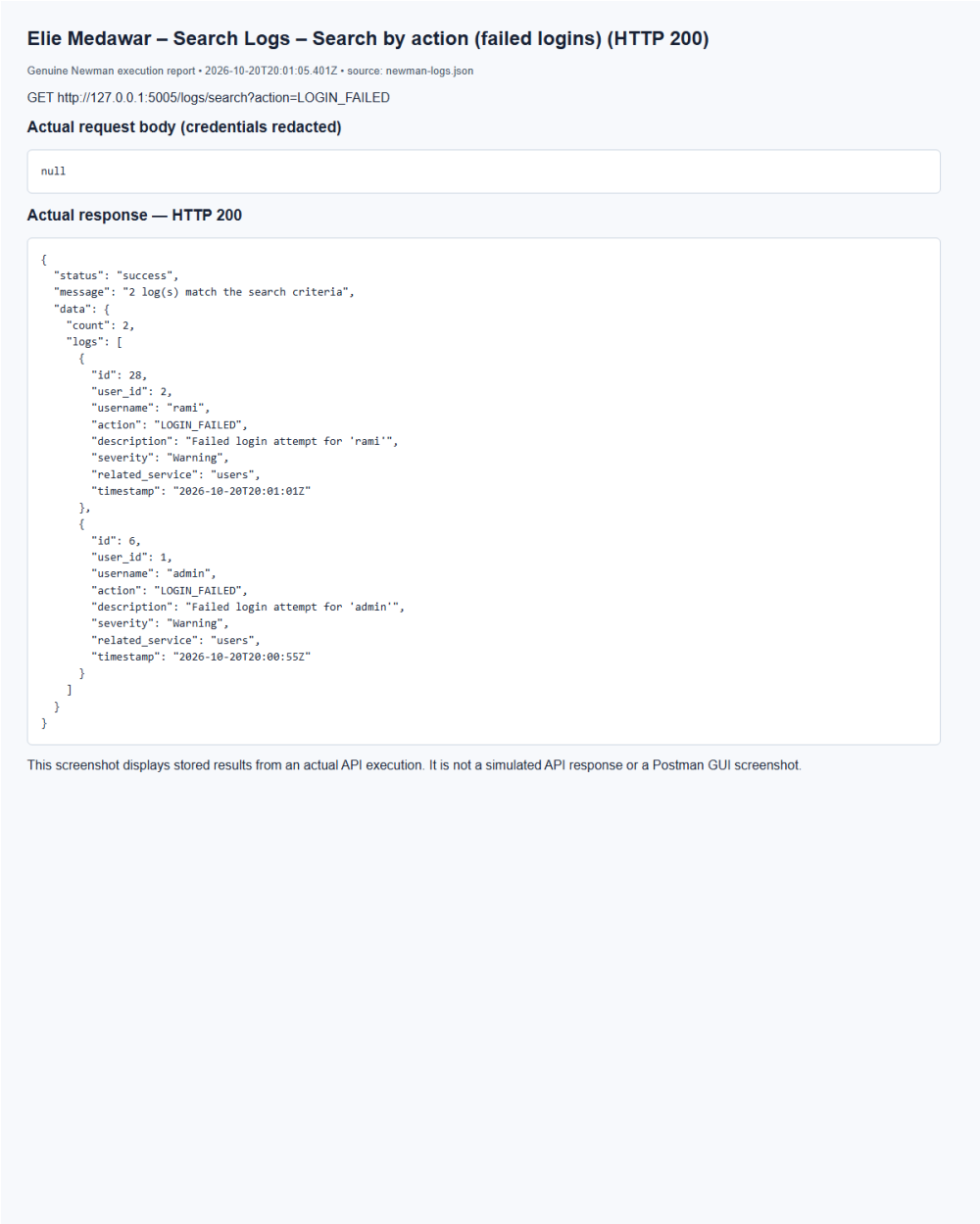
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "13 log(s) match the search criteria",
  "data": {
    "count": 13,
    "logs": [
      {
        "id": 35,
        "user_id": 2,
        "username": "rami",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "user 'rami' was denied GET /logs/29",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:03Z"
      },
      {
        "id": 34,
        "user_id": 2,
        "username": "rami",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "user 'rami' was denied GET /logs",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:03Z"
      },
      {
        "id": 29,
        "user_id": 2,
        "username": "rami",
        "action": "ACCOUNT_REVIEW",
        "description": "Quarterly review of account activity",
        "severity": "Information",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 28,
        "user_id": 2,
        "username": "rami",
        "action": "LOGIN_FAILED",
        "description": "Failed login attempt for 'rami'",
        "severity": "Warning",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:01Z"
      },
      {
        "id": 25,
        "user_id": 2,
        "username": "rami",
        "action": "LOGIN_SUCCESS",
        "description": "User 'rami' logged in",
        "severity": "Information",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:01Z"
      },
      {
        "id": 22,
        "user_id": 2,
```


(Output shortened; the full capture is report/screenshots/elie/newman-logs-23.png in the submission.)

Elie Medawar – Search Logs – Search by action (failed logins) (HTTP 200)



Elie Medawar – Search Logs – Search by action (failed logins) (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:05.401Z • source: newman-logs.json

GET http://127.0.0.1:5005/logs/search?action=LOGIN_FAILED

Actual request body (credentials redacted)

```
null
```

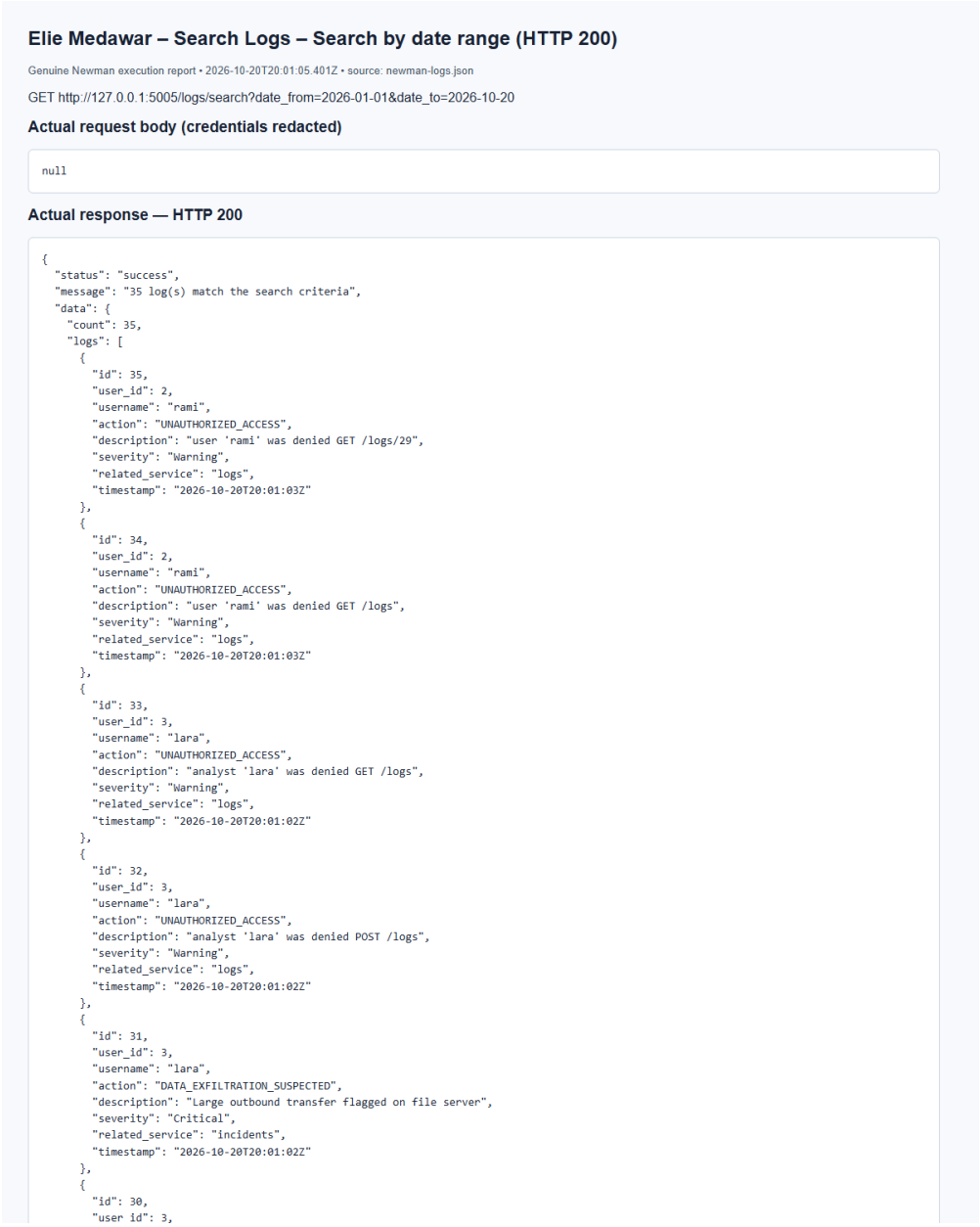
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "2 log(s) match the search criteria",
  "data": {
    "count": 2,
    "logs": [
      {
        "id": 28,
        "user_id": 2,
        "username": "rami",
        "action": "LOGIN_FAILED",
        "description": "Failed login attempt for 'rami'",
        "severity": "Warning",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:01Z"
      },
      {
        "id": 6,
        "user_id": 1,
        "username": "admin",
        "action": "LOGIN_FAILED",
        "description": "Failed login attempt for 'admin'",
        "severity": "Warning",
        "related_service": "users",
        "timestamp": "2026-10-20T20:00:55Z"
      }
    ]
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-logs-25.png in the submission.)

Elie Medawar – Search Logs – Search by date range (HTTP 200)



Elie Medawar – Search Logs – Search by date range (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:05.401Z • source: newman-logs.json

GET http://127.0.0.1:5005/logs/search?date_from=2026-01-01&date_to=2026-10-20

Actual request body (credentials redacted)

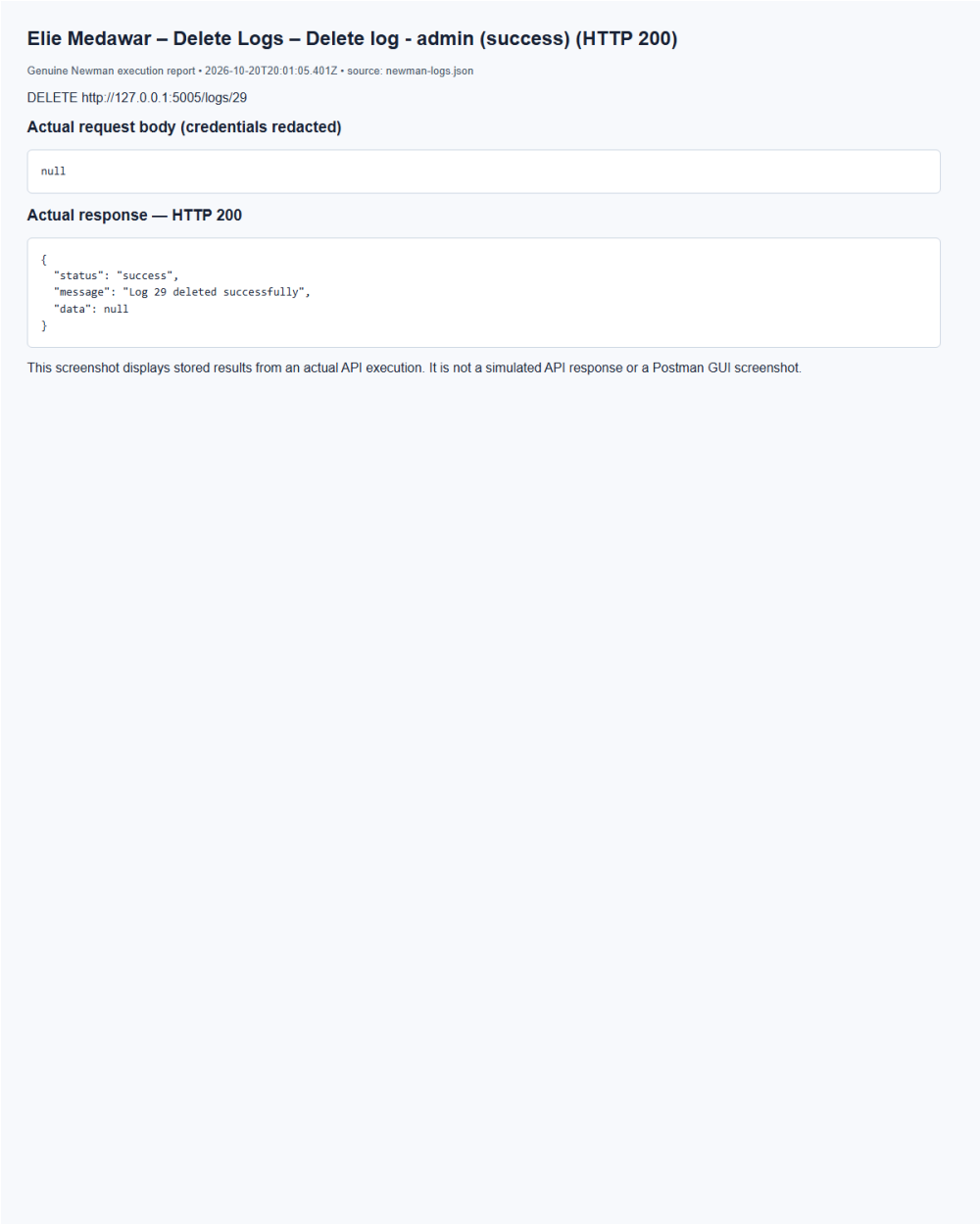
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "35 log(s) match the search criteria",
  "data": {
    "count": 35,
    "logs": [
      {
        "id": 35,
        "user_id": 2,
        "username": "rami",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "user 'rami' was denied GET /logs/29",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:03Z"
      },
      {
        "id": 34,
        "user_id": 2,
        "username": "rami",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "user 'rami' was denied GET /logs",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:03Z"
      },
      {
        "id": 33,
        "user_id": 3,
        "username": "lara",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "analyst 'lara' was denied GET /logs",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 32,
        "user_id": 3,
        "username": "lara",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "analyst 'lara' was denied POST /logs",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 31,
        "user_id": 3,
        "username": "lara",
        "action": "DATA_EXFILTRATION_SUSPECTED",
        "description": "Large outbound transfer flagged on file server",
        "severity": "Critical",
        "related_service": "incidents",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 30,
        "user_id": 3,
```

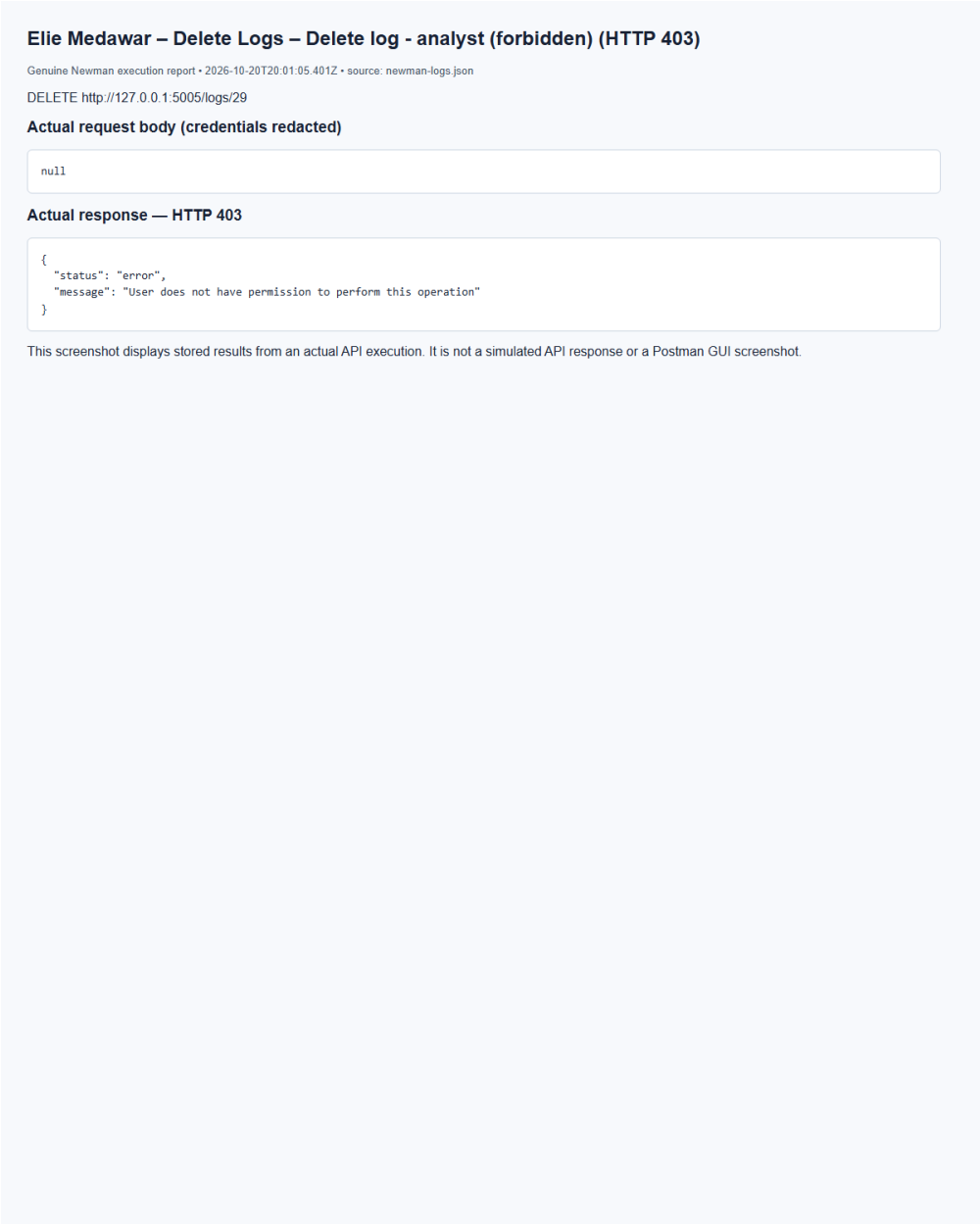

(Output shortened; the full capture is report/screenshots/elie/newman-logs-29.png in the submission.)

Elie Medawar – Delete Logs – Delete log - admin (success) (HTTP 200)



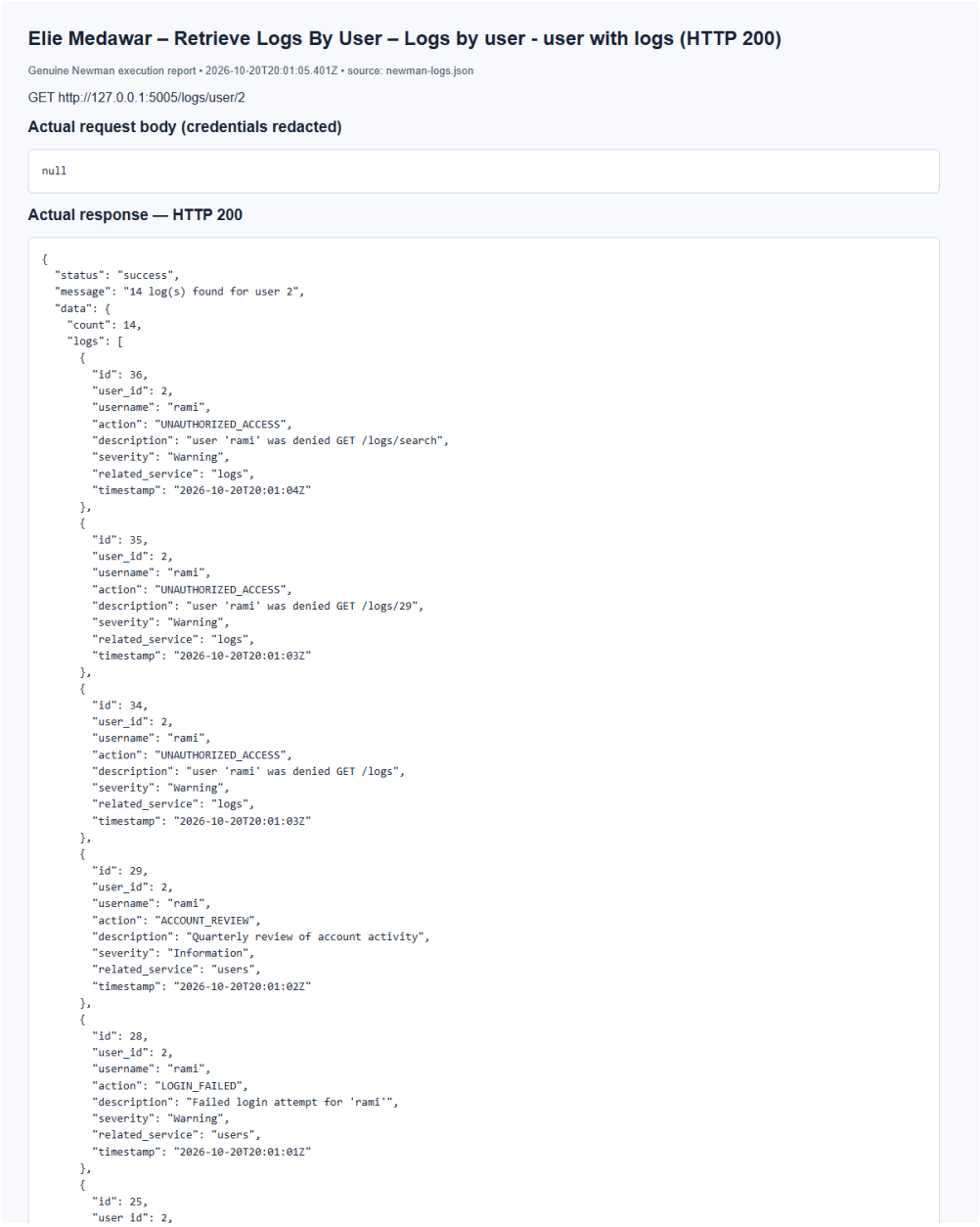
(Output shortened; the full capture is report/screenshots/elie/newman-logs-40.png in the submission.)

Elie Medawar – Delete Logs – Delete log - analyst (forbidden) (HTTP 403)



(Output shortened; the full capture is report/screenshots/elie/newman-logs-38.png in the submission.)

Elie Medawar – Retrieve Logs By User – Logs by user - user with logs (HTTP 200)



Elie Medawar – Retrieve Logs By User – Logs by user - user with logs (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:05.401Z • source: newman-logs.json

GET http://127.0.0.1:5005/logs/user/2

Actual request body (credentials redacted)

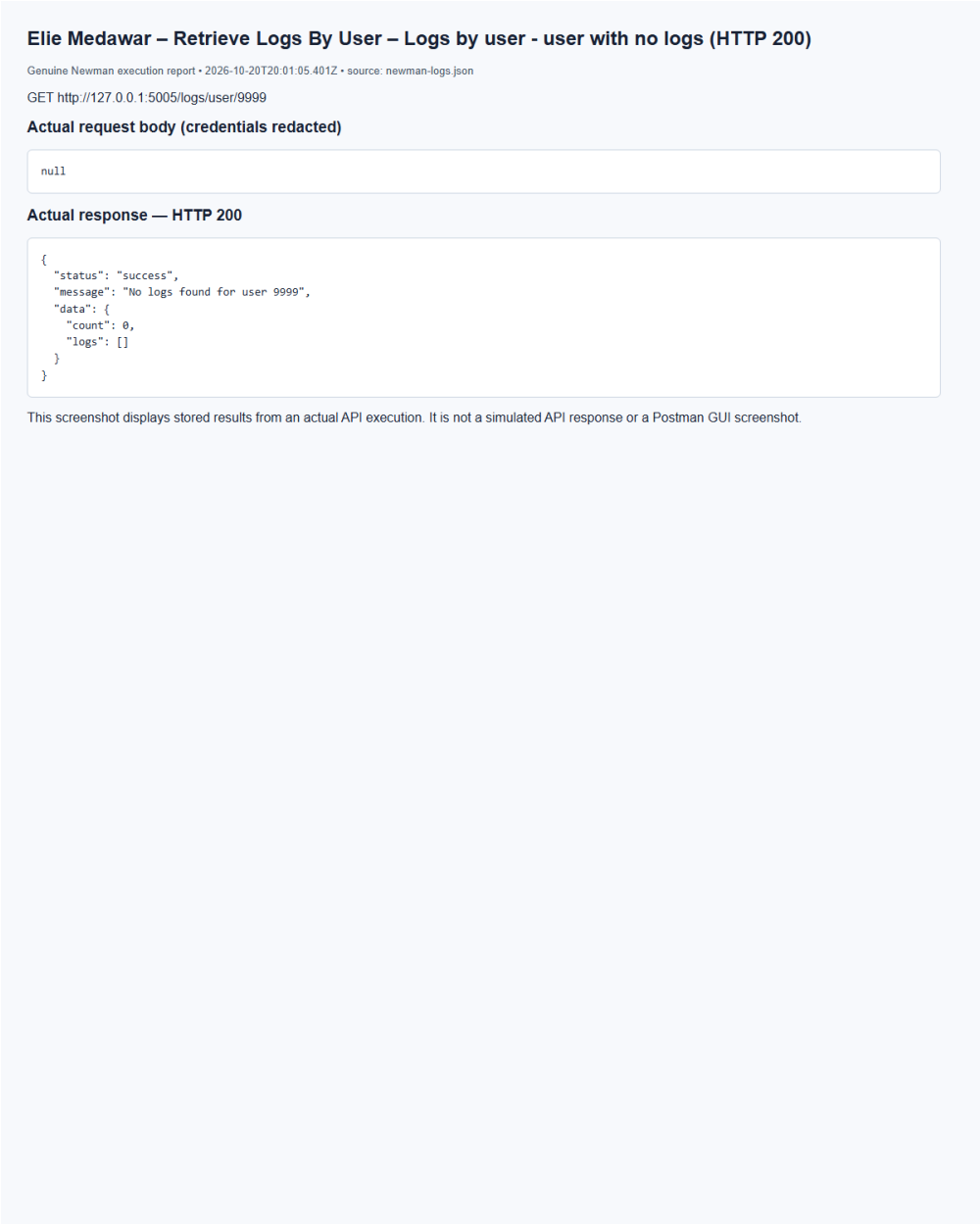
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "14 log(s) found for user 2",
  "data": {
    "count": 14,
    "logs": [
      {
        "id": 36,
        "user_id": 2,
        "username": "rami",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "user 'rami' was denied GET /logs/search",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:04Z"
      },
      {
        "id": 35,
        "user_id": 2,
        "username": "rami",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "user 'rami' was denied GET /logs/29",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:03Z"
      },
      {
        "id": 34,
        "user_id": 2,
        "username": "rami",
        "action": "UNAUTHORIZED_ACCESS",
        "description": "user 'rami' was denied GET /logs",
        "severity": "Warning",
        "related_service": "logs",
        "timestamp": "2026-10-20T20:01:03Z"
      },
      {
        "id": 29,
        "user_id": 2,
        "username": "rami",
        "action": "ACCOUNT_REVIEW",
        "description": "Quarterly review of account activity",
        "severity": "Information",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:02Z"
      },
      {
        "id": 28,
        "user_id": 2,
        "username": "rami",
        "action": "LOGIN_FAILED",
        "description": "Failed login attempt for 'rami'",
        "severity": "Warning",
        "related_service": "users",
        "timestamp": "2026-10-20T20:01:01Z"
      },
      {
        "id": 25,
        "user_id": 2,
```


(Output shortened; the full capture is report/screenshots/elie/newman-logs-35.png in the submission.)

Elie Medawar – Retrieve Logs By User – Logs by user - user with no logs (HTTP 200)



(Output shortened; the full capture is report/screenshots/elie/newman-logs-36.png in the submission.)

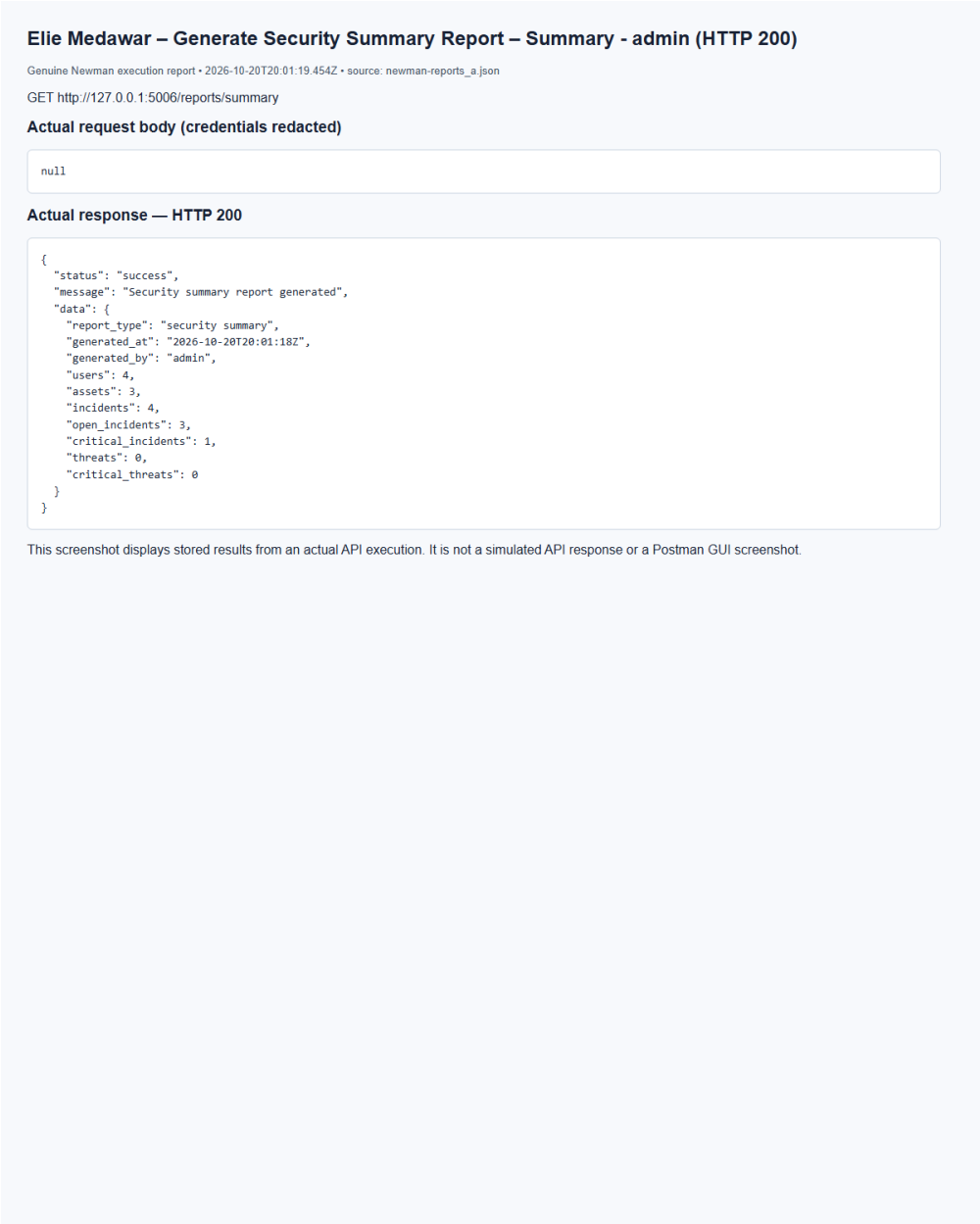
5.6 Security Reports Service – Summary, Incident and Activity Reports (Elie Medawar)

The three reports below run inside the Reports service (port 5006) and own no tables: each one calls the source services' GET /internal/stats endpoints over HTTP with the service key. Code: `services/reports/routes_a.py` (`ReportBuilder`). Marc Bteich's reports app registers this blueprint next to his asset, threat and vulnerability reports. All three are restricted to admin and auditor; analysts and users get 403.

Report	Endpoint	Data collected from	Contents
Security Summary	GET /reports/summary	Users, Assets, Incidents, Threats	users, assets, incidents, open incidents, critical incidents, threats, critical threats
Incident Report	GET /reports/incidents	Incidents	total, open, unassigned open, by severity, by status, by category
Activity Report	GET /reports/activity	Audit Logs	total activities, login attempts, successful and failed logins, failed-login rate, unauthorized attempts, by severity, by service, top 5 actions

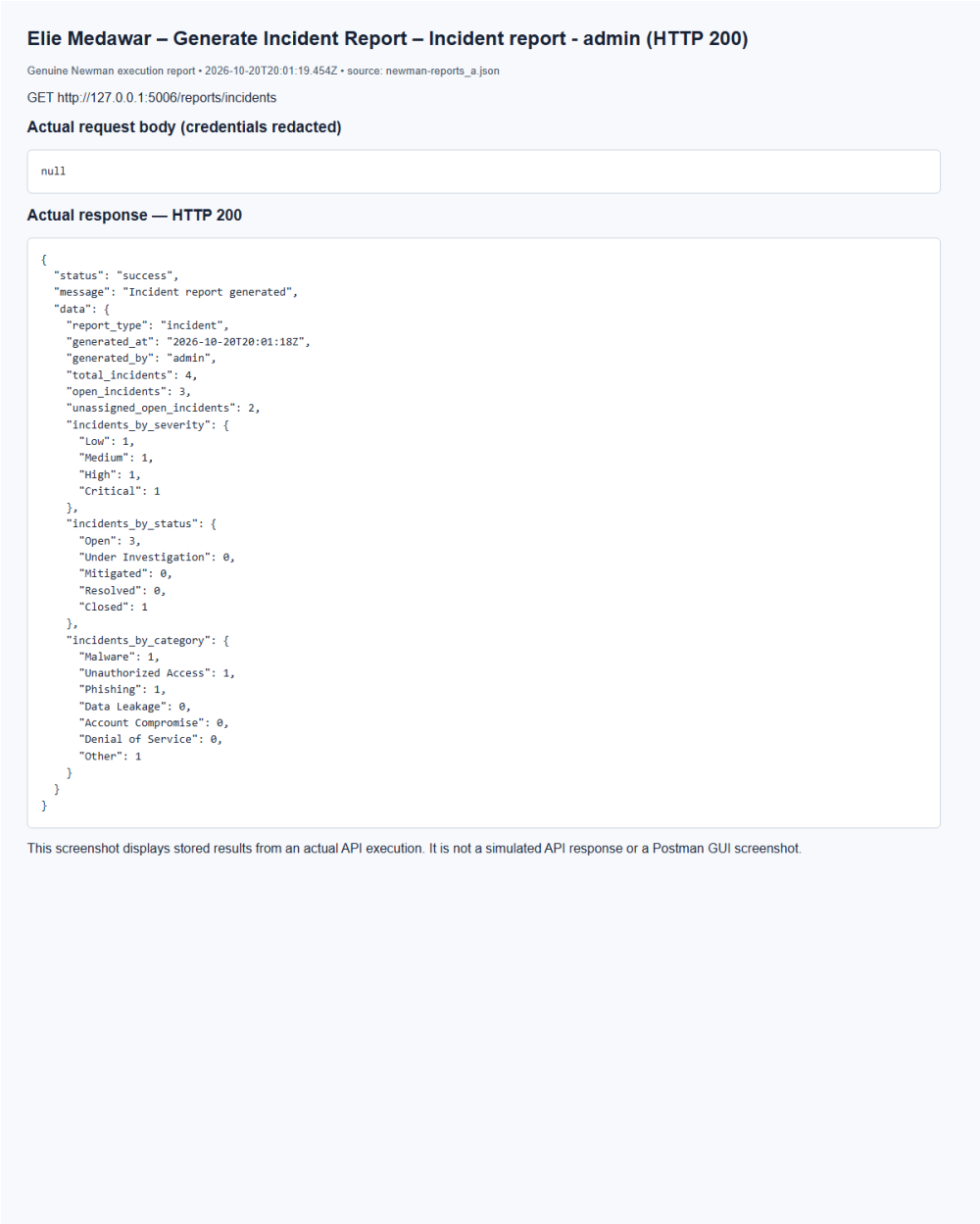
How the information is collected. ReportBuilder.collect() calls each source service's /internal/stats. Those endpoints accept only the X-Service-Key header, so users cannot call them directly. Each service computes its counts with SQL COUNT … GROUP BY on its own tables, so the report values match the database exactly. For the summary, a source that is down shows as null and is listed under unavailable_sources instead of failing the whole report; the incident and activity reports have a single source, so they return 503 if it is unreachable. In the activity report, logins are counted from the LOGIN_SUCCESS, LOGIN_FAILED and LOGIN_BLOCKED actions, unauthorized attempts from UNAUTHORIZED_ACCESS, and the most performed actions from a GROUP BY action ORDER BY count DESC LIMIT 5 query on logs.

Elie Medawar – Generate Security Summary Report – Summary - admin (HTTP 200)



(Output shortened; the full capture is report/screenshots/elie/newman-reports_a-05.png in the submission.)

Elie Medawar – Generate Incident Report – Incident report - admin (HTTP 200)



Elie Medawar – Generate Incident Report – Incident report - admin (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:19.454Z • source: newman-reports_a.json

GET http://127.0.0.1:5006/reports/incidents

Actual request body (credentials redacted)

```
null
```

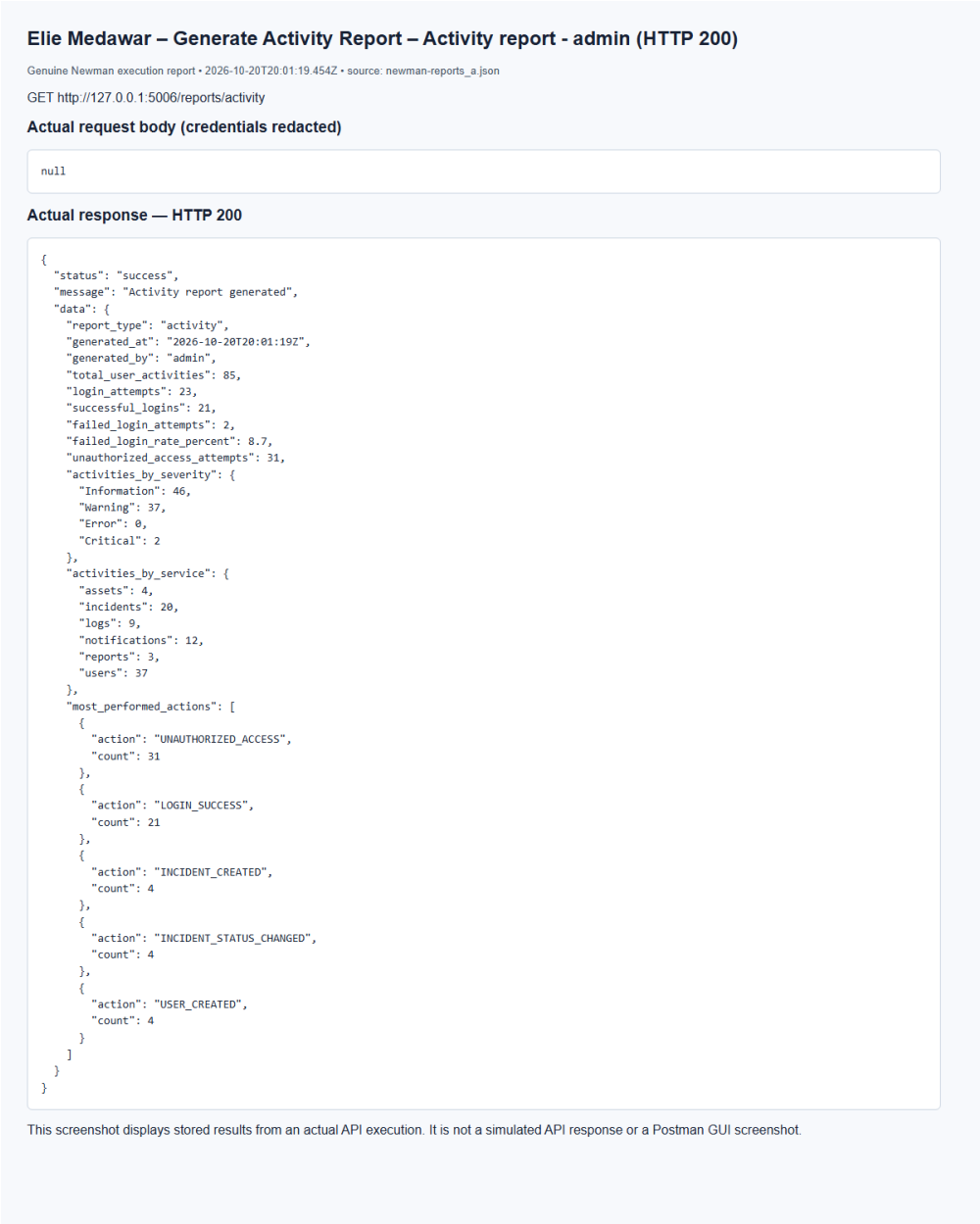
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Incident report generated",
  "data": {
    "report_type": "incident",
    "generated_at": "2026-10-20T20:01:18Z",
    "generated_by": "admin",
    "total_incidents": 4,
    "open_incidents": 3,
    "unassigned_open_incidents": 2,
    "incidents_by_severity": {
      "Low": 1,
      "Medium": 1,
      "High": 1,
      "Critical": 1
    },
    "incidents_by_status": {
      "Open": 3,
      "Under Investigation": 0,
      "Mitigated": 0,
      "Resolved": 0,
      "Closed": 1
    },
    "incidents_by_category": {
      "Malware": 1,
      "Unauthorized Access": 1,
      "Phishing": 1,
      "Data Leakage": 0,
      "Account Compromise": 0,
      "Denial of Service": 0,
      "Other": 1
    }
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-reports_a-10.png in the submission.)

Elie Medawar – Generate Activity Report – Activity report - admin (HTTP 200)



Elie Medawar – Generate Activity Report – Activity report - admin (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:19.454Z • source: newman-reports_a.json

GET http://127.0.0.1:5006/reports/activity

Actual request body (credentials redacted)

```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Activity report generated",
  "data": {
    "report_type": "activity",
    "generated_at": "2026-10-20T20:01:19Z",
    "generated_by": "admin",
    "total_user_activities": 85,
    "login_attempts": 23,
    "successful_logins": 21,
    "failed_login_attempts": 2,
    "failed_login_rate_percent": 8.7,
    "unauthorized_access_attempts": 31,
    "activities_by_severity": {
      "Information": 46,
      "Warning": 37,
      "Error": 0,
      "Critical": 2
    },
    "activities_by_service": {
      "assets": 4,
      "incidents": 20,
      "logs": 9,
      "notifications": 12,
      "reports": 3,
      "users": 37
    },
    "most_performed_actions": [
      {
        "action": "UNAUTHORIZED_ACCESS",
        "count": 31
      },
      {
        "action": "LOGIN_SUCCESS",
        "count": 21
      },
      {
        "action": "INCIDENT_CREATED",
        "count": 4
      },
      {
        "action": "INCIDENT_STATUS_CHANGED",
        "count": 4
      },
      {
        "action": "USER_CREATED",
        "count": 4
      }
    ]
  }
}
```

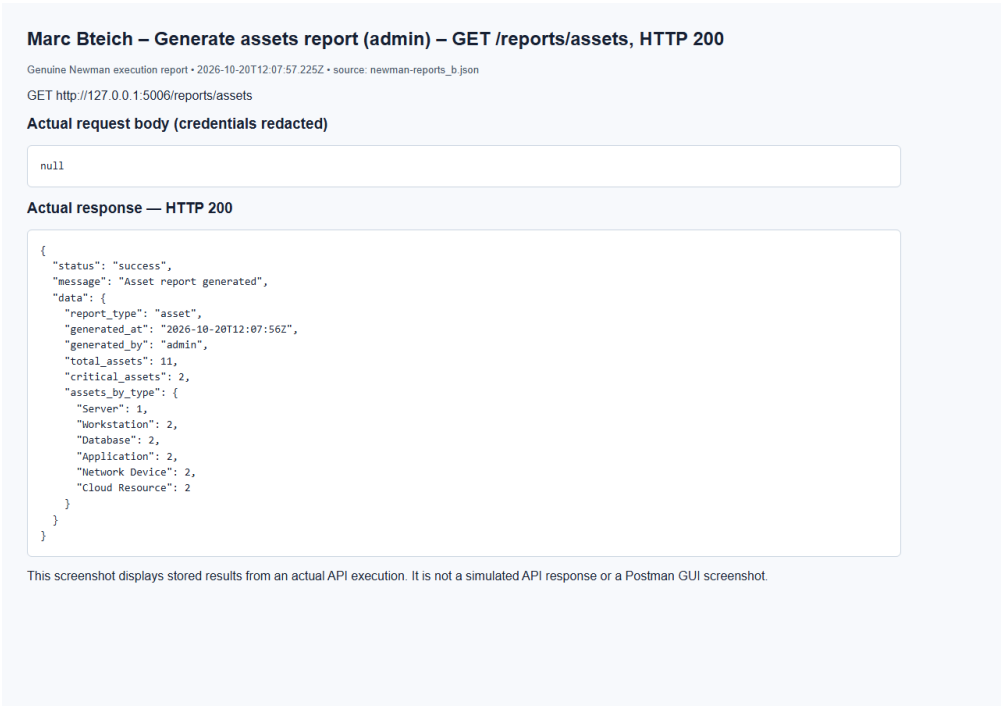

This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-reports_a-13.png in the submission.)

5.6 Security Reports Service – Asset, Threat and Vulnerability Reports (Marc Bteich)

The asset report returns total/critical assets and counts by type; the threat report returns total, counts by severity/type and the latest five records; the vulnerability report returns total/critical vulnerabilities and counts by severity/status. Source services calculate grouped counts with ORM aggregate queries. Admins and auditors may generate reports; analysts and regular users receive 403. Elie’s summary/incidents/activity routes are integrated unchanged.

Marc Bteich – Generate assets report (admin) – GET /reports/assets, HTTP 200



Marc Bteich – Generate assets report (admin) – GET /reports/assets, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:57.225Z • source: newman-reports_b.json

GET http://127.0.0.1:5006/reports/assets

Actual request body (credentials redacted)

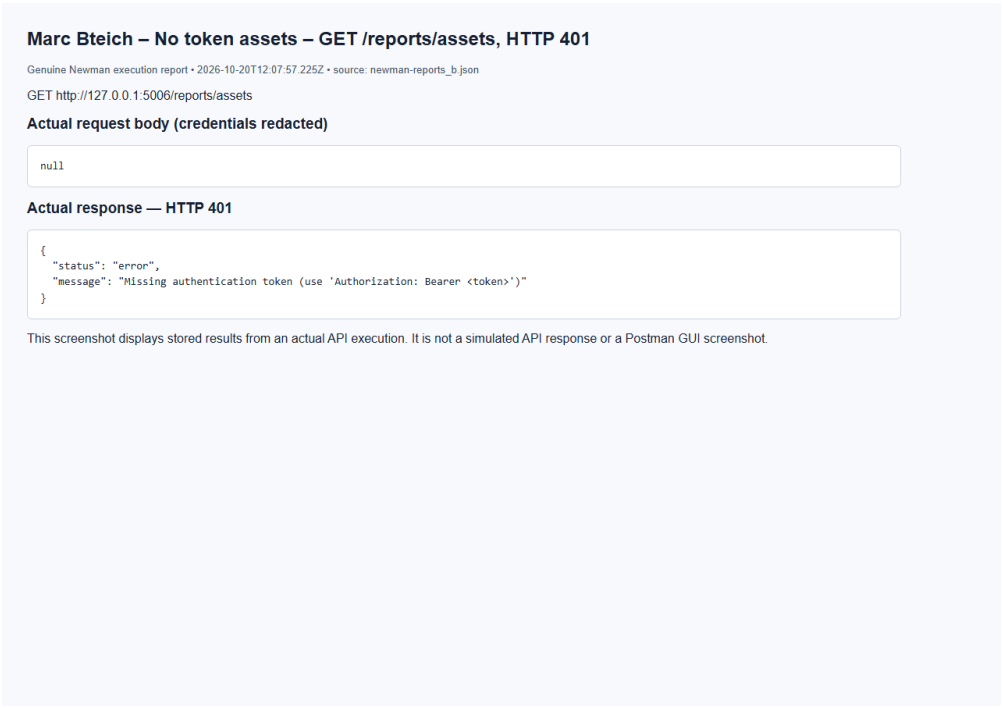
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Asset report generated",
  "data": {
    "report_type": "asset",
    "generated_at": "2026-10-20T12:07:56Z",
    "generated_by": "admin",
    "total_assets": 11,
    "critical_assets": 2,
    "assets_by_type": {
      "Server": 1,
      "Workstation": 2,
      "Database": 2,
      "Application": 2,
      "Network Device": 2,
      "Cloud Resource": 2
    }
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – No token assets – GET /reports/assets, HTTP 401



Marc Bteich – No token assets – GET /reports/assets, HTTP 401

Genuine Newman execution report • 2026-10-20T12:07:57.225Z • source: newman-reports_b.json

GET http://127.0.0.1:5006/reports/assets

Actual request body (credentials redacted)

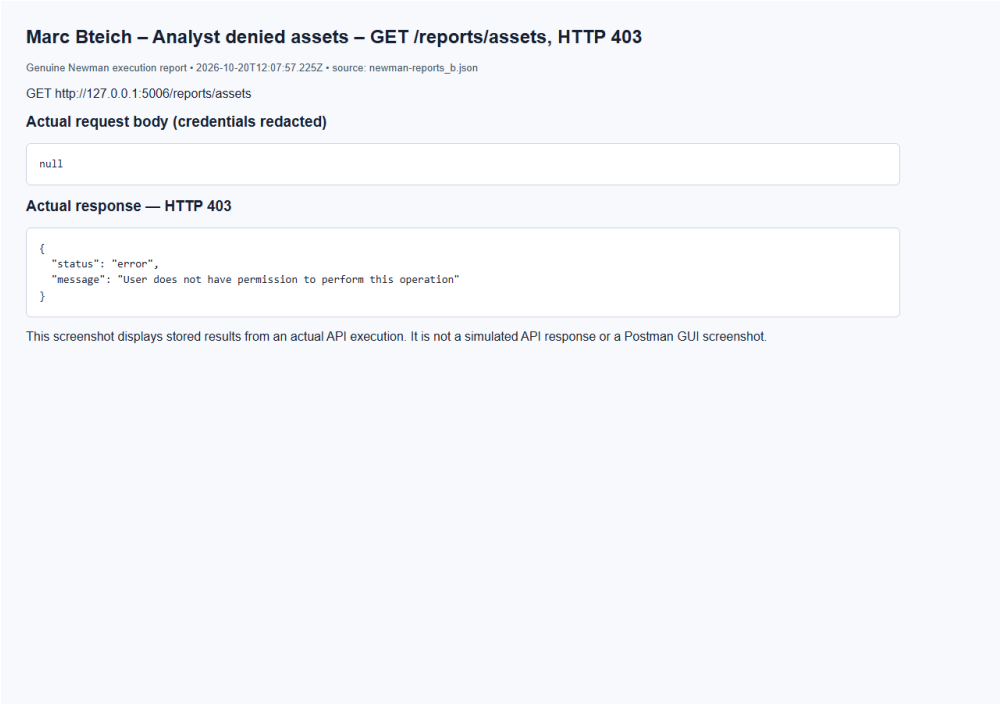
```
null
```

Actual response — HTTP 401


```
{
  "status": "error",
  "message": "Missing authentication token (use 'Authorization: Bearer <token>')"
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

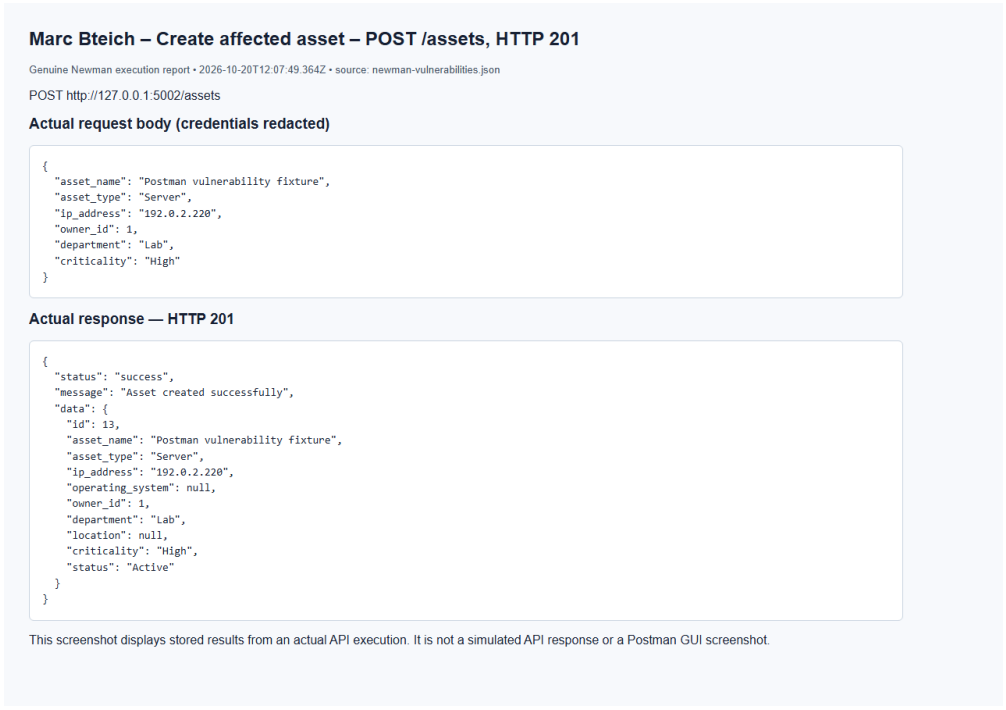
Marc Bteich – Analyst denied assets – GET /reports/assets, HTTP 403



5.7 Vulnerability Management Service (Marc Bteich)

VulnerabilityValidator checks required fields, a CVE-YYYY-NNNN+ identifier, finite numeric CVSS 0–10 and a real asset. Only admins create/delete, while analysts may update. Status advances one step at a time through Open, Under Investigation, In Progress, Resolved and Closed. The server records resolution_date on Resolved. Invalid transitions are rejected before applying any field changes, so a rejected update cannot partially save a description.

Marc Bteich – Create affected asset – POST /assets, HTTP 201



Marc Bteich – Create affected asset – POST /assets, HTTP 201

Genuine Newman execution report • 2026-10-20T12:07:49.364Z • source: newman-vulnerabilities.json

POST http://127.0.0.1:5002/assets

Actual request body (credentials redacted)

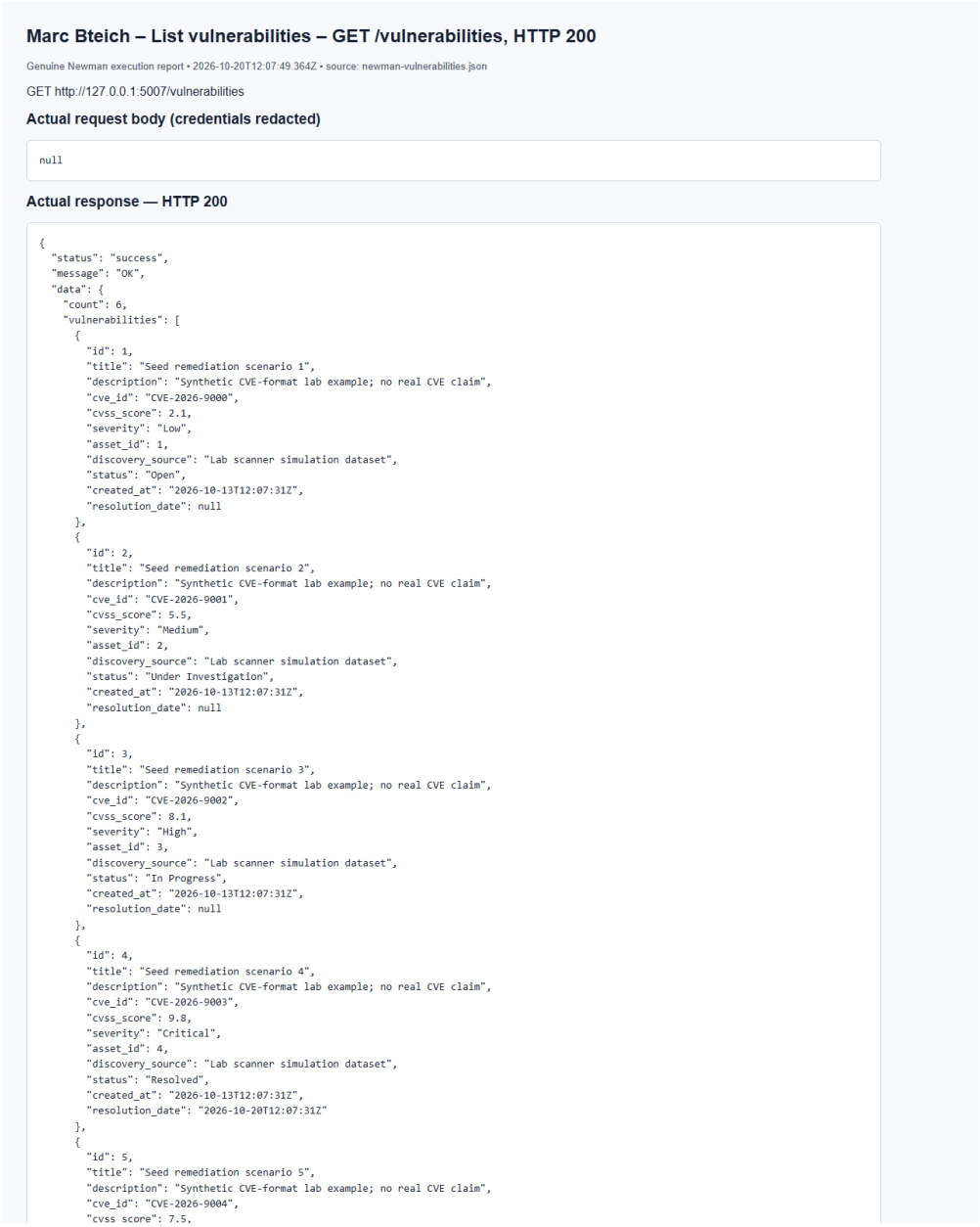

```
{
  "asset_name": "Postman vulnerability fixture",
  "asset_type": "Server",
  "ip_address": "192.0.2.220",
  "owner_id": 1,
  "department": "Lab",
  "criticality": "High"
}
```


Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Asset created successfully",
  "data": {
    "id": 13,
    "asset_name": "Postman vulnerability fixture",
    "asset_type": "Server",
    "ip_address": "192.0.2.220",
    "operating_system": null,
    "owner_id": 1,
    "department": "Lab",
    "location": null,
    "criticality": "High",
    "status": "Active"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – List vulnerabilities – GET /vulnerabilities, HTTP 200



Marc Bteich – List vulnerabilities – GET /vulnerabilities, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:49.364Z • source: newman-vulnerabilities.json

GET http://127.0.0.1:5007/vulnerabilities

Actual request body (credentials redacted)

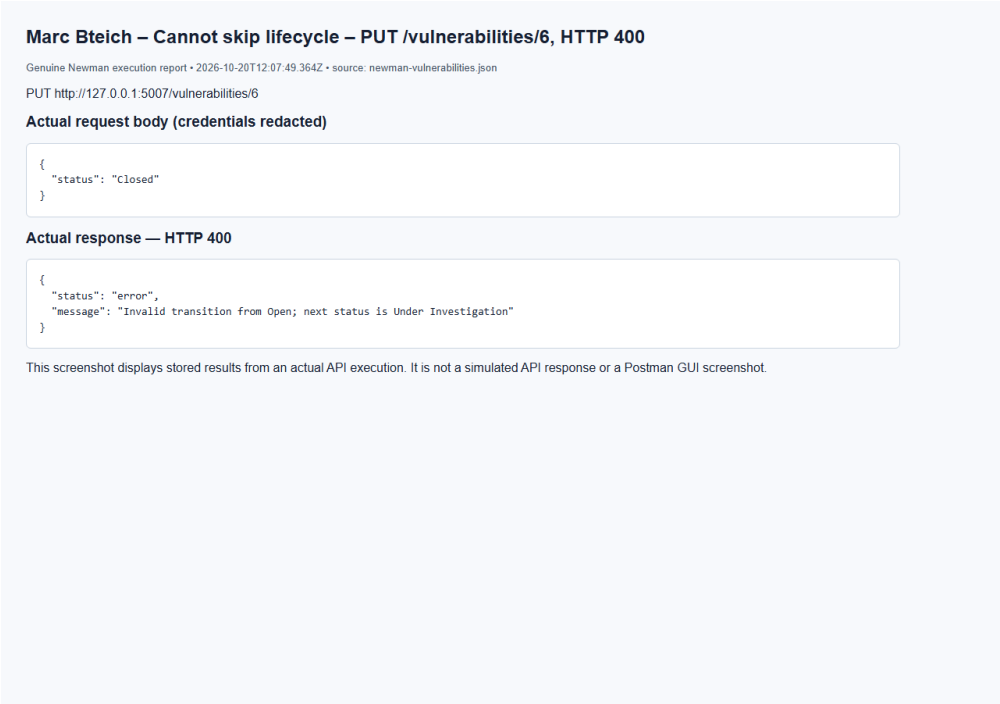
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 6,
    "vulnerabilities": [
      {
        "id": 1,
        "title": "Seed remediation scenario 1",
        "description": "Synthetic CVE-format lab example; no real CVE claim",
        "cve_id": "CVE-2026-9000",
        "cvss_score": 2.1,
        "severity": "Low",
        "asset_id": 1,
        "discovery_source": "Lab scanner simulation dataset",
        "status": "Open",
        "created_at": "2026-10-13T12:07:31Z",
        "resolution_date": null
      },
      {
        "id": 2,
        "title": "Seed remediation scenario 2",
        "description": "Synthetic CVE-format lab example; no real CVE claim",
        "cve_id": "CVE-2026-9001",
        "cvss_score": 5.5,
        "severity": "Medium",
        "asset_id": 2,
        "discovery_source": "Lab scanner simulation dataset",
        "status": "Under Investigation",
        "created_at": "2026-10-13T12:07:31Z",
        "resolution_date": null
      },
      {
        "id": 3,
        "title": "Seed remediation scenario 3",
        "description": "Synthetic CVE-format lab example; no real CVE claim",
        "cve_id": "CVE-2026-9002",
        "cvss_score": 8.1,
        "severity": "High",
        "asset_id": 3,
        "discovery_source": "Lab scanner simulation dataset",
        "status": "In Progress",
        "created_at": "2026-10-13T12:07:31Z",
        "resolution_date": null
      },
      {
        "id": 4,
        "title": "Seed remediation scenario 4",
        "description": "Synthetic CVE-format lab example; no real CVE claim",
        "cve_id": "CVE-2026-9003",
        "cvss_score": 9.8,
        "severity": "Critical",
        "asset_id": 4,
        "discovery_source": "Lab scanner simulation dataset",
        "status": "Resolved",
        "created_at": "2026-10-13T12:07:31Z",
        "resolution_date": "2026-10-20T12:07:31Z"
      },
      {
        "id": 5,
        "title": "Seed remediation scenario 5",
        "description": "Synthetic CVE-format lab example; no real CVE claim",
        "cve_id": "CVE-2026-9004",
        "cvss_score": 7.5,
```


(Output shortened; the full capture is report/screenshots/marc/newman-vulnerabilities-07.png in the submission.)

Marc Bteich – Cannot skip lifecycle – PUT /vulnerabilities/6, HTTP 400



Marc Bteich – Cannot skip lifecycle – PUT /vulnerabilities/6, HTTP 400

Genuine Newman execution report • 2026-10-20T12:07:49.364Z • source: newman-vulnerabilities.json

PUT http://127.0.0.1:5007/vulnerabilities/6

Actual request body (credentials redacted)

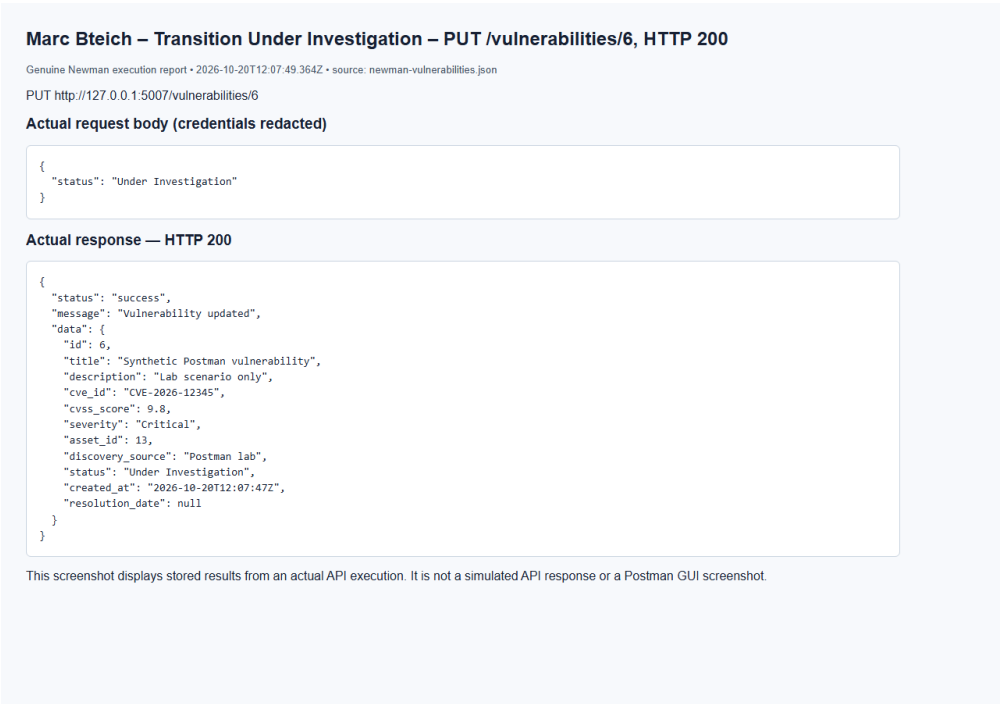
```
{
  "status": "Closed"
}
```

Actual response — HTTP 400

```
{
  "status": "error",
  "message": "Invalid transition from Open; next status is Under Investigation"
}
```

This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Transition Under Investigation – PUT /vulnerabilities/6, HTTP 200



Marc Bteich – Transition Under Investigation – PUT /vulnerabilities/6, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:49.364Z • source: newman-vulnerabilities.json

PUT http://127.0.0.1:5007/vulnerabilities/6

Actual request body (credentials redacted)

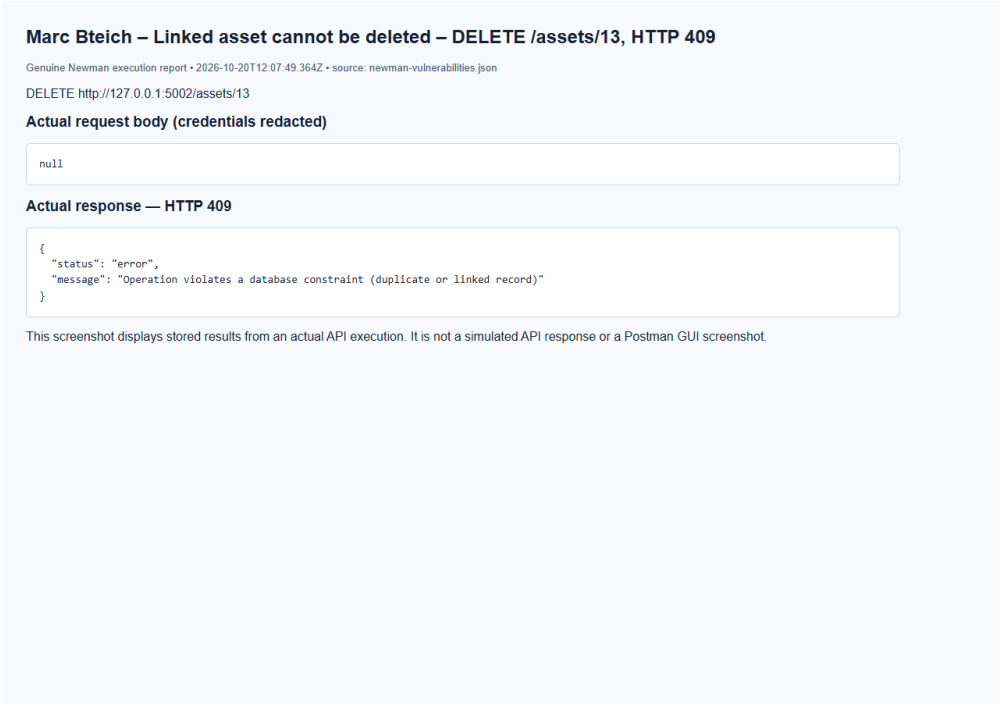
```
{
  "status": "Under Investigation"
}
```

Actual response — HTTP 200

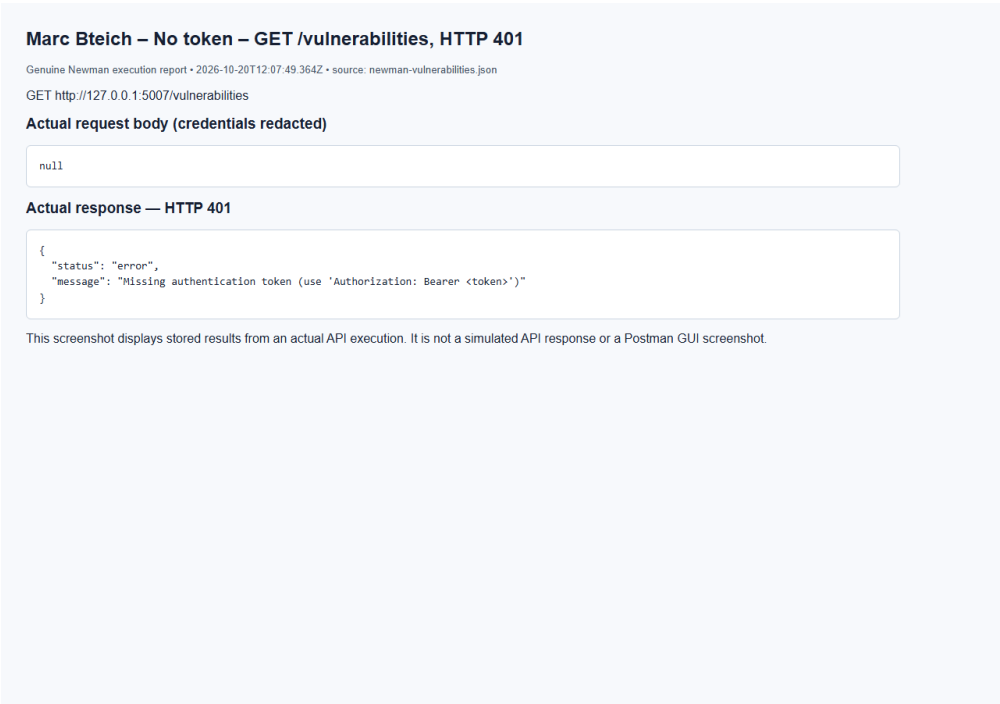
```
{
  "status": "success",
  "message": "Vulnerability updated",
  "data": {
    "id": 6,
    "title": "Synthetic Postman vulnerability",
    "description": "Lab scenario only",
    "cve_id": "CVE-2026-12345",
    "cvss_score": 9.8,
    "severity": "Critical",
    "asset_id": 13,
    "discovery_source": "Postman lab",
    "status": "Under Investigation",
    "created_at": "2026-10-20T12:07:47Z",
    "resolution_date": null
  }
}
```

This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Linked asset cannot be deleted – DELETE /assets/13, HTTP 409



Marc Bteich – No token – GET /vulnerabilities, HTTP 401



5.8 Notification and Alert Management Service (Elie Medawar)

The Notifications service (port 5008) stores alerts for users and tracks whether each one has been read. Other services create notifications through it, for example when an incident is

assigned. Code: `services/notifications/` (`NotificationService`) and `models/notifications.py`.

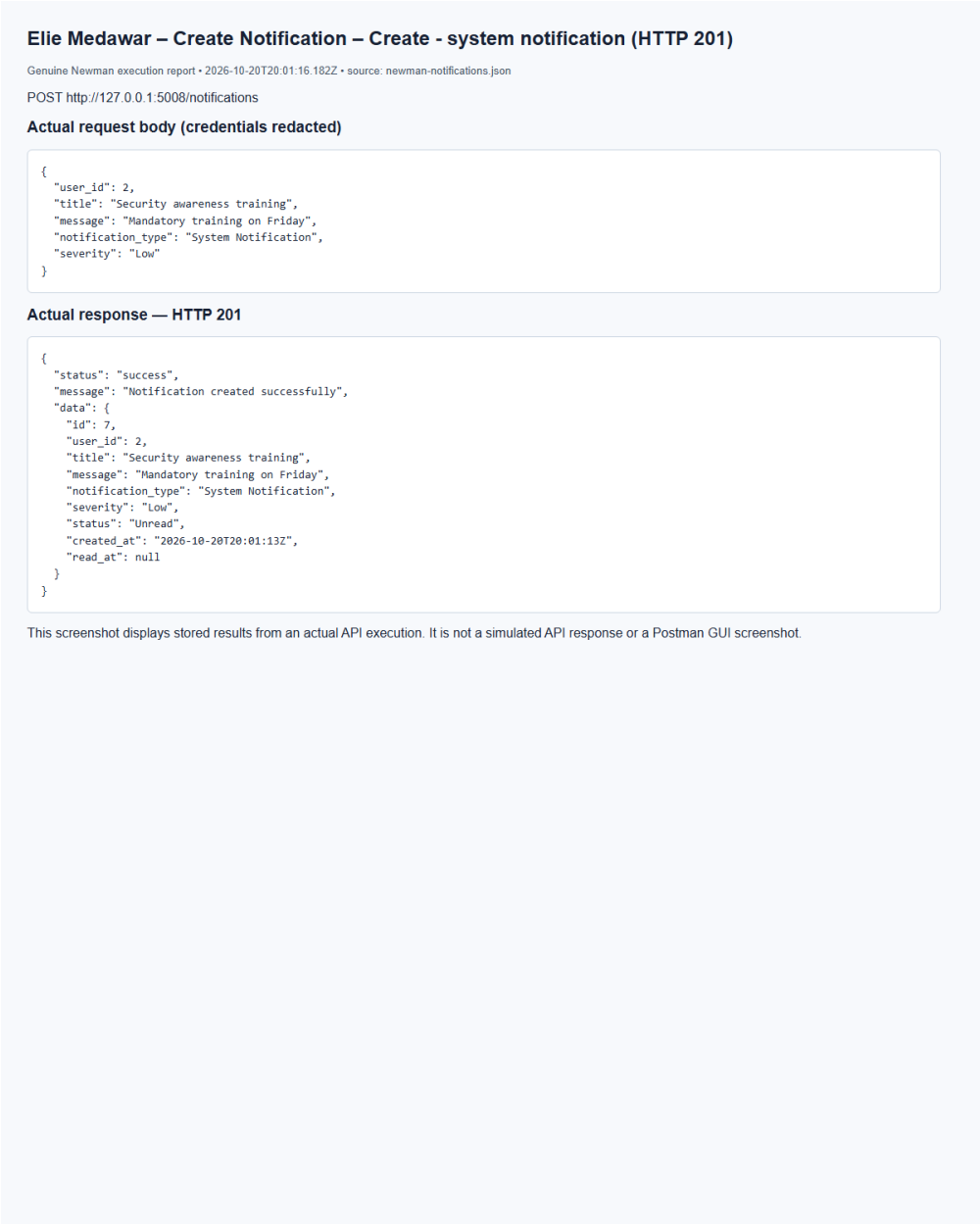
API	Method	Endpoint	Allowed roles	Success
Create Notification	POST	/notifications	admin; other services (X-Service-Key)	201
Retrieve User Notifications	GET	/notifications/user/{user_id} (optional ?status=)	admin, auditor (any user); analyst, user (own only)	200
Retrieve Specific Notification	GET	/notifications/{id}	admin, auditor; analyst, user (own only)	200
Mark Notification as Read	PUT	/notifications/{id}/read	admin; analyst, user (own only)	200
Search Notifications	GET	/notifications/search	admin, auditor; analyst (own results only)	200
Delete Notification	DELETE	/notifications/{id}	admin	200

Trigger	Type	Recipient
An admin assigns an incident	Incident Assignment	The assigned analyst
An incident's status changes	System Notification	The user who reported it
A Critical threat indicator is added	Threat Alert	Every admin (Marc Bteich's Threats service)
A Critical vulnerability is added	Vulnerability Alert	Every admin (Marc Bteich's Vulnerabilities service)
An admin sends an announcement	Any type	The chosen user

Status handling. A notification starts as Unread with read_at empty. PUT /notifications/{id}/read sets it to Read and records read_at. Calling it again returns 200 with "Notification was already marked as read" and keeps the original read_at. Search filters (user, type, severity, status) are combined with AND; an analyst's search is always limited to their own notifications.

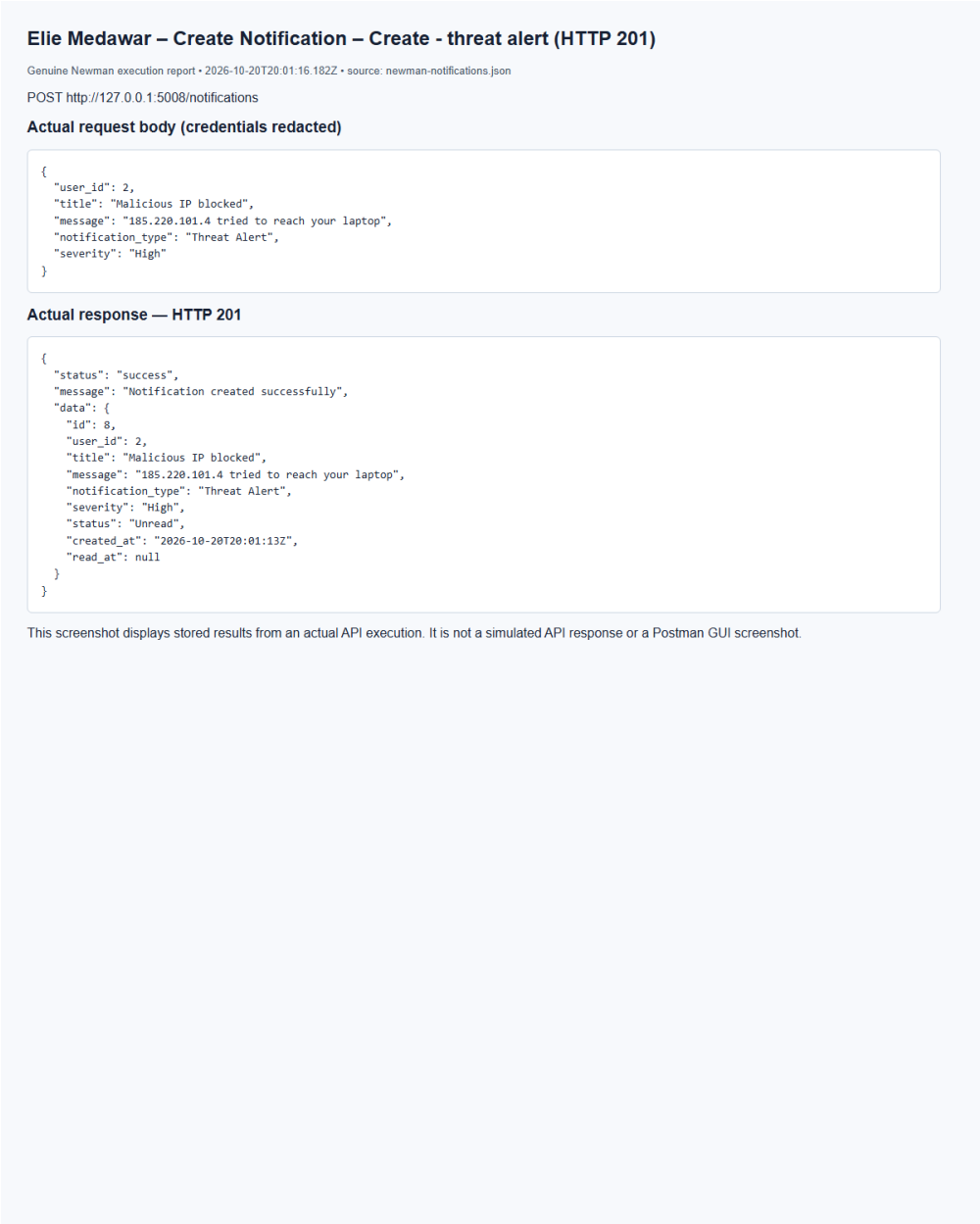
Database interaction. notifications: id (PK), user_id (FK → users, ON DELETE CASCADE, since notifications are personal), title, message, notification_type, severity, status (CHECK: Unread, Read), created_at, read_at. One user receives many notifications.

Elie Medawar – Create Notification – Create - system notification (HTTP 201)



(Output shortened; the full capture is report/screenshots/elie/newman-notifications-05.png in the submission.)

Elie Medawar – Create Notification – Create - threat alert (HTTP 201)



Elie Medawar – Create Notification – Create - threat alert (HTTP 201)

Genuine Newman execution report • 2026-10-20T20:01:16.182Z • source: newman-notifications.json

POST http://127.0.0.1:5008/notifications

Actual request body (credentials redacted)


```
{
  "user_id": 2,
  "title": "Malicious IP blocked",
  "message": "185.220.101.4 tried to reach your laptop",
  "notification_type": "Threat Alert",
  "severity": "High"
}
```

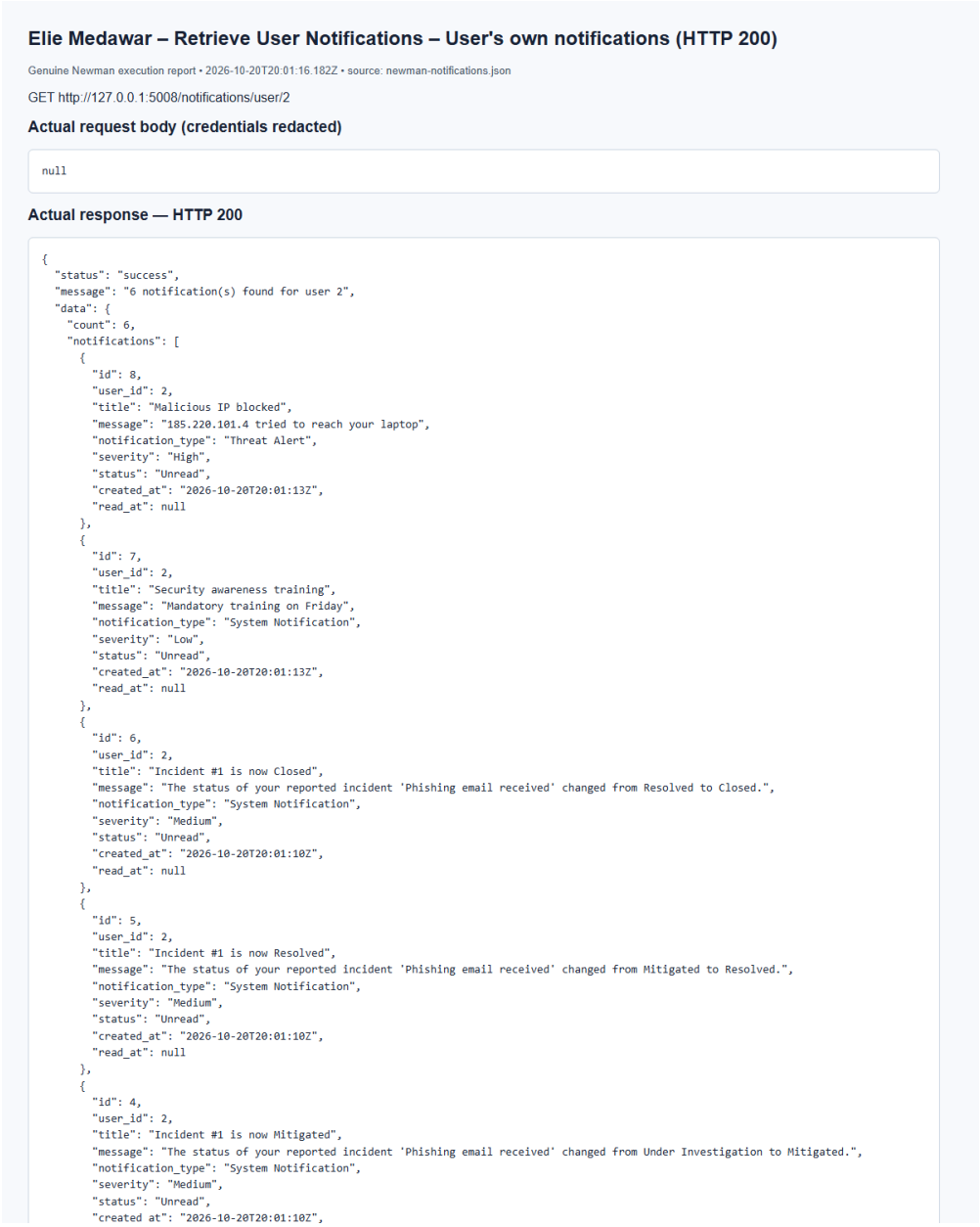

Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Notification created successfully",
  "data": {
    "id": 8,
    "user_id": 2,
    "title": "Malicious IP blocked",
    "message": "185.220.101.4 tried to reach your laptop",
    "notification_type": "Threat Alert",
    "severity": "High",
    "status": "Unread",
    "created_at": "2026-10-20T20:01:13Z",
    "read_at": null
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-notifications-06.png in the submission.)

Elie Medawar – Retrieve User Notifications – User's own notifications (HTTP 200)



Elie Medawar – Retrieve User Notifications – User's own notifications (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:16.182Z • source: newman-notifications.json

GET http://127.0.0.1:5008/notifications/user/2

Actual request body (credentials redacted)

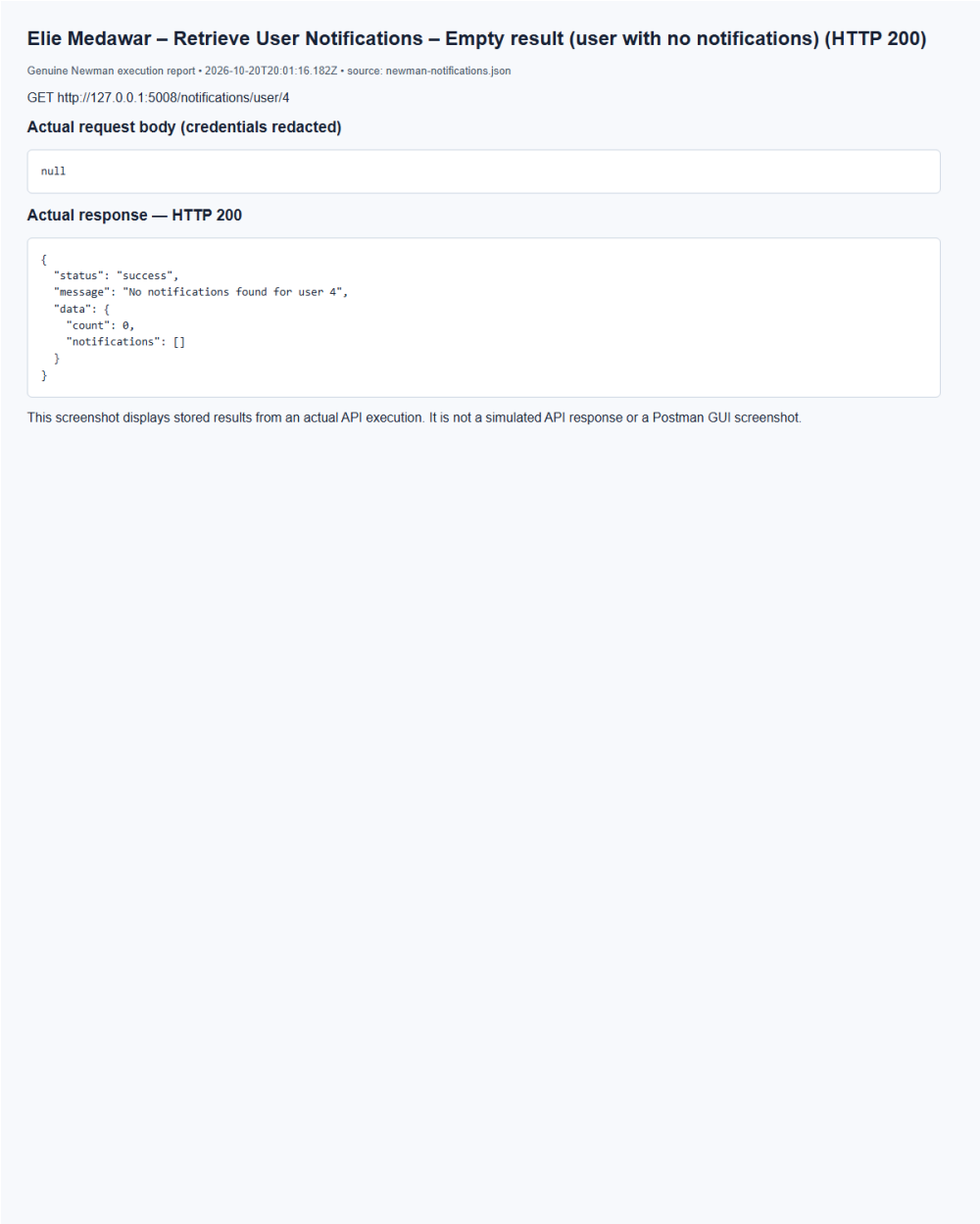
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "6 notification(s) found for user 2",
  "data": {
    "count": 6,
    "notifications": [
      {
        "id": 8,
        "user_id": 2,
        "title": "Malicious IP blocked",
        "message": "185.220.101.4 tried to reach your laptop",
        "notification_type": "Threat Alert",
        "severity": "High",
        "status": "Unread",
        "created_at": "2026-10-20T20:01:13Z",
        "read_at": null
      },
      {
        "id": 7,
        "user_id": 2,
        "title": "Security awareness training",
        "message": "Mandatory training on Friday",
        "notification_type": "System Notification",
        "severity": "Low",
        "status": "Unread",
        "created_at": "2026-10-20T20:01:13Z",
        "read_at": null
      },
      {
        "id": 6,
        "user_id": 2,
        "title": "Incident #1 is now Closed",
        "message": "The status of your reported incident 'Phishing email received' changed from Resolved to Closed.",
        "notification_type": "System Notification",
        "severity": "Medium",
        "status": "Unread",
        "created_at": "2026-10-20T20:01:10Z",
        "read_at": null
      },
      {
        "id": 5,
        "user_id": 2,
        "title": "Incident #1 is now Resolved",
        "message": "The status of your reported incident 'Phishing email received' changed from Mitigated to Resolved.",
        "notification_type": "System Notification",
        "severity": "Medium",
        "status": "Unread",
        "created_at": "2026-10-20T20:01:10Z",
        "read_at": null
      },
      {
        "id": 4,
        "user_id": 2,
        "title": "Incident #1 is now Mitigated",
        "message": "The status of your reported incident 'Phishing email received' changed from Under Investigation to Mitigated.",
        "notification_type": "System Notification",
        "severity": "Medium",
        "status": "Unread",
        "created at": "2026-10-20T20:01:10Z",
```

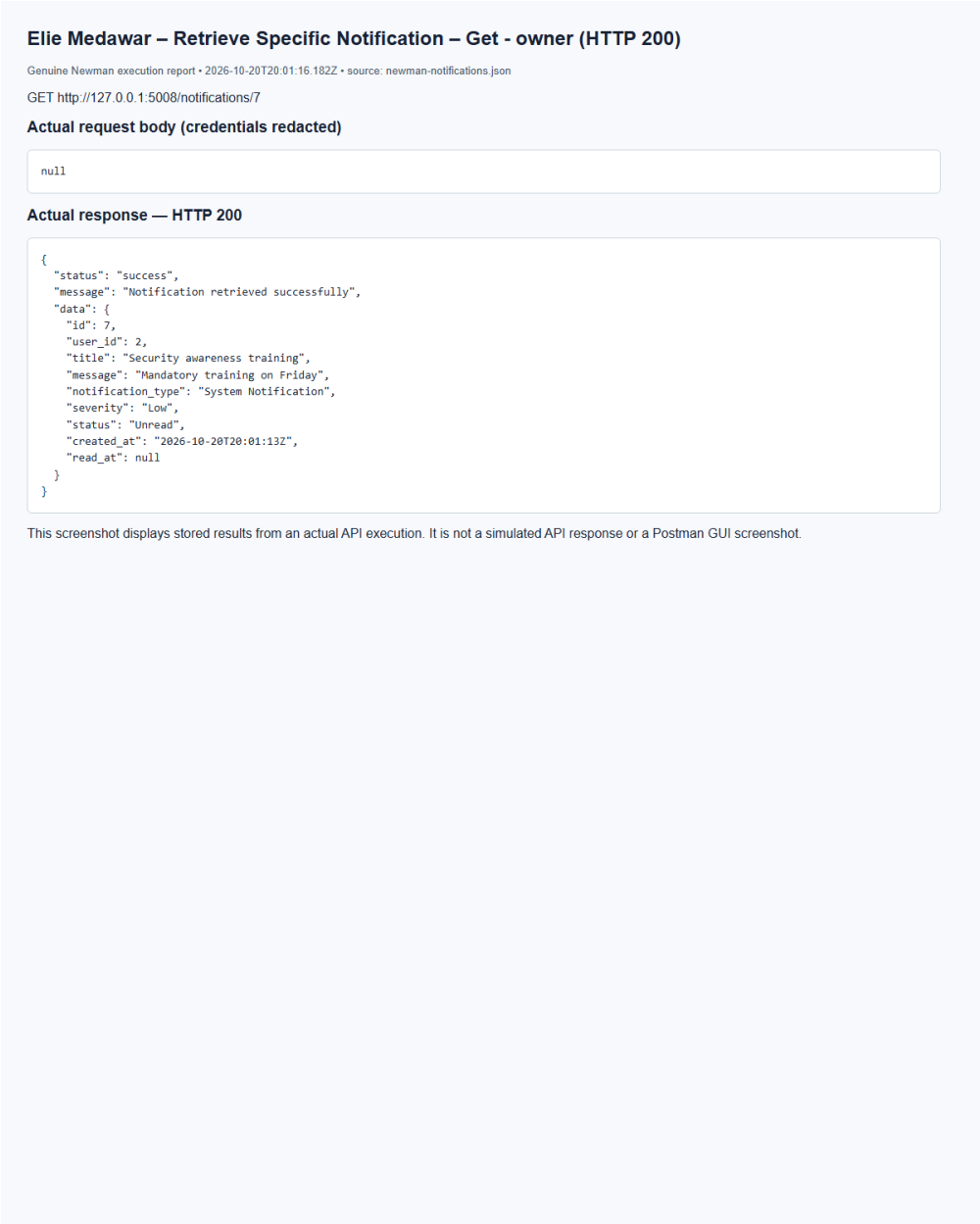

(Output shortened; the full capture is report/screenshots/elie/newman-notifications-12.png in the submission.)

Elie Medawar – Retrieve User Notifications – Empty result (user with no notifications) (HTTP 200)



(Output shortened; the full capture is report/screenshots/elie/newman-notifications-15.png in the submission.)

Elie Medawar – Retrieve Specific Notification – Get - owner (HTTP 200)



Elie Medawar – Retrieve Specific Notification – Get - owner (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:16.182Z • source: newman-notifications.json

GET http://127.0.0.1:5008/notifications/7

Actual request body (credentials redacted)

```
null
```

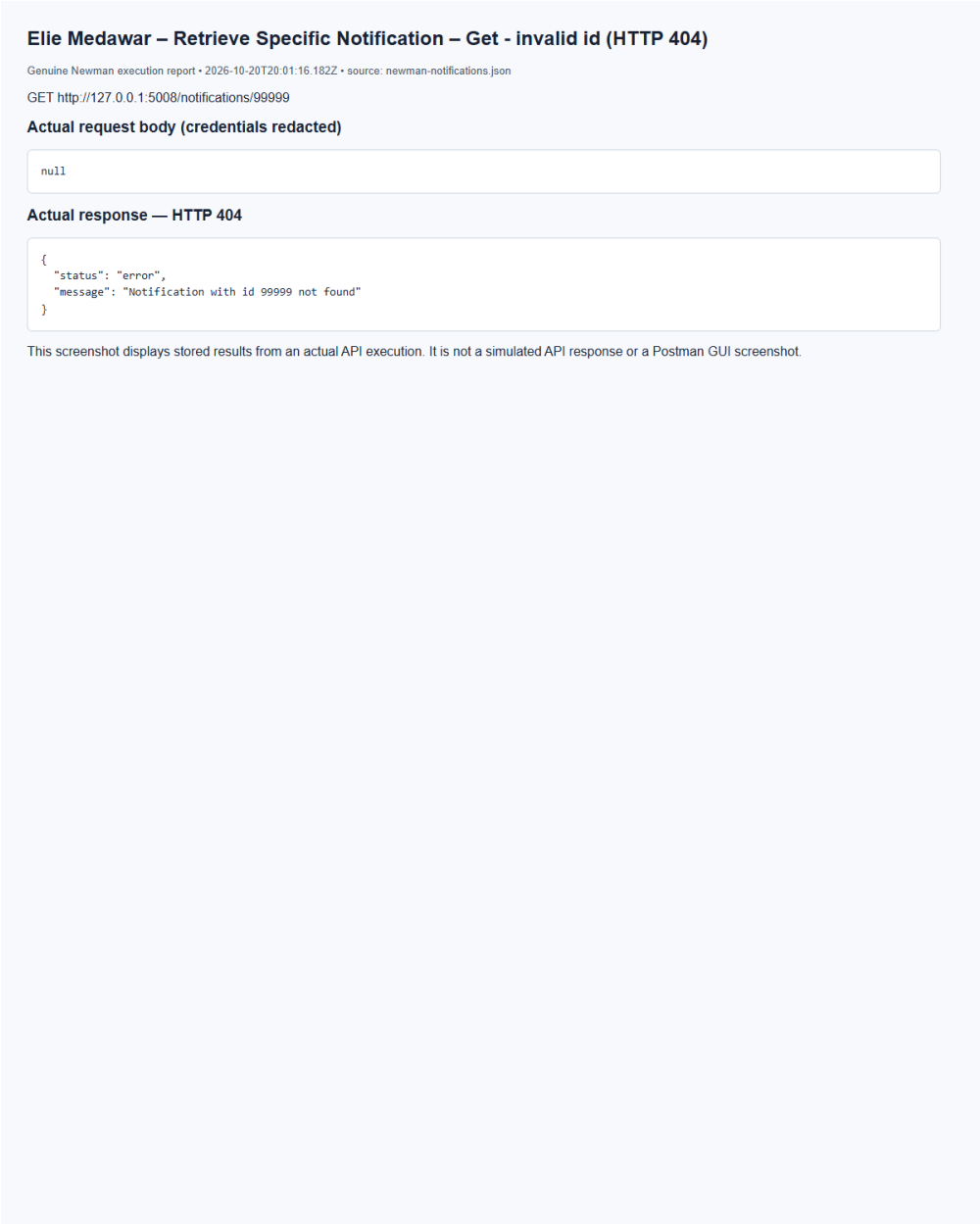
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Notification retrieved successfully",
  "data": {
    "id": 7,
    "user_id": 2,
    "title": "Security awareness training",
    "message": "Mandatory training on Friday",
    "notification_type": "System Notification",
    "severity": "Low",
    "status": "Unread",
    "created_at": "2026-10-20T20:01:13Z",
    "read_at": null
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

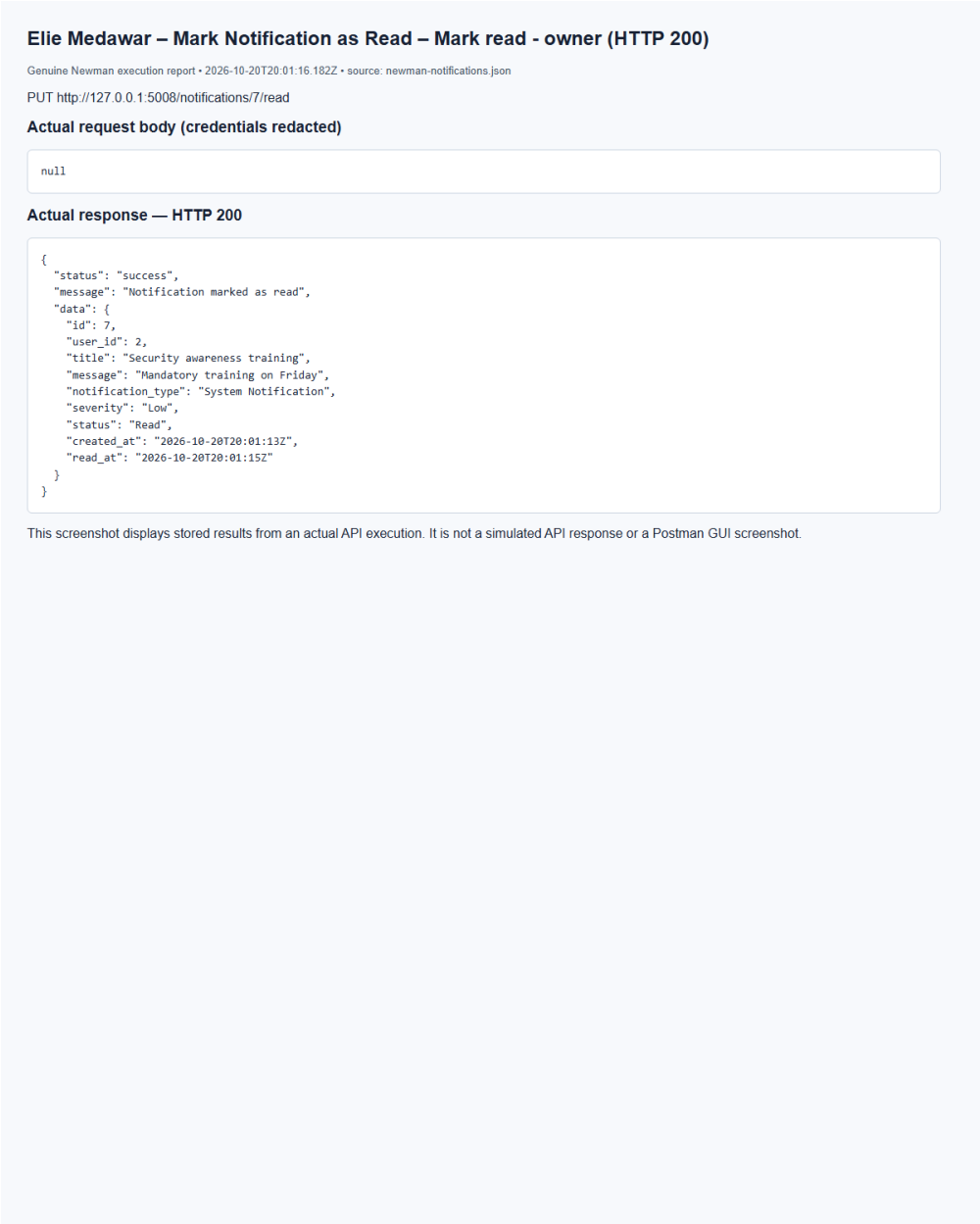
(Output shortened; the full capture is report/screenshots/elie/newman-notifications-17.png in the submission.)

Elie Medawar – Retrieve Specific Notification – Get - invalid id (HTTP 404)



(Output shortened; the full capture is report/screenshots/elie/newman-notifications-18.png in the submission.)

Elie Medawar – Mark Notification as Read – Mark read - owner (HTTP 200)



Elie Medawar – Mark Notification as Read – Mark read - owner (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:16.182Z • source: newman-notifications.json

PUT http://127.0.0.1:5008/notifications/7/read

Actual request body (credentials redacted)

```
null
```

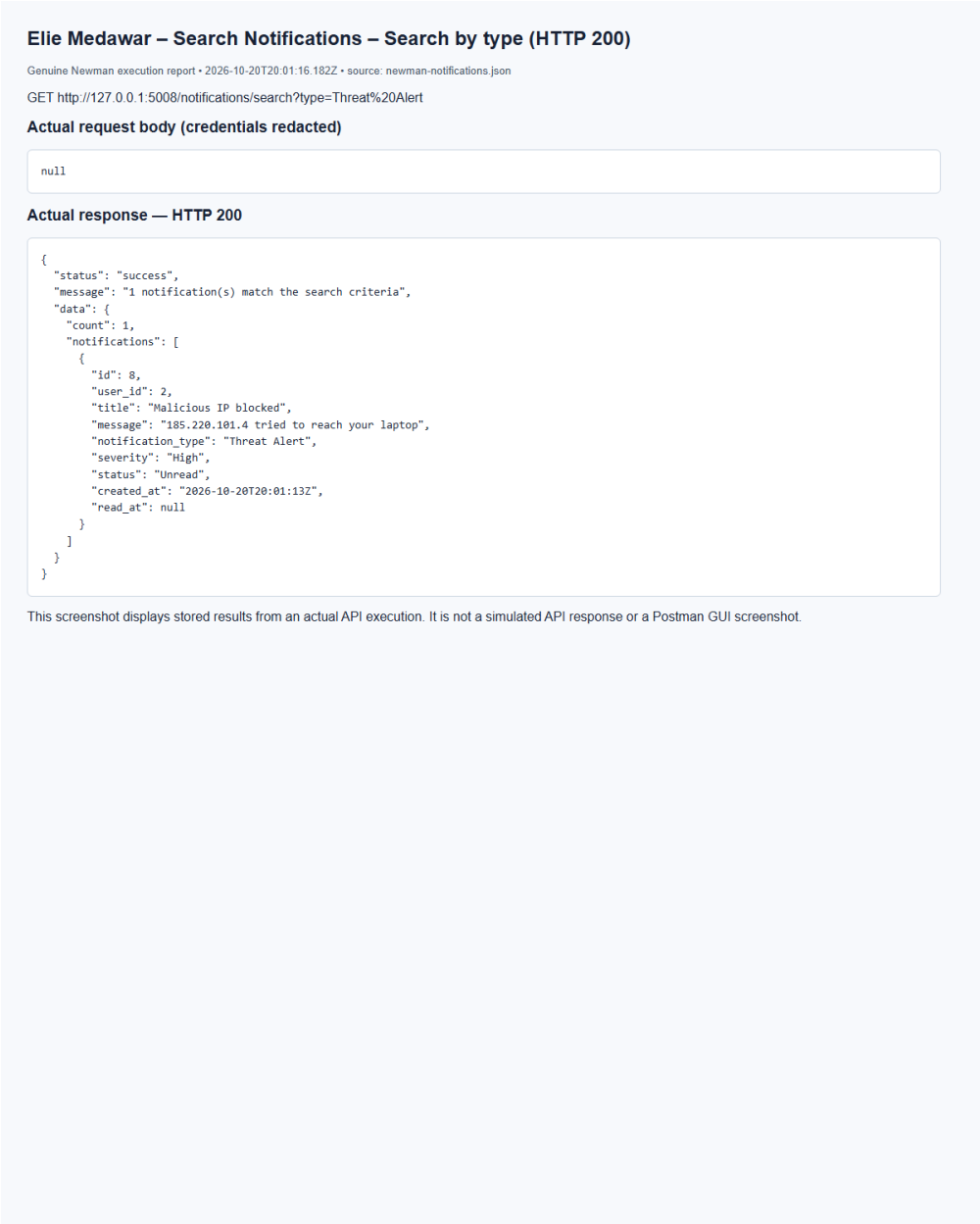
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Notification marked as read",
  "data": {
    "id": 7,
    "user_id": 2,
    "title": "Security awareness training",
    "message": "Mandatory training on Friday",
    "notification_type": "System Notification",
    "severity": "Low",
    "status": "Read",
    "created_at": "2026-10-20T20:01:13Z",
    "read_at": "2026-10-20T20:01:15Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-notifications-21.png in the submission.)

Elie Medawar – Search Notifications – Search by type (HTTP 200)



Elie Medawar – Search Notifications – Search by type (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:16.182Z • source: newman-notifications.json

GET http://127.0.0.1:5008/notifications/search?type=Threat%20Alert

Actual request body (credentials redacted)

```
null
```

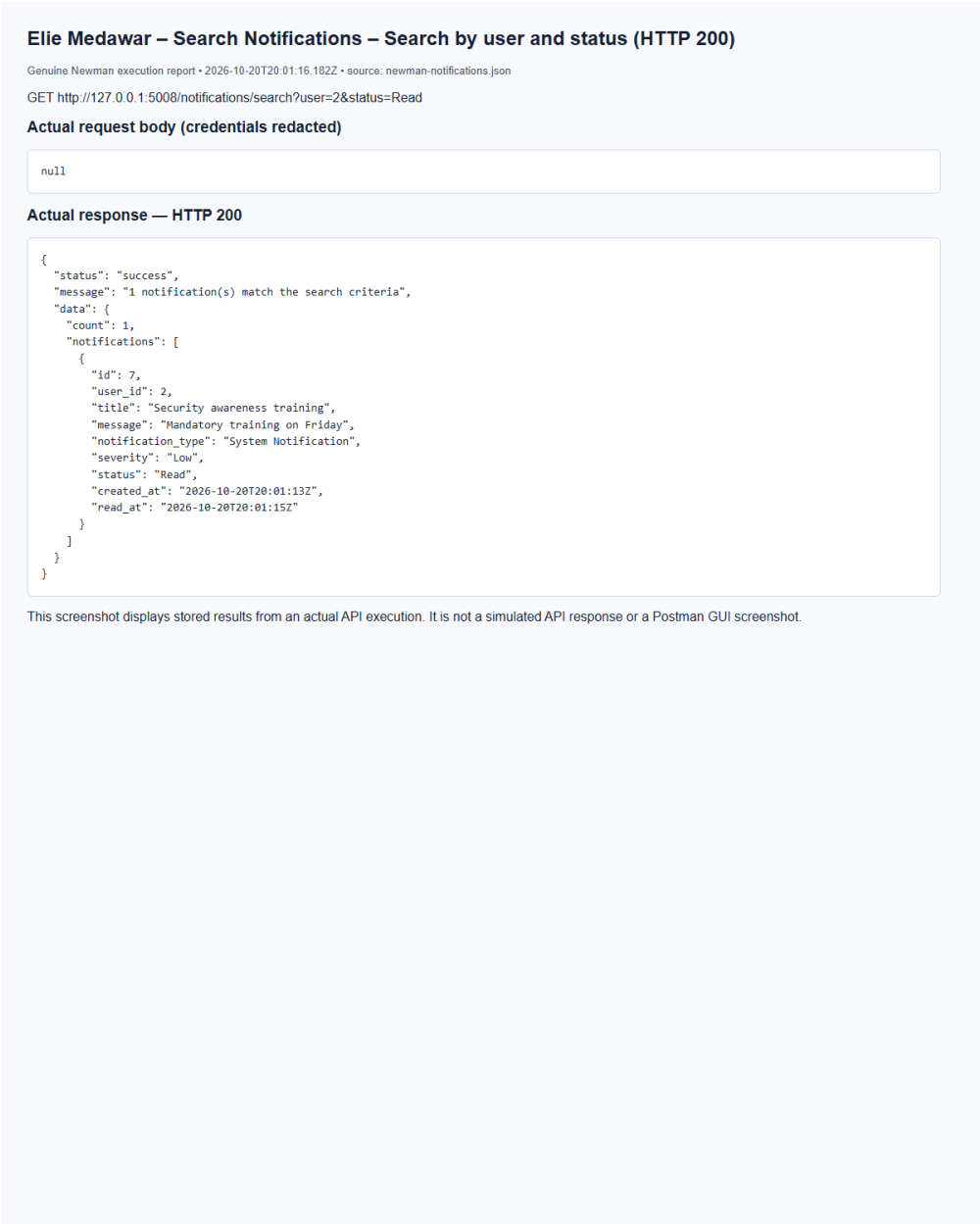
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "1 notification(s) match the search criteria",
  "data": {
    "count": 1,
    "notifications": [
      {
        "id": 8,
        "user_id": 2,
        "title": "Malicious IP blocked",
        "message": "185.220.101.4 tried to reach your laptop",
        "notification_type": "Threat Alert",
        "severity": "High",
        "status": "Unread",
        "created_at": "2026-10-20T20:01:13Z",
        "read_at": null
      }
    ]
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

(Output shortened; the full capture is report/screenshots/elie/newman-notifications-24.png in the submission.)

Elie Medawar – Search Notifications – Search by user and status (HTTP 200)



Elie Medawar – Search Notifications – Search by user and status (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:01:16.182Z • source: newman-notifications.json

GET http://127.0.0.1:5008/notifications/search?user=2&status=Read

Actual request body (credentials redacted)

```
null
```

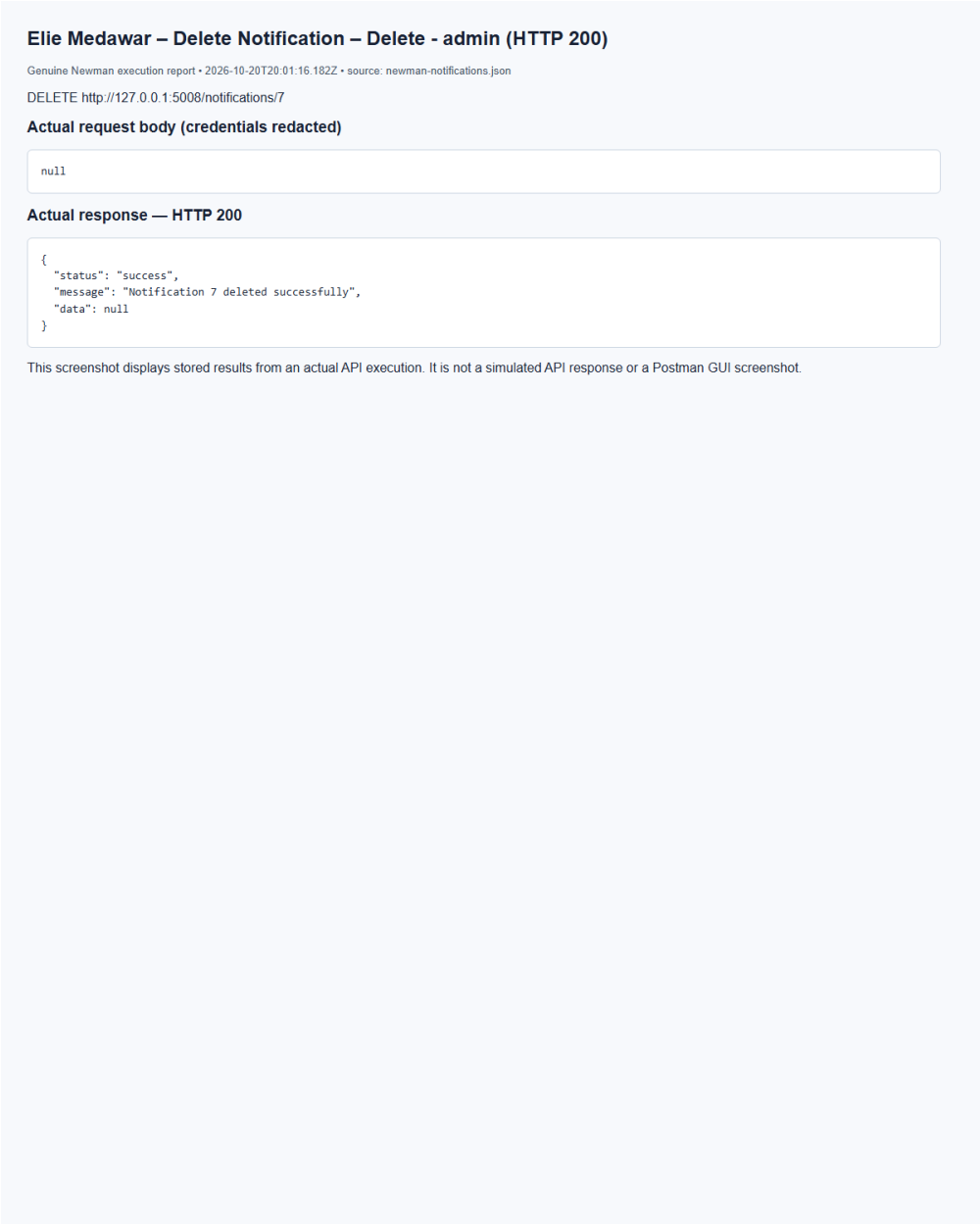
Actual response — HTTP 200


```
{
  "status": "success",
  "message": "1 notification(s) match the search criteria",
  "data": {
    "count": 1,
    "notifications": [
      {
        "id": 7,
        "user_id": 2,
        "title": "Security awareness training",
        "message": "Mandatory training on Friday",
        "notification_type": "System Notification",
        "severity": "Low",
        "status": "Read",
        "created_at": "2026-10-20T20:01:13Z",
        "read_at": "2026-10-20T20:01:15Z"
      }
    ]
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

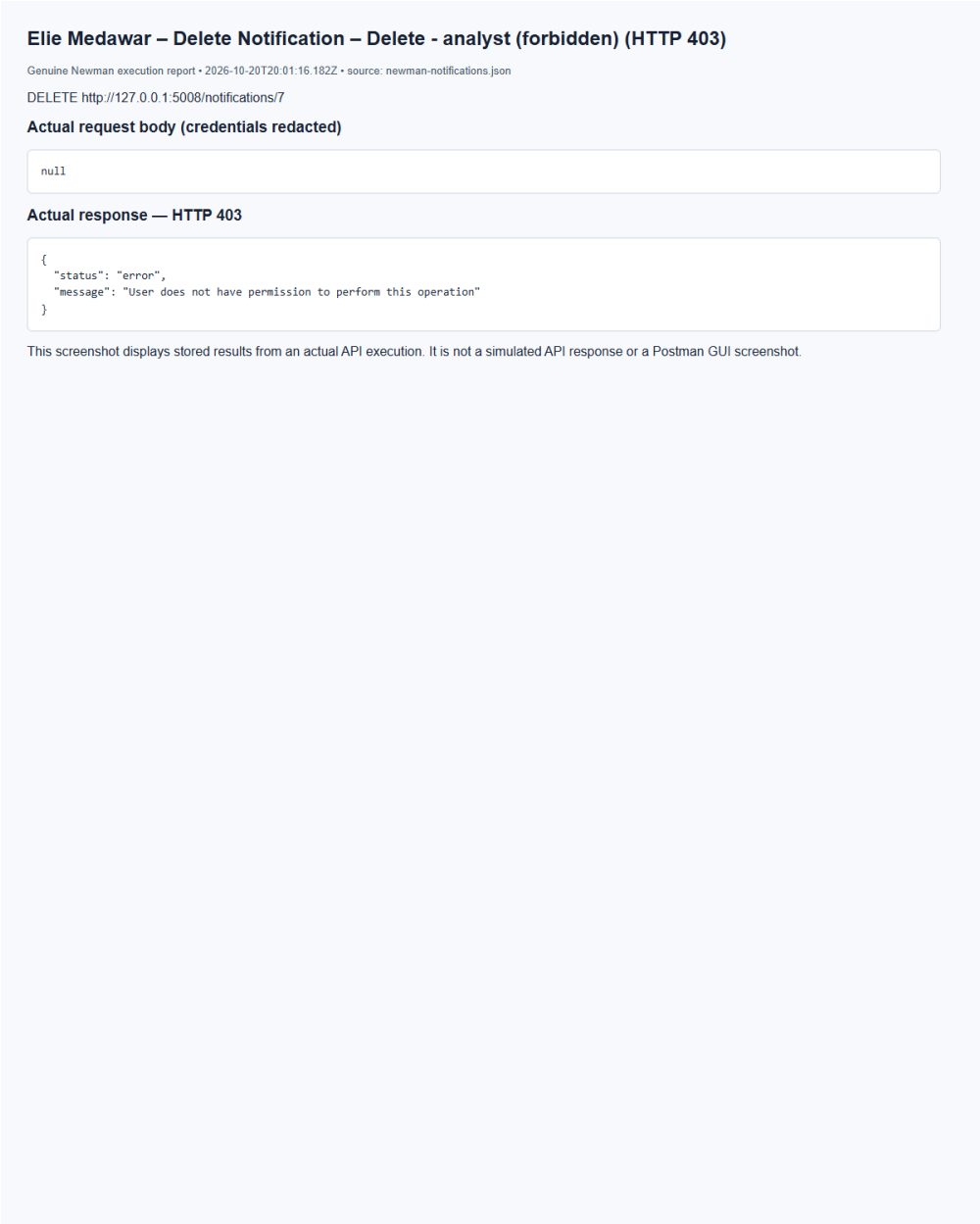
(Output shortened; the full capture is report/screenshots/elie/newman-notifications-25.png in the submission.)

Elie Medawar – Delete Notification – Delete - admin (HTTP 200)



(Output shortened; the full capture is report/screenshots/elie/newman-notifications-32.png in the submission.)

Elie Medawar – Delete Notification – Delete - analyst (forbidden) (HTTP 403)



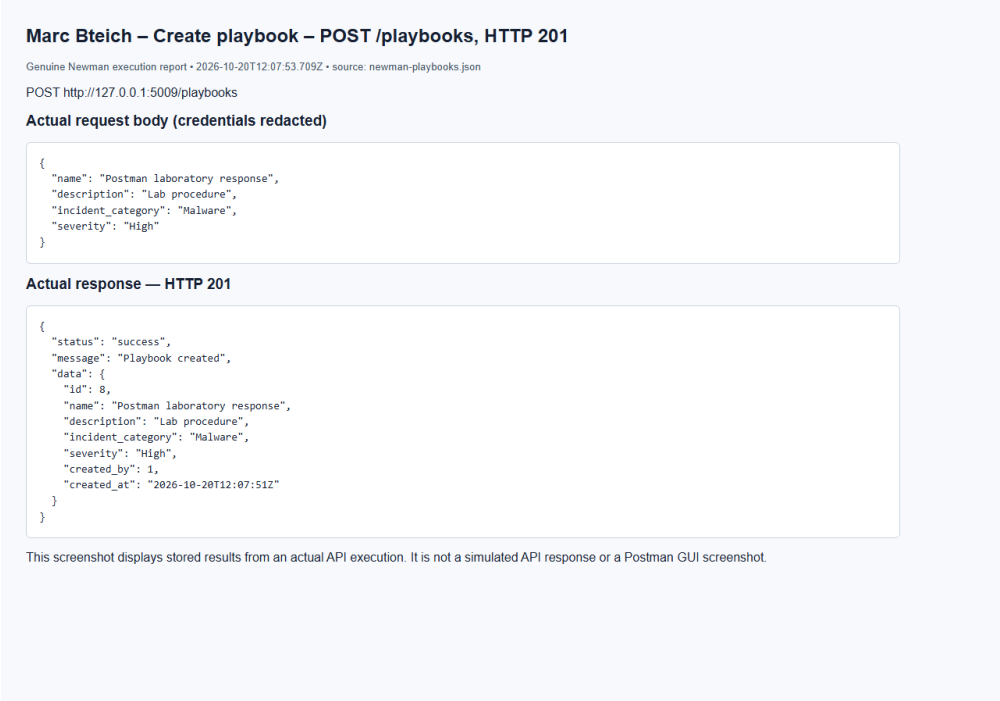
(Output shortened; the full capture is report/screenshots/elie/newman-notifications-31.png in the submission.)

5.9 Security Playbook and Response Automation Service (Marc Bteich)

Playbooks organize response procedures by incident category and severity. Actions have unique positive step numbers within each playbook and are returned in order. Admins manage playbooks; admins/analysts add and progress actions. Pending advances to In Progress and then Completed; execution requires an active assigned user. completion_date is generated automatically. An append-only JSON execution_history field records actor, timestamp, prior/next status, assignment and completion. The JSON list is saved in the same transaction as

the action. Before deletion, history is archived through Logs; a failed archive returns 503 and preserves the playbook. Deletion cascades to actions only after archival.

Marc Bteich – Create playbook – POST /playbooks, HTTP 201



Marc Bteich – Create playbook – POST /playbooks, HTTP 201

Genuine Newman execution report • 2026-10-20T12:07:53.709Z • source: newman-playbooks.json

POST http://127.0.0.1:5009/playbooks

Actual request body (credentials redacted)

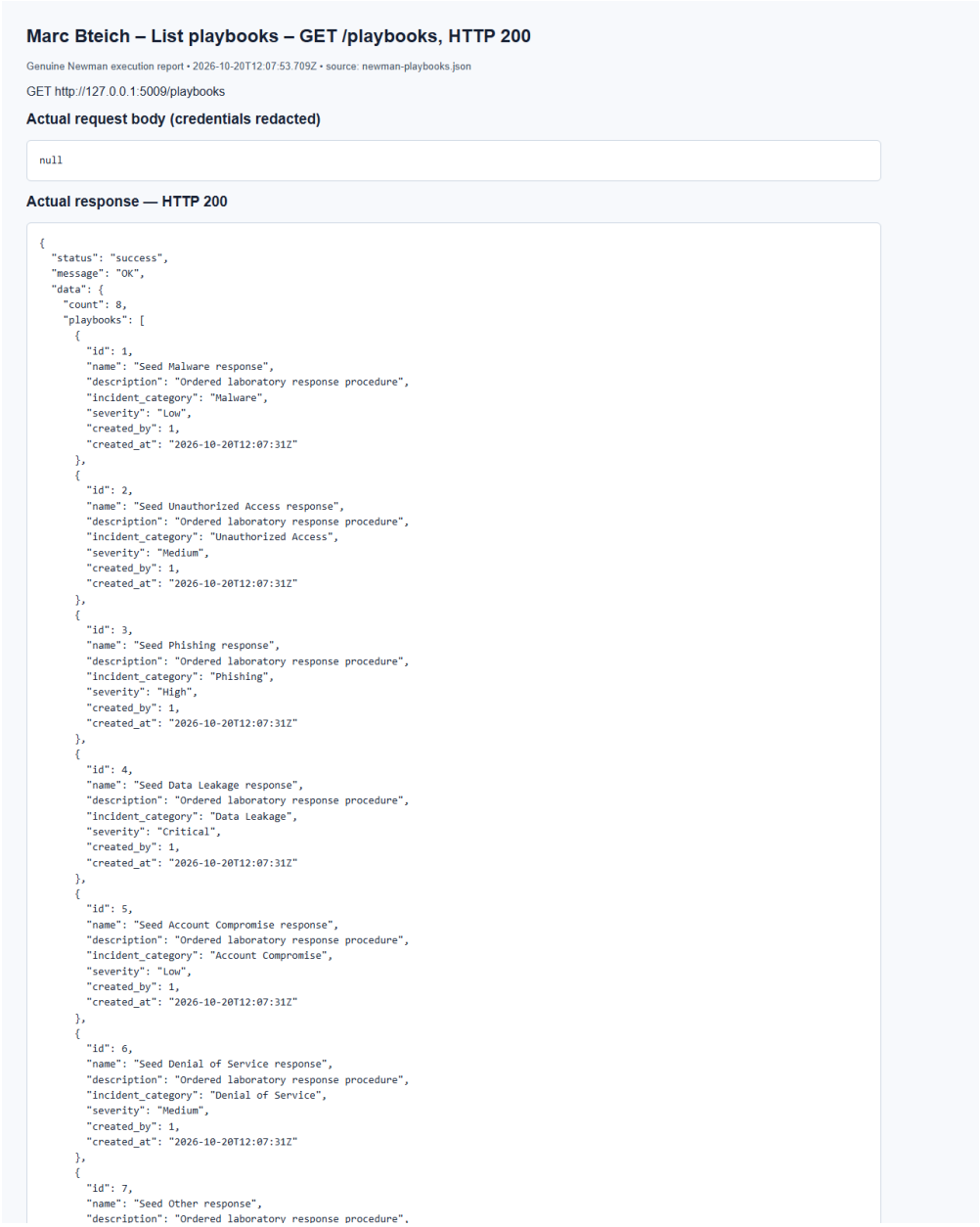

```
{
  "name": "Postman laboratory response",
  "description": "Lab procedure",
  "incident_category": "Malware",
  "severity": "High"
}
```


Actual response — HTTP 201


```
{
  "status": "success",
  "message": "Playbook created",
  "data": {
    "id": 8,
    "name": "Postman laboratory response",
    "description": "Lab procedure",
    "incident_category": "Malware",
    "severity": "High",
    "created_by": 1,
    "created_at": "2026-10-20T12:07:51Z"
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – List playbooks – GET /playbooks, HTTP 200



Marc Bteich – List playbooks – GET /playbooks, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:53.709Z • source: newman-playbooks.json

GET http://127.0.0.1:5009/playbooks

Actual request body (credentials redacted)

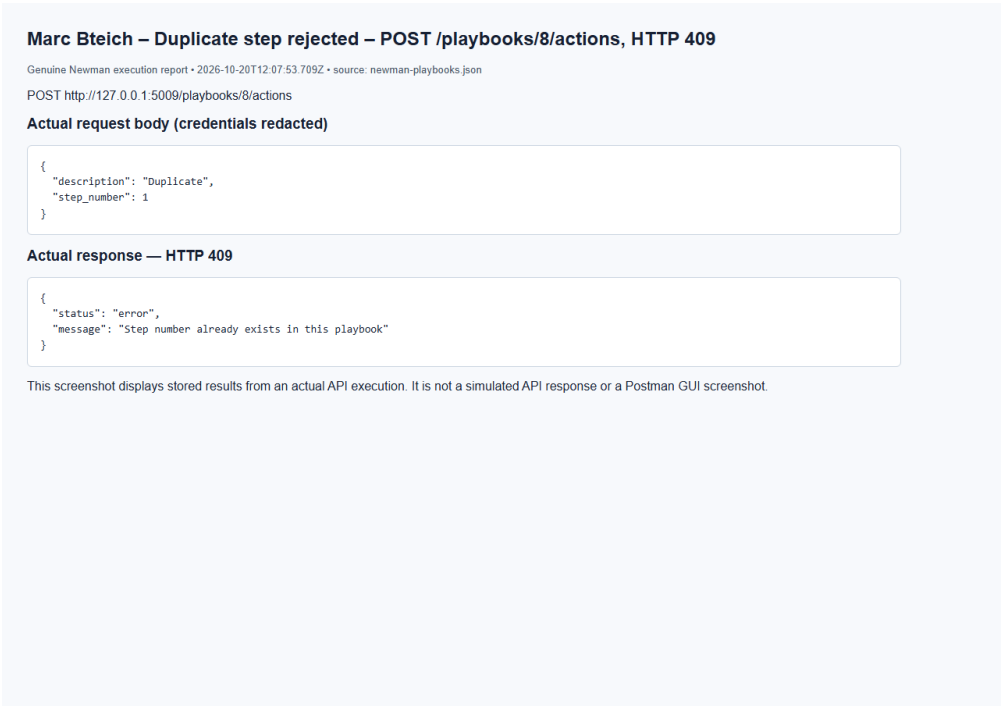
```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 8,
    "playbooks": [
      {
        "id": 1,
        "name": "Seed Malware response",
        "description": "Ordered laboratory response procedure",
        "incident_category": "Malware",
        "severity": "Low",
        "created_by": 1,
        "created_at": "2026-10-20T12:07:31Z"
      },
      {
        "id": 2,
        "name": "Seed Unauthorized Access response",
        "description": "Ordered laboratory response procedure",
        "incident_category": "Unauthorized Access",
        "severity": "Medium",
        "created_by": 1,
        "created_at": "2026-10-20T12:07:31Z"
      },
      {
        "id": 3,
        "name": "Seed Phishing response",
        "description": "Ordered laboratory response procedure",
        "incident_category": "Phishing",
        "severity": "High",
        "created_by": 1,
        "created_at": "2026-10-20T12:07:31Z"
      },
      {
        "id": 4,
        "name": "Seed Data Leakage response",
        "description": "Ordered laboratory response procedure",
        "incident_category": "Data Leakage",
        "severity": "Critical",
        "created_by": 1,
        "created_at": "2026-10-20T12:07:31Z"
      },
      {
        "id": 5,
        "name": "Seed Account Compromise response",
        "description": "Ordered laboratory response procedure",
        "incident_category": "Account Compromise",
        "severity": "Low",
        "created_by": 1,
        "created_at": "2026-10-20T12:07:31Z"
      },
      {
        "id": 6,
        "name": "Seed Denial of Service response",
        "description": "Ordered laboratory response procedure",
        "incident_category": "Denial of Service",
        "severity": "Medium",
        "created_by": 1,
        "created_at": "2026-10-20T12:07:31Z"
      },
      {
        "id": 7,
        "name": "Seed Other response",
        "description": "Ordered laboratory response procedure",
```


(Output shortened; the full capture is report/screenshots/marc/newman-playbooks-06.png in the submission.)

Marc Bteich – Duplicate step rejected – POST /playbooks/8/actions, HTTP 409



Marc Bteich – Duplicate step rejected – POST /playbooks/8/actions, HTTP 409

Genuine Newman execution report • 2026-10-20T12:07:53.709Z • source: newman-playbooks.json

POST http://127.0.0.1:5009/playbooks/8/actions

Actual request body (credentials redacted)

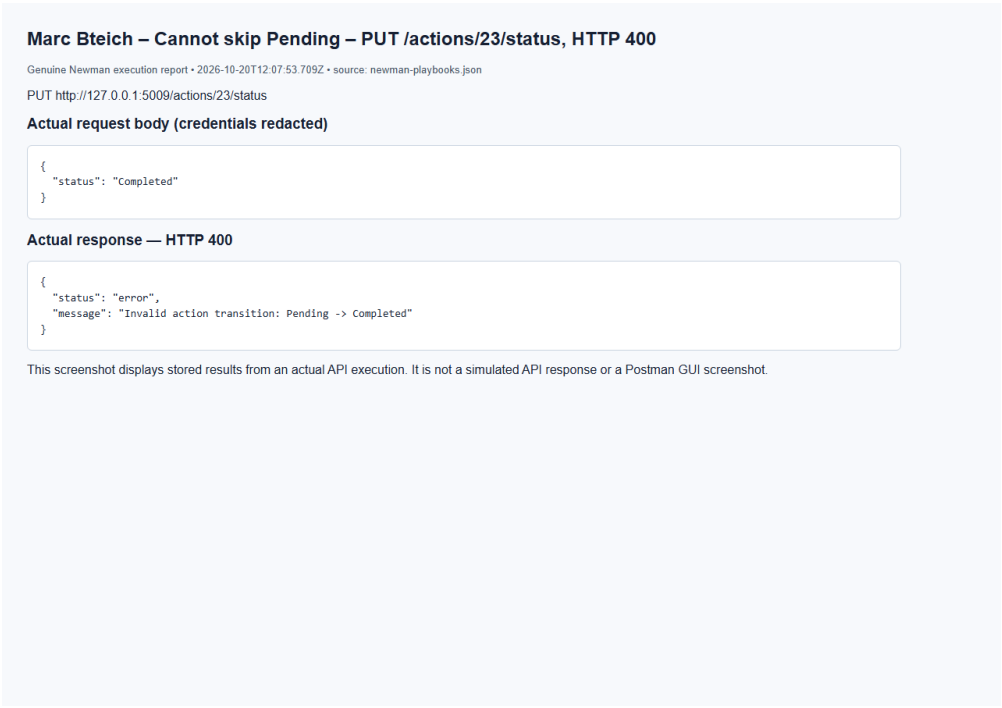
```
{
  "description": "Duplicate",
  "step_number": 1
}
```

Actual response — HTTP 409

```
{
  "status": "error",
  "message": "Step number already exists in this playbook"
}
```

This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Cannot skip Pending – PUT /actions/23/status, HTTP 400



Marc Bteich – Cannot skip Pending – PUT /actions/23/status, HTTP 400

Genuine Newman execution report • 2026-10-20T12:07:53.709Z • source: newman-playbooks.json

PUT http://127.0.0.1:5009/actions/23/status

Actual request body (credentials redacted)

```
{
  "status": "Completed"
}
```

Actual response — HTTP 400

```
{
  "status": "error",
  "message": "Invalid action transition: Pending -> Completed"
}
```

This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – Start assigned action – PUT /actions/23/status, HTTP 200

Marc Bteich – Start assigned action – PUT /actions/23/status, HTTP 200

Genuine Newman execution report • 2026-10-20T12:07:53.709Z • source: newman-playbooks.json

PUT http://127.0.0.1:5009/actions/23/status

Actual request body (credentials redacted)


```
{
  "status": "In Progress",
  "assigned_user_id": 3
}
```


Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Action updated",
  "data": {
    "id": 23,
    "playbook_id": 8,
    "description": "Collect evidence",
    "step_number": 1,
    "status": "In Progress",
    "assigned_user_id": 3,
    "completion_date": null,
    "execution_history": [
      {
        "actor_id": 3,
        "at": "2026-10-20T12:07:52Z",
        "from_status": null,
        "to_status": "Pending",
        "assigned_user_id": null,
        "completion_date": null
      },
      {
        "actor_id": 3,
        "at": "2026-10-20T12:07:52Z",
        "from_status": "Pending",
        "to_status": "In Progress",
        "assigned_user_id": 3,
        "completion_date": null
      }
    ]
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

Marc Bteich – No token – GET /playbooks, HTTP 401

Marc Bteich – No token – GET /playbooks, HTTP 401

Genuine Newman execution report • 2026-10-20T12:07:53.709Z • source: newman-playbooks.json

GET http://127.0.0.1:5009/playbooks

Actual request body (credentials redacted)


```
null
```


Actual response — HTTP 401

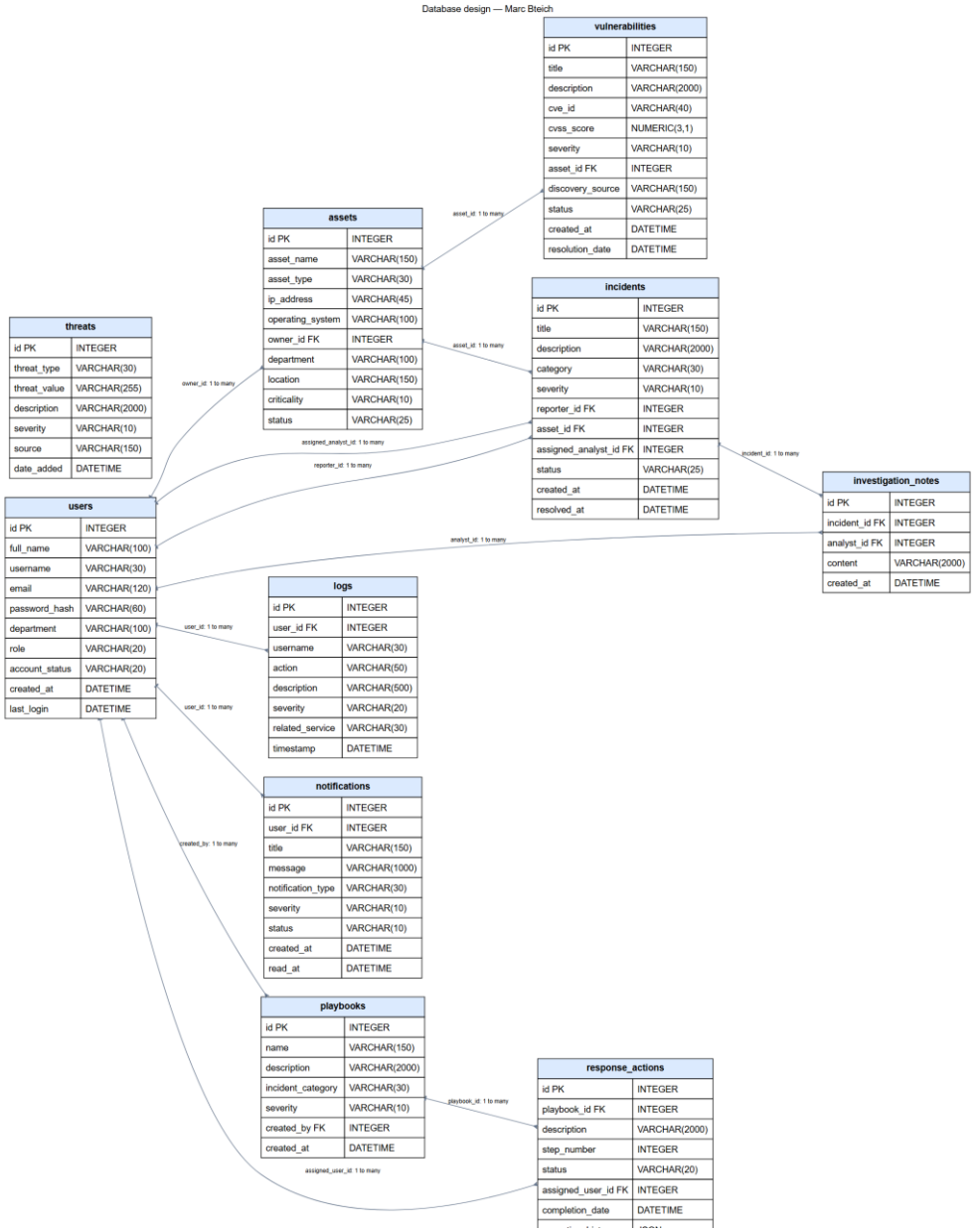

```
{
  "status": "error",
  "message": "Missing authentication token (use 'Authorization: Bearer <token>')"
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

6. Database Design (Marc Bteich)

The schema contains exactly ten tables with integer primary keys. Foreign keys enforce ownership, reporting/assignment, affected assets, notes, notifications and response actions. Enum CHECK constraints restrict known roles, statuses, categories and severities; UNIQUE constraints protect usernames/emails, threat indicator pairs and playbook step numbers. CVSS and action-step bounds are checked by the database. IP/CVE formats and status transitions are validated by services because a membership CHECK cannot express an ordered lifecycle.

Entity relationships



(Output shortened; the full capture is report/diagrams/database.png in the submission.)

Users→Assets is one-to-many through owner_id; Users→Incidents uses reporter_id and optional assigned_analyst_id; Assets→Incidents and Assets→Vulnerabilities identify affected resources; Incidents→Investigation Notes is one-to-many; Users→Logs/Notifications identifies actors and recipients; Users→Playbooks identifies creators; Playbooks→Response Actions is ordered one-to-many; Users→Response Actions identifies assignees. Departments are fields rather than an extra table, preserving the agreed ten-table design.

Deletion choices preserve useful records: incident asset/analyst links and log user links become null; log usernames remain as snapshots; incident notes and user notifications cascade with their parents; an asset owner and a vulnerable asset cannot be deleted while required references exist. Playbook actions cascade after their execution history is archived. Action-history schema migration adds one column to an existing table without dropping data. All ten ORM column definitions were compared with schema.sql.

Tables and field definitions (Marc Bteich)

assets (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
asset_name	VARCHAR(150)	NOT NULL
asset_type	VARCHAR(30)	NOT NULL
ip_address	VARCHAR(45)	NOT NULL
operating_system	VARCHAR(100)	nullable
owner_id	INTEGER	NOT NULL; FK → users.id; delete NO ACTION
department	VARCHAR(100)	NOT NULL
location	VARCHAR(150)	nullable
criticality	VARCHAR(10)	NOT NULL
status	VARCHAR(25)	NOT NULL

incidents (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
title	VARCHAR(150)	NOT NULL
description	VARCHAR(2000)	NOT NULL
category	VARCHAR(30)	NOT NULL
severity	VARCHAR(10)	NOT NULL
reporter_id	INTEGER	NOT NULL; FK → users.id; delete NO ACTION
asset_id	INTEGER	nullable; FK → assets.id; delete SET NULL
assigned_analyst_id	INTEGER	nullable; FK → users.id; delete SET NULL
status	VARCHAR(25)	NOT NULL
created_at	DATETIME	NOT NULL
resolved_at	DATETIME	nullable

investigation_notes (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
incident_id	INTEGER	NOT NULL; FK → incidents.id; delete CASCADE

analyst_id	INTEGER	nullable; FK → users.id; delete SET NULL
content	VARCHAR(2000)	NOT NULL
created_at	DATETIME	NOT NULL

logs (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
user_id	INTEGER	nullable; FK → users.id; delete SET NULL
username	VARCHAR(30)	nullable
action	VARCHAR(50)	NOT NULL
description	VARCHAR(500)	NOT NULL
severity	VARCHAR(20)	NOT NULL
related_service	VARCHAR(30)	NOT NULL
timestamp	DATETIME	NOT NULL

notifications (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
user_id	INTEGER	NOT NULL; FK → users.id; delete CASCADE

title	VARCHAR(150)	NOT NULL
message	VARCHAR(1000)	NOT NULL
notification_type	VARCHAR(30)	NOT NULL
severity	VARCHAR(10)	NOT NULL
status	VARCHAR(10)	NOT NULL
created_at	DATETIME	NOT NULL
read_at	DATETIME	nullable

playbooks (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
name	VARCHAR(150)	NOT NULL
description	VARCHAR(2000)	NOT NULL
incident_category	VARCHAR(30)	NOT NULL
severity	VARCHAR(10)	NOT NULL
created_by	INTEGER	NOT NULL; FK → users.id; delete NO ACTION
created_at	DATETIME	NOT NULL

response_actions (Marc Bteich)

Field	Type	Constraint / relationship

id	INTEGER	PK
playbook_id	INTEGER	NOT NULL; FK → playbooks.id; delete CASCADE
description	VARCHAR(2000)	NOT NULL
step_number	INTEGER	NOT NULL
status	VARCHAR(20)	NOT NULL
assigned_user_id	INTEGER	nullable; FK → users.id; delete SET NULL
completion_date	DATETIME	nullable
execution_history	JSON	NOT NULL; SQL default '[]'

threats (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
threat_type	VARCHAR(30)	NOT NULL
threat_value	VARCHAR(255)	NOT NULL
description	VARCHAR(2000)	NOT NULL
severity	VARCHAR(10)	NOT NULL
source	VARCHAR(150)	NOT NULL
date_added	DATETIME	NOT NULL

users (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
full_name	VARCHAR(100)	NOT NULL
username	VARCHAR(30)	NOT NULL
email	VARCHAR(120)	NOT NULL
password_hash	VARCHAR(60)	NOT NULL
department	VARCHAR(100)	NOT NULL
role	VARCHAR(20)	NOT NULL
account_status	VARCHAR(20)	NOT NULL
created_at	DATETIME	NOT NULL
last_login	DATETIME	nullable

vulnerabilities (Marc Bteich)

Field	Type	Constraint / relationship
id	INTEGER	PK
title	VARCHAR(150)	NOT NULL
description	VARCHAR(2000)	NOT NULL
cve_id	VARCHAR(40)	NOT NULL
cvss_score	NUMERIC(3,1)	NOT NULL

severity	**VARCHAR(10)**	**NOT NULL**
asset_id	**INTEGER**	**NOT NULL; FK → assets.id; delete NO ACTION**
discovery_source	**VARCHAR(150)**	**NOT NULL**
status	**VARCHAR(25)**	**NOT NULL**
created_at	**DATETIME**	**NOT NULL**
resolution_date	**DATETIME**	**nullable**

Sample data and design decisions (Marc Bteich)

database.seed is additive and idempotent. It creates synthetic lab scenarios spanning all roles, account statuses, asset types/statuses, threat types/severities, incident/vulnerability lifecycles, notification types/read states and action statuses. Passwords are generated randomly, hashed with the existing User method and stored privately in .local/seed-credentials.json for local test access. No existing password is reset. Seeded lifecycle states/history are explicitly labelled scenario snapshots; HTTP tests separately demonstrate real transitions.

Marc Bteich – Database structure and stored records – actual local SQLite inspection



Marc Bteich – Database structure and stored records – actual local SQLite inspection

assets


```
[
  [
    0,
    "id",
    "INTEGER",
    0,
    null,
    1
  ],
  [
    1,
    "asset_name",
    "VARCHAR(150)",
    1,
    null,
    0
  ],
  [
    2,
    "asset_type",
    "VARCHAR(30)",
    1,
    null,
    0
  ],
  [
    3,
    "ip_address",
    "VARCHAR(45)",
    1,
    null,
    0
  ],
  [
    4,
    "operating_system",
    "VARCHAR(100)",
    0,
    null,
    0
  ],
  [
    5,
    "owner_id",
    "INTEGER",
    1,
    null,
    0
  ],
  [
    6,
    "department",
    "VARCHAR(100)",
    1,
    null,
    0
  ],
  [
    7,
    "location",
    "VARCHAR(150)",
    0,
    null,
    0
  ],
  [
    8,
    "criticality",
    "VARCHAR(10)",
    1,
    null,
    0
```


(Output shortened; the full capture is report/screenshots/marc/database-inspection.png in the submission.)

Elie Medawar – Database records after the Newman run – users, incidents, investigation_notes, notifications, logs

Elie Medawar – Database records – users, incidents, investigation_notes, logs, notifications (actual SQLite after the Newman run)

users (4 rows, first 4 shown)


```
[
  {
    "id": 1,
    "full_name": "Elie Medawar",
    "username": "admin",
    "email": "admin@corp.com",
    "department": "Security",
    "role": "admin",
    "account_status": "Active",
    "created_at": "2026-10-20 20:00:51.395457",
    "last_login": "2026-10-20 20:01:17.245381"
  },
  {
    "id": 2,
    "full_name": "Rami Haddad",
    "username": "rami",
    "email": "rami@corp.com",
    "department": "Accounting",
    "role": "user",
    "account_status": "Active",
    "created_at": "2026-10-20 20:00:55.552172",
    "last_login": "2026-10-20 20:01:17.543789"
  },
  {
    "id": 3,
    "full_name": "Lara Khoury",
    "username": "lara",
    "email": "lara@corp.com",
    "department": "Security Operations Center",
    "role": "analyst",
    "account_status": "Active",
    "created_at": "2026-10-20 20:00:55.838769",
    "last_login": "2026-10-20 20:01:17.820747"
  },
  {
    "id": 4,
    "full_name": "Nadia Saab",
    "username": "nadia",
    "email": "nadia@corp.com",
    "department": "Compliance",
    "role": "auditor",
    "account_status": "Active",
    "created_at": "2026-10-20 20:00:56.104349",
    "last_login": "2026-10-20 20:01:18.108727"
  }
]
```


incidents (4 rows, first 4 shown)


```
[
  {
    "id": 1,
    "title": "Phishing email received",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 1,
    "assigned_analyst_id": 3,
    "status": "Closed",
    "created_at": "2026-10-20 20:01:07.522252",
    "resolved_at": "2026-10-20 20:01:10.281049"
  },
  {
    "id": 2,
    "title": "Ransomware on web server",
```


(Output shortened; the full capture is report/screenshots/elie/database-inspection-elie.png in the submission.)

7. Error Handling and Validation (Elie Medawar)

All nine services return errors in one format, produced by the shared common/errors.py, so a client handles every error the same way: {"status": "error", "message": "User does not have permission to perform this operation"}.

Error handling approach. Service code raises a typed exception, and a handler registered on every app by create_service_app() turns it into the JSON above with the matching HTTP code. Flask's own errors (unknown URL, wrong method) and database constraint violations go through the same handlers. Any unexpected exception is rolled back, logged on the server and returned as a generic 500 that never leaks internal details.

Exception / cause	HTTP code	Example message
ValidationError	400	Invalid severity 'Extreme'. Allowed values: Low, Medium, High, Critical
AuthError	401	Missing authentication token (use 'Authorization: Bearer <token>')
ForbiddenError	403	User does not have permission to perform this operation
NotFoundError	404	Incident with id 999 not found
ConflictError, database IntegrityError	409	Username 'RAMI' already exists
Wrong HTTP method	405	HTTP method not allowed on this endpoint
Required service unreachable	503	Asset Management Service is unavailable; cannot verify the affected asset
Any other exception	500	Internal server error

Validation rules. The shared helpers in common/validators.py are used by every service:

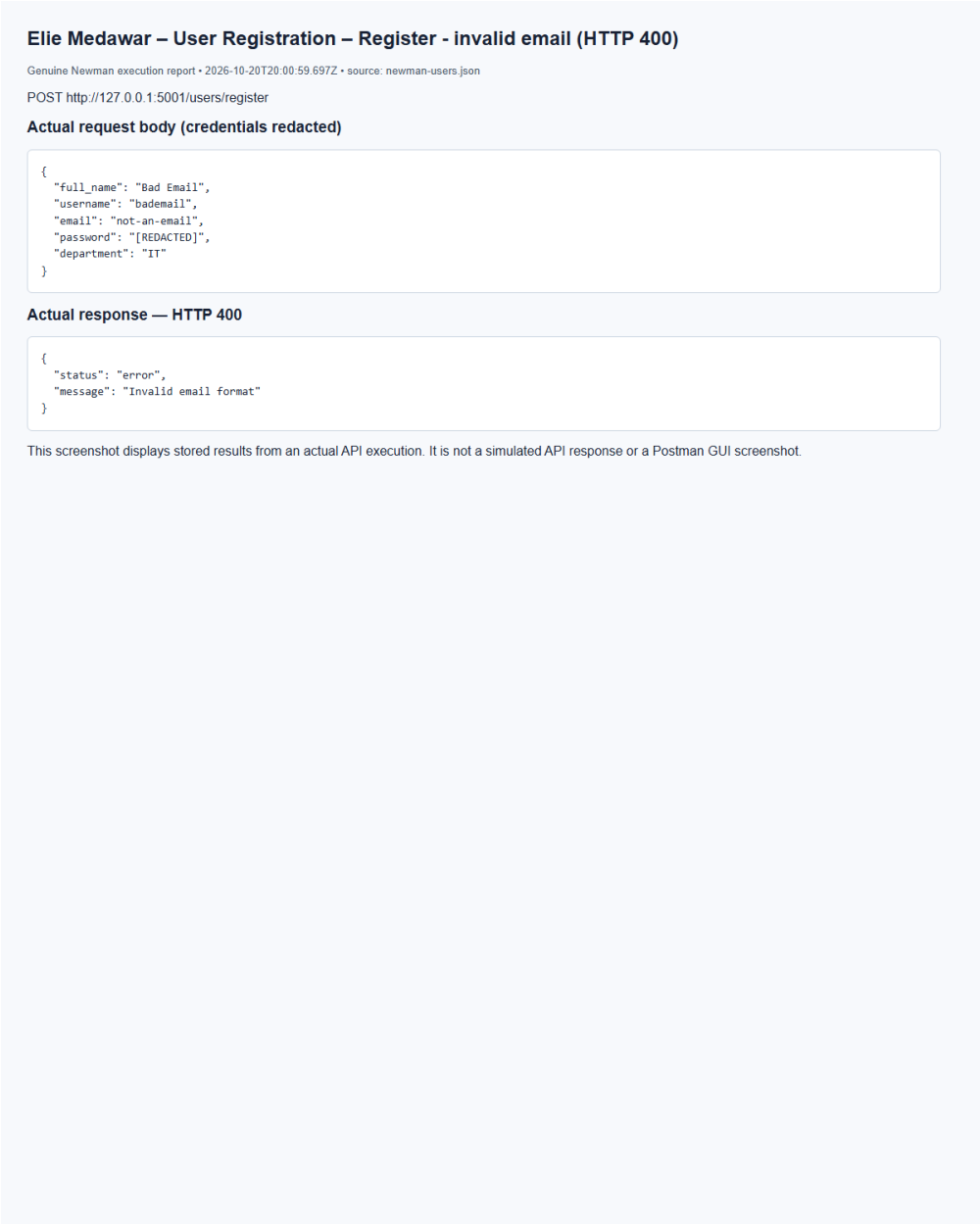
- get_json_body() rejects a missing, non-JSON or empty body.
- require_fields() lists every missing or blank required field in one message.
- reject_unknown_fields() rejects unexpected fields, for example is_admin or reporter_id.
- clean_string() checks the type, trims the value and enforces a maximum length.
- validate_enum() accepts only the allowed values (severities, statuses, roles, categories, types).
- validate_email() checks the email format.

Service-specific rules: password strength and username format (Users); integer ids and existing assets and analysts (Incidents); the status lifecycle (Incidents); action format, existing user and date format (Logs); notification type and existing user (Notifications).

Security checks against malicious input.

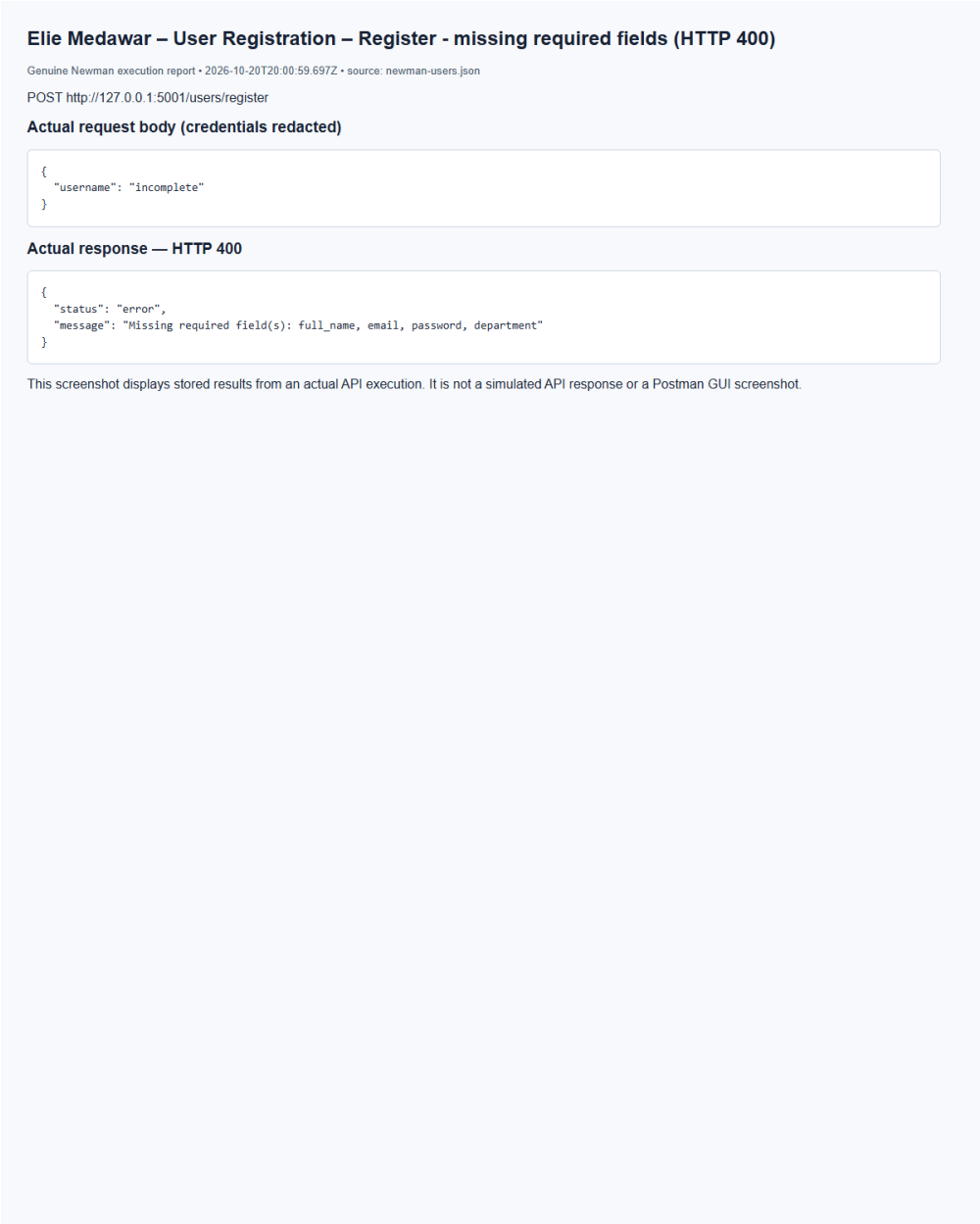
- SQL injection: all database access goes through the SQLAlchemy ORM with bound parameters; no SQL is built from strings.
- Unexpected values: enum fields are checked against fixed lists in the code and again by CHECK constraints in the database.
- Mass assignment: unknown fields are rejected, so a client cannot set role, reporter_id or account_status where it is not allowed.
- Data protection: passwords are stored only as bcrypt hashes and never returned.

Elie Medawar – User Registration – Register - invalid email (HTTP 400)



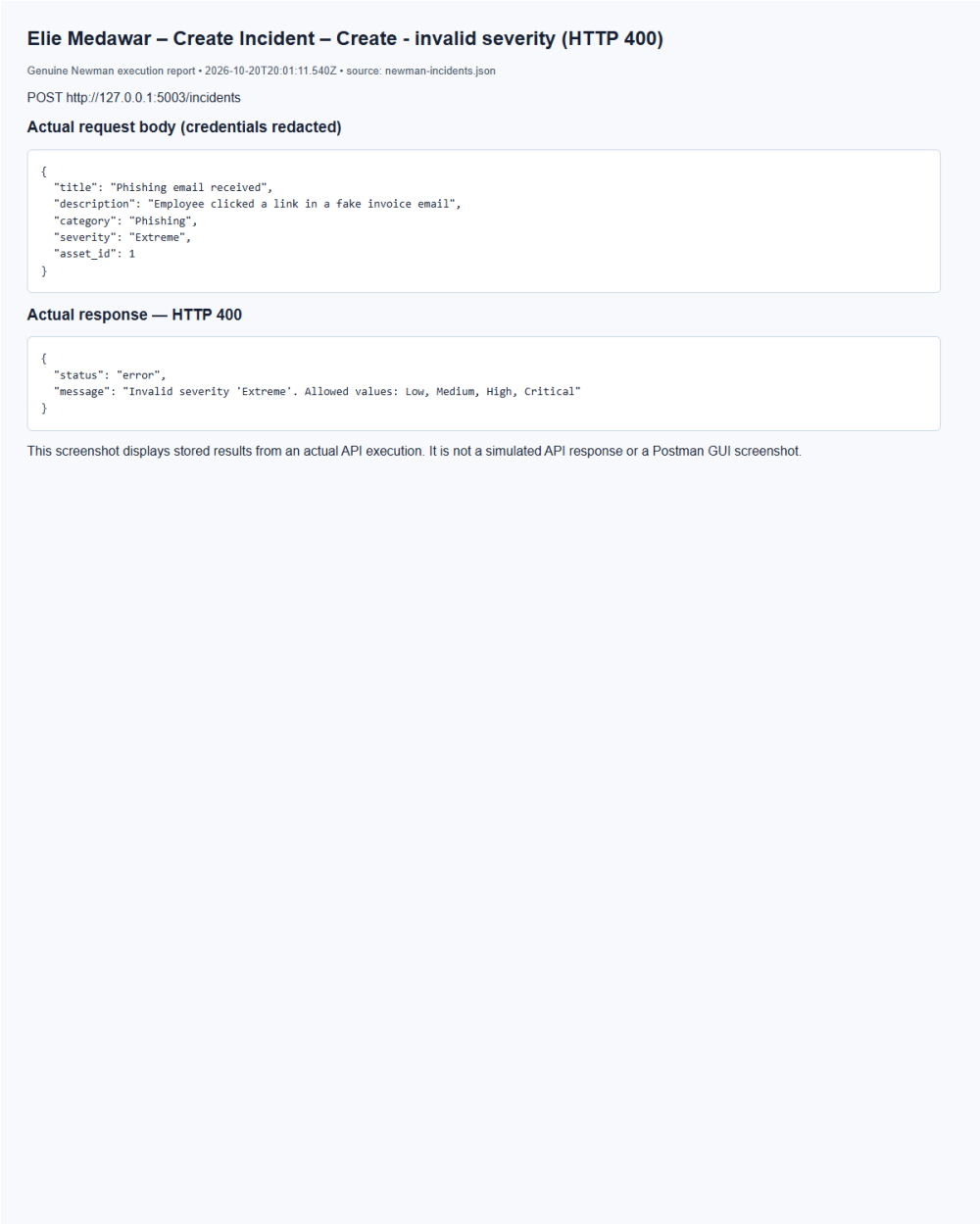
(Output shortened; the full capture is report/screenshots/elie/newman-users-09.png in the submission.)

Elie Medawar – User Registration – Register - missing required fields (HTTP 400)



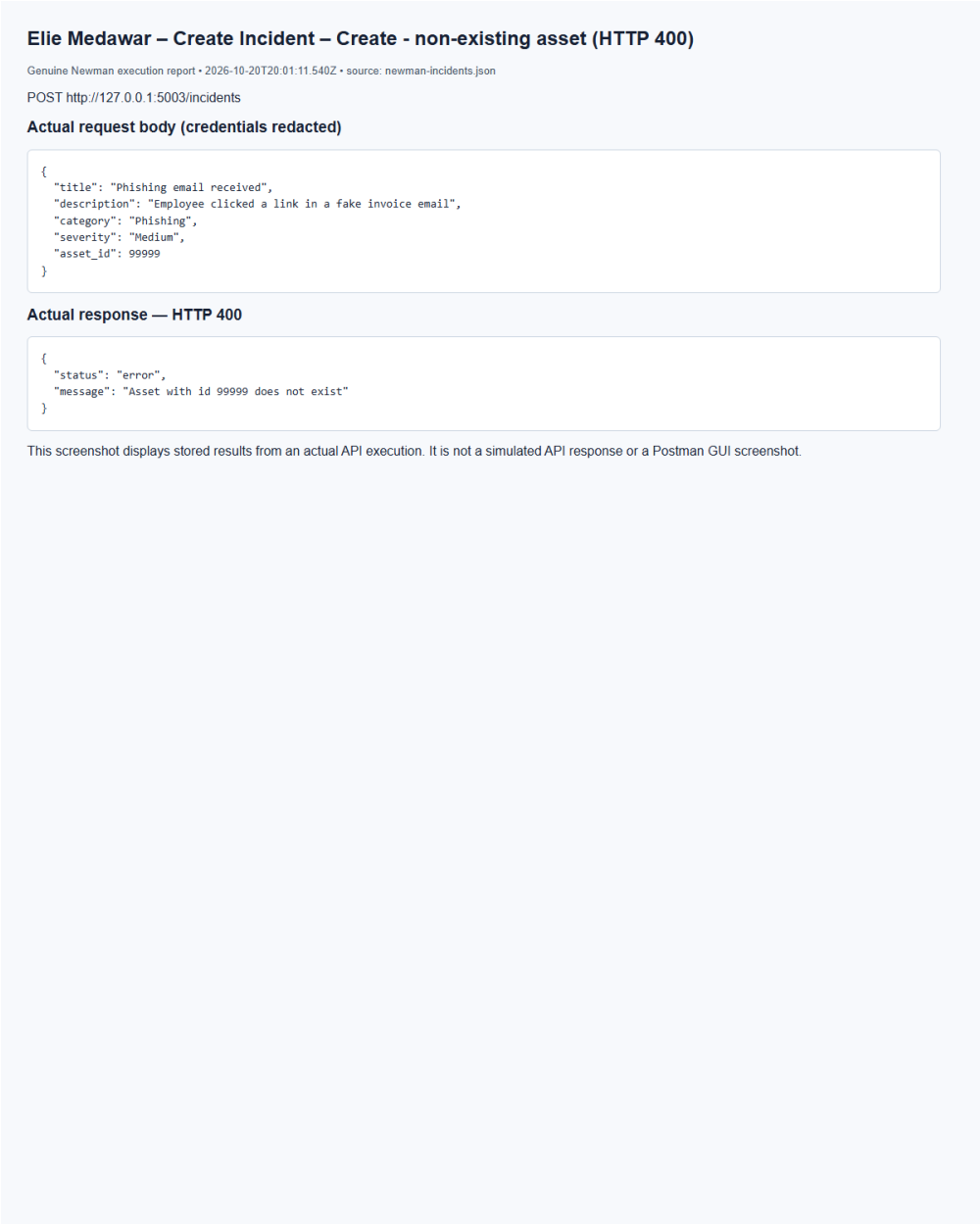
(Output shortened; the full capture is report/screenshots/elie/newman-users-11.png in the submission.)

Elie Medawar – Create Incident – Create - invalid severity (HTTP 400)



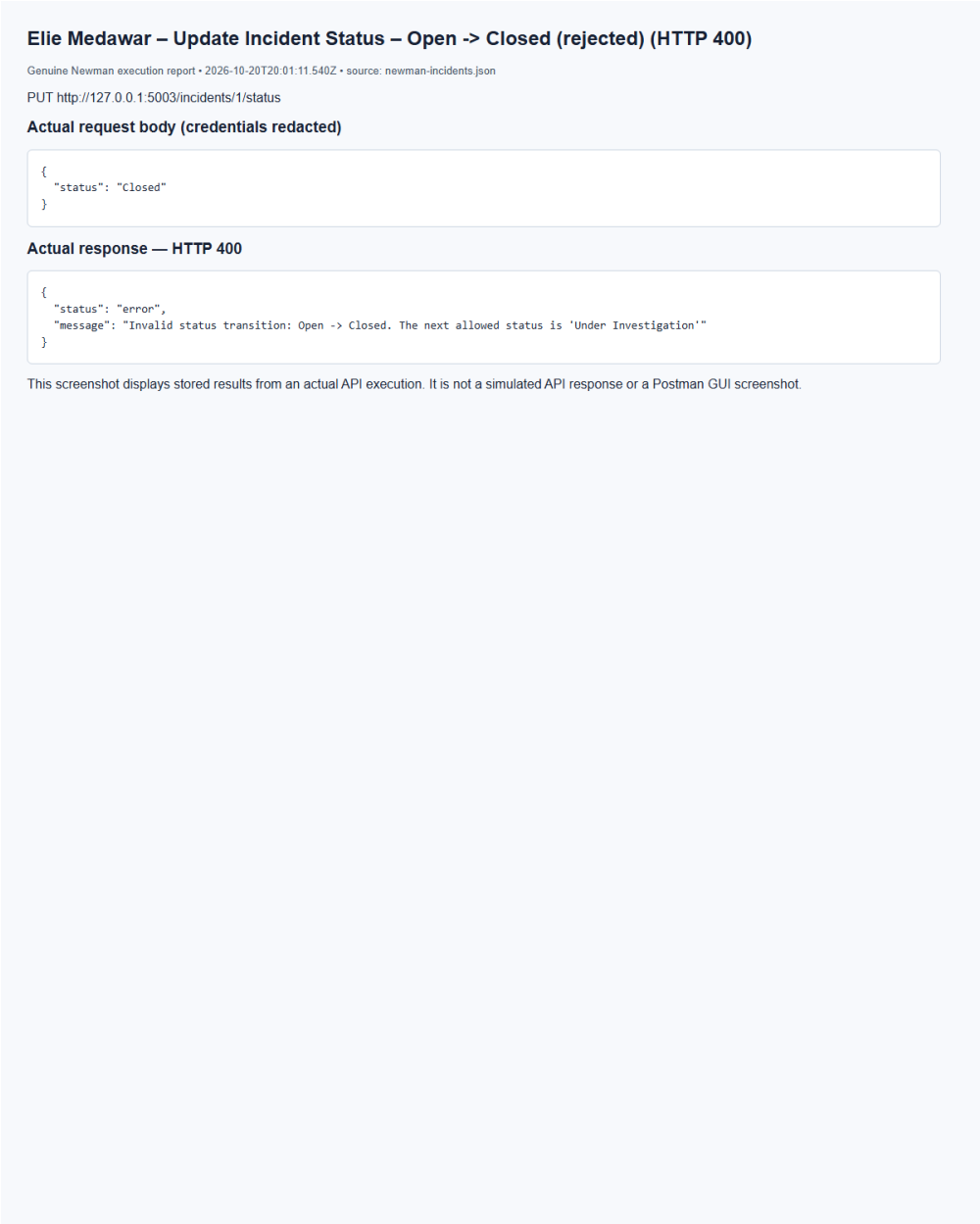
(Output shortened; the full capture is report/screenshots/elie/newman-incidents-09.png in the submission.)

Elie Medawar – Create Incident – Create - non-existing asset (HTTP 400)



(Output shortened; the full capture is report/screenshots/elie/newman-incidents-12.png in the submission.)

Elie Medawar – Update Incident Status – Open -> Closed (rejected) (HTTP 400)



(Output shortened; the full capture is report/screenshots/elie/newman-incidents-29.png in the submission.)

8. API Documentation (Marc Bteich)

The merged endpoint reference is API_DOCUMENTATION.md. SIDRMS.postman_collection.json combines both students' requests without altering Elie's source collections or scripts; collection-level hooks are preserved as folder hooks. Marc's individual collections exercise positive and negative cases and automatically create/clean relationship fixtures. Actual Newman results and redacted request/response records are in evidence/.

The following endpoint inventory is extracted from the actual route source. Request examples and negative tests are in the merged Postman collection. Roles shown are the route-level checks; object-level checks in Elie's services still apply.

Public authentication uses Authorization: Bearer <JWT>. Internal endpoints require X-Service-Key. JSON writes use Content-Type: application/json. Success: {status: success, message, data}; errors: {status: error, message}. Typical errors: 400 invalid input, 401 absent/invalid credentials, 403 insufficient permission, 404 missing record, 409 database conflict, 503 unavailable dependency. A 409 is only expected when an applicable constraint exists.

Users (Marc Bteich)

POST /users/register (Marc Bteich)

Purpose: register user. Access: public. Authentication: Public.

Postman example: `POST {{base_users}}/users/register`.

Request body example (template; credentials remain private):

```
{
  "full_name": "Rami Haddad",
  "username": "rami",
  "email": "rami@corp.com",
  "password": "{{user_password}}",
  "department": "Finance",
  "role": "user"
}
```

Executed response example: HTTP 201; Newman request "Register - regular user (self-registration)". Credentials are redacted.

```
{
  "status": "success",
  "message": "User registered successfully",
  "data": {
    "id": 2,
    "full_name": "Rami Haddad",
    "username": "rami",
    "email": "rami@corp.com",
    "department": "Finance",
    "role": "user",
    "account_status": "Active",
    "created_at": "2026-10-20T12:08:48Z",
    "last_login": null
  }
}
```

POST /users/login (Marc Bteich)

Purpose: login. Access: public. Authentication: Public.

Postman example: `POST {{base_users}}/users/login`.

Request body example (template; credentials remain private):

```
{
  "username": "{{admin_username}}",
  "password": "{{admin_password}}"
}
```

Executed response example: HTTP 200; Newman request "Login - admin (success)". Credentials are redacted.

```
{
  "status": "success",
  "message": "Login successful",
  "data": {
    "token": "[REDACTED]",
    "token_type": "[REDACTED]",
    "user": {
      "id": 1,
      "full_name": "Local test admin",
      "username": "admin",
      "email": "admin@example.test",
      "department": "Lab",
      "role": "admin",
      "account_status": "Active",
      "created_at": "2026-10-20T12:08:43Z",
      "last_login": "2026-10-20T12:08:48Z"
    }
  }
}
```

PUT /users/{user_id} (Marc Bteich)

Purpose: update user. Access: admin,user. Authentication: Bearer JWT.

Path parameters: user_id. IDs must identify existing records.

Postman example: `PUT {{base_users}}/users/{{user_id}}`.

Request body example (template; credentials remain private):

```
{
  "department": "Accounting",
  "full_name": "Rami Haddad"
```

```
}
```


Executed response example: HTTP 200; Newman request "Update own info - regular user". Credentials are redacted.


```
{
  "status": "success",
  "message": "User updated successfully",
  "data": {
    "id": 2,
    "full_name": "Rami Haddad",
    "username": "rami",
    "email": "rami@corp.com",
    "department": "Accounting",
    "role": "user",
    "account_status": "Active",
    "created_at": "2026-10-20T12:08:48Z",
    "last_login": "2026-10-20T12:08:50Z"
  }
}
```


DELETE /users/{user_id} (Marc Bteich)

Purpose: delete user. Access: admin. Authentication: Bearer JWT.

Path parameters: user_id. IDs must identify existing records.

Postman example: `DELETE {{base_users}}/users/{{temp_user_id}}`.

Executed response example: HTTP 200; Newman request "Delete user - admin (success)". Credentials are redacted.


```
{
  "status": "success",
  "message": "User 5 deleted successfully",
  "data": null
}
```


GET /users (Marc Bteich)

Purpose: get all users. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_users}}/users`.

Executed response example: HTTP 200; Newman request "Get all users - admin". Credentials are redacted.


```
{
```

```
"status": "success",
"message": "Users retrieved successfully",
"data": {
  "count": 5,
  "users": [
    {
      "id": 1,
      "full_name": "Local test admin",
      "username": "admin",
      "email": "admin@example.test",
      "department": "Lab",
      "role": "admin",
      "account_status": "Active",
      "created_at": "2026-10-20T12:08:43Z",
      "last_login": "2026-10-20T12:08:48Z"
    },
    {
      "id": 2,
      "full_name": "Rami Haddad",
      "username": "rami",
      "email": "rami@corp.com",
      "department": "Finance",
      "role": "user",
      "account_status": "Active",
      "created_at": "2026-10-20T12:08:48Z",
      "last_login": "2026-10-20T12:08:50Z"
    },
    {
      "id": 3,
      "full_name": "Lara Khoury",
      "username": "lara",
      "email": "lara@corp.com",
      "department": "SOC",
      "role": "analyst",
      "account_status": "Active",
      "created_at": "2026-10-20T12:08:49Z",
      "last_login": "2026-10-20T12:08:50Z"
    },
    {
      "id": 4,
      "full_name": "Nadia Saab",
      "username": "nadia",
      "email": "nadia@corp.com",
      "department": "Compliance",
      "role": "auditor",
      "account_status": "Active",
      "created_at": "2026-10-20T12:08:49Z",
      "last_login": "2026-10-20T12:08:51Z"
    },
    {
      "id": 5,
      "full_name": "Temp Employee",
      "username": "temp.employee",
      "email": "temp@corp.com",
      "department": "HR",
      "role": "user",
```

```
        "account_status": "Active",
        "created_at": "2026-10-20T12:08:49Z",
        "last_login": null
      }
    ]
  }
}
```


GET /users/{username} (Marc Bteich)

Purpose: get user by username. Access: admin,auditor,user. Authentication: Bearer JWT.

Path parameters: username. IDs must identify existing records.

Postman example: `GET {{base_users}}/users/lara`.

Executed response example: HTTP 200; Newman request "Get user by username - existing". Credentials are redacted.


```
{
  "status": "success",
  "message": "User retrieved successfully",
  "data": {
    "id": 3,
    "full_name": "Lara Khoury",
    "username": "lara",
    "email": "lara@corp.com",
    "department": "SOC",
    "role": "analyst",
    "account_status": "Active",
    "created_at": "2026-10-20T12:08:49Z",
    "last_login": "2026-10-20T12:08:50Z"
  }
}
```


GET /internal/users/{user_id} (Marc Bteich)

Purpose: internal get user. Access: internal service only. Authentication: Service key.

Path parameters: user_id. IDs must identify existing records.

GET /internal/users (Marc Bteich)

Purpose: internal list users. Access: internal service only. Authentication: Service key.

GET /internal/stats (Marc Bteich)

Purpose: internal stats. Access: internal service only. Authentication: Service key.

Postman example: `GET {{base_assets}}/internal/stats`.

Executed response example: HTTP 200; Newman request "Internal statistics". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "total_assets": 6,
    "critical_assets": 1,
    "by_type": {
      "Server": 1,
      "Workstation": 1,
      "Database": 1,
      "Application": 1,
      "Network Device": 1,
      "Cloud Resource": 1
    }
  }
}
```


Assets (Marc Bteich)

POST /assets (Marc Bteich)

Purpose: create asset. Access: admin. Authentication: Bearer JWT.

Postman example: `POST {{base_assets}}/assets`.

Request body example (template; credentials remain private):


```
{
  "asset_name": "Postman Lab Server",
  "asset_type": "Server",
  "ip_address": "192.0.2.10",
  "operating_system": "Linux",
  "department": "Security",
  "owner_id": {{owner_id}},
  "location": "Lab",
  "criticality": "Critical",
  "status": "Active"
}
```


Executed response example: HTTP 201; Newman request "Create asset". Credentials are redacted.


```
{
  "status": "success",
  "message": "Asset created successfully",
  "data": {
```

```
    "id": 1,
    "asset_name": "Postman Lab Server",
    "asset_type": "Server",
    "ip_address": "192.0.2.10",
    "operating_system": "Linux",
    "owner_id": 1,
    "department": "Security",
    "location": "Lab",
    "criticality": "Critical",
    "status": "Active"
  }
}
```


PUT /assets/{asset_id} (Marc Bteich)

Purpose: update asset. Access: admin. Authentication: Bearer JWT.

Path parameters: asset_id. IDs must identify existing records.

Postman example: `PUT {{base_assets}}/assets/{{asset_id}}`.

Request body example (template; credentials remain private):


```
{
  "asset_name": "Updated Postman Server",
  "department": "IT",
  "criticality": "High",
  "status": "Under Maintenance"
}
```


Executed response example: HTTP 200; Newman request "Update asset (admin)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Asset updated successfully",
  "data": {
    "id": 1,
    "asset_name": "Updated Postman Server",
    "asset_type": "Server",
    "ip_address": "192.0.2.10",
    "operating_system": "Linux",
    "owner_id": 1,
    "department": "IT",
    "location": "Lab",
    "criticality": "High",
    "status": "Under Maintenance"
  }
}
```

DELETE /assets/{asset_id} (Marc Bteich)

Purpose: delete asset. Access: admin. Authentication: Bearer JWT.

Path parameters: asset_id. IDs must identify existing records.

Postman example: `DELETE {{base_assets}}/assets/{{asset_id}}`.

Executed response example: HTTP 200; Newman request "Delete asset (admin)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Asset 1 deleted successfully",
  "data": null
}
```


GET /assets (Marc Bteich)

Purpose: list assets. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_assets}}/assets`.

Executed response example: HTTP 200; Newman request "List assets (auditor)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Assets retrieved successfully",
  "data": {
    "count": 6,
    "assets": [
      {
        "id": 1,
        "asset_name": "Postman Lab Server",
        "asset_type": "Server",
        "ip_address": "192.0.2.10",
        "operating_system": "Linux",
        "owner_id": 1,
        "department": "Security",
        "location": "Lab",
        "criticality": "Critical",
        "status": "Active"
      },
      {
        "id": 2,
        "asset_name": "Postman Workstation",
        "asset_type": "Workstation",
        "ip_address": "192.0.2.10",
        "operating_system": "Linux",
        "owner_id": 1,
        "department": "Security",
```

```
      "location": "Lab",
      "criticality": "Low",
      "status": "Active"
    },
    {
      "id": 3,
      "asset_name": "Postman Database",
      "asset_type": "Database",
      "ip_address": "192.0.2.10",
      "operating_system": "Linux",
      "owner_id": 1,
      "department": "Security",
      "location": "Lab",
      "criticality": "Medium",
      "status": "Inactive"
    },
    {
      "id": 4,
      "asset_name": "Postman Application",
      "asset_type": "Application",
      "ip_address": "192.0.2.10",
      "operating_system": "Linux",
      "owner_id": 1,
      "department": "Security",
      "location": "Lab",
      "criticality": "High",
      "status": "Under Maintenance"
    },
    {
      "id": 5,
      "asset_name": "Postman Network Device",
      "asset_type": "Network Device",
      "ip_address": "192.0.2.10",
      "operating_system": "Linux",
      "owner_id": 1,
      "department": "Security",
      "location": "Lab",
      "criticality": "High",
      "status": "Active"
    },
    {
      "id": 6,
      "asset_name": "Postman Cloud Resource",
      "asset_type": "Cloud Resource",
      "ip_address": "192.0.2.10",
      "operating_system": "Linux",
      "owner_id": 1,
      "department": "Security",
      "location": "Lab",
      "criticality": "Critical",
      "status": "Active"
    }
  ]
 }
}
```

GET /assets/{asset_id} (Marc Bteich)

Purpose: get asset. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Path parameters: asset_id. IDs must identify existing records.

Postman example: `GET {{base_assets}}/assets/{{asset_id}}`.

Executed response example: HTTP 200; Newman request "Get asset before update (analyst)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Asset retrieved successfully",
  "data": {
    "id": 1,
    "asset_name": "Postman Lab Server",
    "asset_type": "Server",
    "ip_address": "192.0.2.10",
    "operating_system": "Linux",
    "owner_id": 1,
    "department": "Security",
    "location": "Lab",
    "criticality": "Critical",
    "status": "Active"
  }
}
```


GET /assets/search (Marc Bteich)

Purpose: search assets. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_assets}}/assets/search?name=Postman`.

Search parameters demonstrated: criticality, department, ip, name, type; consult field rules below for allowed keys.

Executed response example: HTTP 200; Newman request "Search by name". Credentials are redacted.


```
{
  "status": "success",
  "message": "Asset search completed",
  "data": {
    "count": 6,
    "assets": [
      {
        "id": 1,
        "asset_name": "Updated Postman Server",
        "asset_type": "Server",
        "ip_address": "192.0.2.10",
```

```
  "operating_system": "Linux",
  "owner_id": 1,
  "department": "IT",
  "location": "Lab",
  "criticality": "High",
  "status": "Under Maintenance"
},
{
  "id": 2,
  "asset_name": "Postman Workstation",
  "asset_type": "Workstation",
  "ip_address": "192.0.2.10",
  "operating_system": "Linux",
  "owner_id": 1,
  "department": "Security",
  "location": "Lab",
  "criticality": "Low",
  "status": "Active"
},
{
  "id": 3,
  "asset_name": "Postman Database",
  "asset_type": "Database",
  "ip_address": "192.0.2.10",
  "operating_system": "Linux",
  "owner_id": 1,
  "department": "Security",
  "location": "Lab",
  "criticality": "Medium",
  "status": "Inactive"
},
{
  "id": 4,
  "asset_name": "Postman Application",
  "asset_type": "Application",
  "ip_address": "192.0.2.10",
  "operating_system": "Linux",
  "owner_id": 1,
  "department": "Security",
  "location": "Lab",
  "criticality": "High",
  "status": "Under Maintenance"
},
{
  "id": 5,
  "asset_name": "Postman Network Device",
  "asset_type": "Network Device",
  "ip_address": "192.0.2.10",
  "operating_system": "Linux",
  "owner_id": 1,
  "department": "Security",
  "location": "Lab",
  "criticality": "High",
  "status": "Active"
},
{
```

```
        "id": 6,
        "asset_name": "Postman Cloud Resource",
        "asset_type": "Cloud Resource",
        "ip_address": "192.0.2.10",
        "operating_system": "Linux",
        "owner_id": 1,
        "department": "Security",
        "location": "Lab",
        "criticality": "Critical",
        "status": "Active"
      }
    ]
  }
}
```


GET /internal/stats (Marc Bteich)

Purpose: internal stats. Access: internal service only. Authentication: Service key.

Postman example: `GET {{base_assets}}/internal/stats`.

Executed response example: HTTP 200; Newman request "Internal statistics". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "total_assets": 6,
    "critical_assets": 1,
    "by_type": {
      "Server": 1,
      "Workstation": 1,
      "Database": 1,
      "Application": 1,
      "Network Device": 1,
      "Cloud Resource": 1
    }
  }
}
```


GET /internal/assets/{asset_id} (Marc Bteich)

Purpose: internal get asset. Access: internal service only. Authentication: Service key.

Path parameters: asset_id. IDs must identify existing records.

Postman example: `GET {{base_assets}}/internal/assets/{{asset_id}}`.

Executed response example: HTTP 200; Newman request "Internal asset lookup". Credentials are redacted.

```
{
  "status": "success",
  "message": "OK",
  "data": {
    "id": 1,
    "asset_name": "Updated Postman Server",
    "asset_type": "Server",
    "ip_address": "192.0.2.10",
    "operating_system": "Linux",
    "owner_id": 1,
    "department": "IT",
    "location": "Lab",
    "criticality": "High",
    "status": "Under Maintenance"
  }
}
```


Incidents (Marc Bteich)

POST /incidents (Marc Bteich)

Purpose: create incident. Access: admin,user. Authentication: Bearer JWT.

Postman example: `POST {{base_incidents}}/incidents`.

Request body example (template; credentials remain private):


```
{
  "title": "Phishing email received",
  "description": "Employee clicked a link in a fake invoice email",
  "category": "Phishing",
  "severity": "Medium",
  "asset_id": {{asset_id}}
}
```


Executed response example: HTTP 201; Newman request "Create - regular user (Medium)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Incident created successfully",
  "data": {
    "id": 1,
    "title": "Phishing email received",
    "description": "Employee clicked a link in a fake invoice email",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 7,
    "assigned_analyst_id": null,
```

```
    "status": "Open",
    "created_at": "2026-10-20T12:08:59Z",
    "resolved_at": null
  }
}
```


GET /incidents (Marc Bteich)

Purpose: get all incidents. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_incidents}}/incidents`.

Executed response example: HTTP 200; Newman request "Get all - admin". Credentials are redacted.


```
{
  "status": "success",
  "message": "Incidents retrieved successfully",
  "data": {
    "count": 4,
    "incidents": [
      {
        "id": 4,
        "title": "Port scan detected",
        "description": "Firewall blocked a port scan",
        "category": "Other",
        "severity": "Low",
        "reporter_id": 2,
        "asset_id": 7,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T12:08:59Z",
        "resolved_at": null
      },
      {
        "id": 3,
        "title": "Brute force on HR database",
        "description": "Thousands of failed logins from one IP",
        "category": "Unauthorized Access",
        "severity": "High",
        "reporter_id": 1,
        "asset_id": 7,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T12:08:59Z",
        "resolved_at": null
      },
      {
        "id": 2,
        "title": "Ransomware on web server",
        "description": "Files encrypted, ransom note found",
        "category": "Malware",
        "severity": "Critical",
        "reporter_id": 1,
```

```
        "asset_id": 7,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T12:08:59Z",
        "resolved_at": null
      },
      {
        "id": 1,
        "title": "Phishing email received",
        "description": "Employee clicked a link in a fake invoice email",
        "category": "Phishing",
        "severity": "Medium",
        "reporter_id": 2,
        "asset_id": 7,
        "assigned_analyst_id": null,
        "status": "Open",
        "created_at": "2026-10-20T12:08:59Z",
        "resolved_at": null
      }
    ]
  }
}
```


GET /incidents/{incident_id} (Marc Bteich)

Purpose: get incident. Access: admin,auditor,analyst,user. Authentication: Bearer JWT.

Path parameters: incident_id. IDs must identify existing records.

Postman example: `GET {{base_incidents}}/incidents/{{incident_id}}`.

Executed response example: HTTP 200; Newman request "Get - reporter views own incident". Credentials are redacted.


```
{
  "status": "success",
  "message": "Incident retrieved successfully",
  "data": {
    "id": 1,
    "title": "Phishing email received",
    "description": "Employee clicked a link in a fake invoice email",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 7,
    "assigned_analyst_id": null,
    "status": "Open",
    "created_at": "2026-10-20T12:08:59Z",
    "resolved_at": null,
    "investigation_notes": []
  }
}
```

PUT /incidents/{incident_id}/assign (Marc Bteich)

Purpose: assign incident. Access: admin. Authentication: Bearer JWT.

Path parameters: incident_id. IDs must identify existing records.

Postman example: `PUT {{base_incidents}}/incidents/{{incident_id}}/assign`.

Request body example (template; credentials remain private):

```
{
  "analyst_id": {{analyst_id}}
}
```

Executed response example: HTTP 200; Newman request "Assign - admin to analyst". Credentials are redacted.


```
{
  "status": "success",
  "message": "Incident assigned successfully",
  "data": {
    "id": 1,
    "title": "Phishing email received",
    "description": "Employee clicked a link in a fake invoice email",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 7,
    "assigned_analyst_id": 3,
    "status": "Open",
    "created_at": "2026-10-20T12:08:59Z",
    "resolved_at": null
  }
}
```


PUT /incidents/{incident_id}/status (Marc Bteich)

Purpose: update incident status. Access: admin,analyst. Authentication: Bearer JWT.

Path parameters: incident_id. IDs must identify existing records.

Postman example: `PUT {{base_incidents}}/incidents/{{incident_id}}/status`.

Request body example (template; credentials remain private):

```
{
  "status": "Under Investigation"
}
```

Executed response example: HTTP 200; Newman request "Open -> Under Investigation". Credentials are redacted.


```
{
  "status": "success",
  "message": "Incident status updated to 'Under Investigation'",
  "data": {
    "id": 1,
    "title": "Phishing email received",
    "description": "Employee clicked a link in a fake invoice email",
    "category": "Phishing",
    "severity": "Medium",
    "reporter_id": 2,
    "asset_id": 7,
    "assigned_analyst_id": 3,
    "status": "Under Investigation",
    "created_at": "2026-10-20T12:08:59Z",
    "resolved_at": null
  }
}
```


POST /incidents/{incident_id}/notes (Marc Bteich)

Purpose: add investigation note. Access: admin,analyst. Authentication: Bearer JWT.

Path parameters: incident_id. IDs must identify existing records.

Postman example: `POST {{base_incidents}}/incidents/{{incident2_id}}/notes`.

Request body example (template; credentials remain private):

```
{
  "content": "Analysis: ransomware strain identified as LockBit"
}
```

Executed response example: HTTP 201; Newman request "Add note - assigned analyst". Credentials are redacted.


```
{
  "status": "success",
  "message": "Investigation note added successfully",
  "data": {
    "id": 2,
    "title": "Ransomware on web server",
    "description": "Files encrypted, ransom note found",
    "category": "Malware",
    "severity": "Critical",
    "reporter_id": 1,
    "asset_id": 7,
```

```
    "assigned_analyst_id": 3,
    "status": "Open",
    "created_at": "2026-10-20T12:08:59Z",
    "resolved_at": null,
    "investigation_notes": [
      {
        "id": 1,
        "incident_id": 2,
        "analyst_id": 3,
        "content": "Analysis: ransomware strain identified as LockBit",
        "created_at": "2026-10-20T12:09:01Z"
      }
    ]
  }
}
```


GET /internal/stats (Marc Bteich)

Purpose: internal stats. Access: internal service only. Authentication: Service key.

Postman example: `GET {{base_assets}}/internal/stats`.

Executed response example: HTTP 200; Newman request "Internal statistics". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "total_assets": 6,
    "critical_assets": 1,
    "by_type": {
      "Server": 1,
      "Workstation": 1,
      "Database": 1,
      "Application": 1,
      "Network Device": 1,
      "Cloud Resource": 1
    }
  }
}
```


Threats (Marc Bteich)

POST /threats (Marc Bteich)

Purpose: create. Access: admin. Authentication: Bearer JWT.

Postman example: `POST {{base_threats}}/threats`.

Request body example (template; credentials remain private):

```
{
  "threat_type": "Domain",
  "threat_value": "{{indicator_domain}}",
  "description": "Synthetic Postman lab indicator",
  "severity": "Critical",
  "source": "Postman lab"
}
```

Executed response example: HTTP 201; Newman request "Create Critical threat". Credentials are redacted.

```
{
  "status": "success",
  "message": "Threat created",
  "data": {
    "id": 1,
    "threat_type": "Domain",
    "threat_value": "postman.example.test",
    "description": "Synthetic Postman lab indicator",
    "severity": "Critical",
    "source": "Postman lab",
    "date_added": "2026-10-20T12:09:03Z"
  }
}
```

GET /threats (Marc Bteich)

Purpose: list all. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_threats}}/threats`.

Executed response example: HTTP 200; Newman request "List threats". Credentials are redacted.

```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 1,
    "threats": [
      {
        "id": 1,
        "threat_type": "Domain",
        "threat_value": "postman.example.test",
        "description": "Synthetic Postman lab indicator",
        "severity": "Critical",
        "source": "Postman lab",
        "date_added": "2026-10-20T12:09:03Z"
      }
    ]
```

```
  }
}
```


GET /threats/{record_id} (Marc Bteich)

Purpose: get. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `GET {{base_threats}}/threats/{{threat_id}}`.

Executed response example: HTTP 200; Newman request "Get threat". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "id": 1,
    "threat_type": "Domain",
    "threat_value": "postman.example.test",
    "description": "Synthetic Postman lab indicator",
    "severity": "Critical",
    "source": "Postman lab",
    "date_added": "2026-10-20T12:09:03Z"
  }
}
```


GET /threats/search (Marc Bteich)

Purpose: search. Access: admin,analyst. Authentication: Bearer JWT.

Postman example: `GET {{base_threats}}/threats/search?type=Domain`.

Search parameters demonstrated: severity, source, type, value; consult field rules below for allowed keys.

Executed response example: HTTP 200; Newman request "Search type=Domain". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 1,
    "threats": [
      {
        "id": 1,
        "threat_type": "Domain",
        "threat_value": "postman.example.test",
        "description": "Synthetic Postman lab indicator",
        "severity": "Critical",
```

```
        "source": "Postman lab",
        "date_added": "2026-10-20T12:09:03Z"
      }
    ]
  }
}
```

PUT /threats/{record_id} (Marc Bteich)

Purpose: update. Access: admin. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `PUT {{base_threats}}/threats/{{threat_id}}`.

Request body example (template; credentials remain private):

```
{
  "description": "Reviewed",
  "severity": "High",
  "source": "Reviewed lab"
}
```

Executed response example: HTTP 200; Newman request "Update threat". Credentials are redacted.

```
{
  "status": "success",
  "message": "Threat updated",
  "data": {
    "id": 1,
    "threat_type": "Domain",
    "threat_value": "postman.example.test",
    "description": "Reviewed",
    "severity": "High",
    "source": "Reviewed lab",
    "date_added": "2026-10-20T12:09:03Z"
  }
}
```

DELETE /threats/{record_id} (Marc Bteich)

Purpose: delete. Access: admin. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `DELETE {{base_threats}}/threats/{{threat_id}}`.

Executed response example: HTTP 200; Newman request "Delete threat". Credentials are redacted.


```
{
  "status": "success",
  "message": "Threat deleted",
  "data": null
}
```


GET /internal/stats (Marc Bteich)

Purpose: stats. Access: internal service only. Authentication: Service key.

Postman example: `GET {{base_assets}}/internal/stats`.

Executed response example: HTTP 200; Newman request "Internal statistics". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "total_assets": 6,
    "critical_assets": 1,
    "by_type": {
      "Server": 1,
      "Workstation": 1,
      "Database": 1,
      "Application": 1,
      "Network Device": 1,
      "Cloud Resource": 1
    }
  }
}
```


Logs (Marc Bteich)

POST /logs (Marc Bteich)

Purpose: create log. Access: admin. Authentication: Bearer JWT.

Postman example: `POST {{base_logs}}/logs`.

Request body example (template; credentials remain private):


```
{
  "user_id": {{user_id}},
  "action": "ACCOUNT_REVIEW",
  "description": "Quarterly review of account activity",
```

```
  "severity": "Information",
  "related_service": "users"
}
```


Executed response example: HTTP 201; Newman request "Create log - admin (Information)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Log record created successfully",
  "data": {
    "id": 79,
    "user_id": 2,
    "username": "rami",
    "action": "ACCOUNT_REVIEW",
    "description": "Quarterly review of account activity",
    "severity": "Information",
    "related_service": "users",
    "timestamp": "2026-10-20T12:09:07Z"
  }
}
```


GET /logs (Marc Bteich)

Purpose: get all logs. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_logs}}/logs`.

Executed response example: HTTP 200; Newman request "Get all logs - admin". Credentials are redacted.

Full response is retained without truncation in evidence/newman-combined.json for that request; large list output is omitted here for readability.

GET /logs/search (Marc Bteich)

Purpose: search logs. Access: admin,auditor,analyst. Authentication: Bearer JWT.

Postman example: `GET {{base_logs}}/logs/search?service=assets`.

Search parameters demonstrated: action, date, date_from, date_to, service, severity, user; consult field rules below for allowed keys.

Executed response example: HTTP 200; Newman request "Inspect Assets audit records". Credentials are redacted.


```
{
  "status": "success",
  "message": "7 log(s) match the search criteria",
```

```
"data": {
  "count": 7,
  "logs": [
    {
      "id": 30,
      "user_id": 1,
      "username": "admin",
      "action": "ASSET_UPDATED",
      "description": "Updated asset 1: asset_name, criticality, department, status",
      "severity": "Information",
      "related_service": "assets",
      "timestamp": "2026-10-20T12:08:54Z"
    },
    {
      "id": 29,
      "user_id": 1,
      "username": "admin",
      "action": "ASSET_CREATED",
      "description": "Created asset 6",
      "severity": "Information",
      "related_service": "assets",
      "timestamp": "2026-10-20T12:08:54Z"
    },
    {
      "id": 28,
      "user_id": 1,
      "username": "admin",
      "action": "ASSET_CREATED",
      "description": "Created asset 5",
      "severity": "Information",
      "related_service": "assets",
      "timestamp": "2026-10-20T12:08:54Z"
    },
    {
      "id": 27,
      "user_id": 1,
      "username": "admin",
      "action": "ASSET_CREATED",
      "description": "Created asset 4",
      "severity": "Information",
      "related_service": "assets",
      "timestamp": "2026-10-20T12:08:54Z"
    },
    {
      "id": 26,
      "user_id": 1,
      "username": "admin",
      "action": "ASSET_CREATED",
      "description": "Created asset 3",
      "severity": "Information",
      "related_service": "assets",
      "timestamp": "2026-10-20T12:08:54Z"
    },
    {
      "id": 25,
      "user_id": 1,
```

```
        "username": "admin",
        "action": "ASSET_CREATED",
        "description": "Created asset 2",
        "severity": "Information",
        "related_service": "assets",
        "timestamp": "2026-10-20T12:08:54Z"
      },
      {
        "id": 24,
        "user_id": 1,
        "username": "admin",
        "action": "ASSET_CREATED",
        "description": "Created asset 1",
        "severity": "Information",
        "related_service": "assets",
        "timestamp": "2026-10-20T12:08:53Z"
      }
    ]
  }
}
```


GET /logs/user/{user_id} (Marc Bteich)

Purpose: get logs by user. Access: admin,auditor. Authentication: Bearer JWT.

Path parameters: user_id. IDs must identify existing records.

Postman example: `GET {{base_logs}}/logs/user/{{user_id}}`.

Executed response example: HTTP 200; Newman request "Logs by user - user with logs". Credentials are redacted.

Full response is retained without truncation in evidence/newman-combined.json for that request; large list output is omitted here for readability.

GET /logs/{log_id} (Marc Bteich)

Purpose: get log. Access: admin,auditor,analyst. Authentication: Bearer JWT.

Path parameters: log_id. IDs must identify existing records.

Postman example: `GET {{base_logs}}/logs/search?service=assets`.

Executed response example: HTTP 200; Newman request "Get log by id - admin". Credentials are redacted.


```
{
  "status": "success",
  "message": "Log retrieved successfully",
  "data": {
    "id": 79,
    "user_id": 2,
```

```
    "username": "rami",
    "action": "ACCOUNT_REVIEW",
    "description": "Quarterly review of account activity",
    "severity": "Information",
    "related_service": "users",
    "timestamp": "2026-10-20T12:09:07Z"
  }
}
```


DELETE /logs/{log_id} (Marc Bteich)

Purpose: delete log. Access: admin. Authentication: Bearer JWT.

Path parameters: log_id. IDs must identify existing records.

Postman example: `DELETE {{base_logs}}/logs/{{log_id}}`.

Executed response example: HTTP 200; Newman request "Delete log - admin (success)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Log 79 deleted successfully",
  "data": null
}
```


GET /internal/stats (Marc Bteich)

Purpose: internal stats. Access: internal service only. Authentication: Service key.

Postman example: `GET {{base_assets}}/internal/stats`.

Executed response example: HTTP 200; Newman request "Internal statistics". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "total_assets": 6,
    "critical_assets": 1,
    "by_type": {
      "Server": 1,
      "Workstation": 1,
      "Database": 1,
      "Application": 1,
      "Network Device": 1,
      "Cloud Resource": 1
    }
  }
```

```
}
```

Reports (Marc Bteich)

GET /reports/summary (Marc Bteich)

Purpose: security summary report. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_reports}}/reports/summary`.

Executed response example: HTTP 200; Newman request "Summary - admin". Credentials are redacted.


```
{
  "status": "success",
  "message": "Security summary report generated",
  "data": {
    "report_type": "security summary",
    "generated_at": "2026-10-20T12:09:21Z",
    "generated_by": "admin",
    "users": 4,
    "assets": 6,
    "incidents": 4,
    "open_incidents": 3,
    "critical_incidents": 1,
    "threats": 0,
    "critical_threats": 0
  }
}
```


GET /reports/incidents (Marc Bteich)

Purpose: incident report. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_reports}}/reports/incidents`.

Executed response example: HTTP 200; Newman request "Incident report - admin". Credentials are redacted.


```
{
  "status": "success",
  "message": "Incident report generated",
  "data": {
    "report_type": "incident",
    "generated_at": "2026-10-20T12:09:22Z",
    "generated_by": "admin",
    "total_incidents": 4,
    "open_incidents": 3,
    "unassigned_open_incidents": 2,
    "incidents_by_severity": {
      "Low": 1,
```

```
      "Medium": 1,
      "High": 1,
      "Critical": 1
    },
    "incidents_by_status": {
      "Open": 3,
      "Under Investigation": 0,
      "Mitigated": 0,
      "Resolved": 0,
      "Closed": 1
    },
    "incidents_by_category": {
      "Malware": 1,
      "Unauthorized Access": 1,
      "Phishing": 1,
      "Data Leakage": 0,
      "Account Compromise": 0,
      "Denial of Service": 0,
      "Other": 1
    }
  }
}
```


GET /reports/activity (Marc Bteich)

Purpose: activity report. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_reports}}/reports/activity`.

Executed response example: HTTP 200; Newman request "Activity report - admin". Credentials are redacted.


```
{
  "status": "success",
  "message": "Activity report generated",
  "data": {
    "report_type": "activity",
    "generated_at": "2026-10-20T12:09:22Z",
    "generated_by": "admin",
    "total_user_activities": 144,
    "login_attempts": 38,
    "successful_logins": 36,
    "failed_login_attempts": 2,
    "failed_login_rate_percent": 5.3,
    "unauthorized_access_attempts": 44,
    "activities_by_severity": {
      "Information": 92,
      "Warning": 50,
      "Error": 0,
      "Critical": 2
    },
    "activities_by_service": {
      "assets": 18,
      "incidents": 20,
```

```
      "logs": 9,
      "notifications": 12,
      "playbooks": 13,
      "reports": 3,
      "threats": 6,
      "users": 52,
      "vulnerabilities": 11
    },
    "most_performed_actions": [
      {
        "action": "UNAUTHORIZED_ACCESS",
        "count": 44
      },
      {
        "action": "LOGIN_SUCCESS",
        "count": 36
      },
      {
        "action": "ASSET_CREATED",
        "count": 9
      },
      {
        "action": "ACTION_HISTORY_ARCHIVED",
        "count": 4
      },
      {
        "action": "INCIDENT_CREATED",
        "count": 4
      }
    ]
  }
}
```


GET /reports/assets (Marc Bteich)

Purpose: assets. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_reports}}/reports/assets`.

Executed response example: HTTP 200; Newman request "Generate assets report (admin)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Asset report generated",
  "data": {
    "report_type": "asset",
    "generated_at": "2026-10-20T12:09:24Z",
    "generated_by": "admin",
    "total_assets": 6,
    "critical_assets": 1,
    "assets_by_type": {
      "Server": 1,
      "Workstation": 1,
```

```
      "Database": 1,
      "Application": 1,
      "Network Device": 1,
      "Cloud Resource": 1
    }
  }
}
```


GET /reports/threats (Marc Bteich)

Purpose: threats. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_reports}}/reports/threats`.

Executed response example: HTTP 200; Newman request "Generate threats report (admin)". Credentials are redacted.


```
{
  "status": "success",
  "message": "Threat report generated",
  "data": {
    "report_type": "threat",
    "generated_at": "2026-10-20T12:09:24Z",
    "generated_by": "admin",
    "total_threats": 0,
    "threats_by_severity": {
      "Low": 0,
      "Medium": 0,
      "High": 0,
      "Critical": 0
    },
    "threats_by_type": {
      "IP Address": 0,
      "Domain": 0,
      "File Hash": 0,
      "Malware Signature": 0,
      "Other": 0
    },
    "recently_added": []
  }
}
```


GET /reports/vulnerabilities (Marc Bteich)

Purpose: vulnerabilities. Access: admin,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_reports}}/reports/vulnerabilities`.

Executed response example: HTTP 200; Newman request "Generate vulnerabilities report (admin)". Credentials are redacted.

```
{
  "status": "success",
  "message": "Vulnerability report generated",
  "data": {
    "report_type": "vulnerability",
    "generated_at": "2026-10-20T12:09:25Z",
    "generated_by": "admin",
    "total_vulnerabilities": 0,
    "critical_vulnerabilities": 0,
    "vulnerabilities_by_severity": {
      "Low": 0,
      "Medium": 0,
      "High": 0,
      "Critical": 0
    },
    "vulnerabilities_by_status": {
      "Open": 0,
      "Under Investigation": 0,
      "In Progress": 0,
      "Resolved": 0,
      "Closed": 0
    }
  }
}
```


Vulnerabilities (Marc Bteich)

POST /vulnerabilities (Marc Bteich)

Purpose: create. Access: admin. Authentication: Bearer JWT.

Postman example: `POST {{base_vulnerabilities}}/vulnerabilities`.

Request body example (template; credentials remain private):


```
{
  "title": "Synthetic Postman vulnerability",
  "description": "Lab scenario only",
  "cve_id": "CVE-2026-12345",
  "cvss_score": 9.8,
  "severity": "Low",
  "asset_id": {{linked_asset_id}},
  "discovery_source": "Postman lab"
}
```


Executed response example: HTTP 201; Newman request "Link vulnerability". Credentials are redacted.


```
{
  "status": "success",
```

```
  "message": "Vulnerability created",
  "data": {
    "id": 1,
    "title": "Synthetic Postman vulnerability",
    "description": "Lab scenario only",
    "cve_id": "CVE-2026-12345",
    "cvss_score": 9.8,
    "severity": "Low",
    "asset_id": 7,
    "discovery_source": "Postman lab",
    "status": "Open",
    "created_at": "2026-10-20T12:08:57Z",
    "resolution_date": null
  }
}
```


GET /vulnerabilities (Marc Bteich)

Purpose: list all. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_vulnerabilities}}/vulnerabilities`.

Executed response example: HTTP 200; Newman request "List vulnerabilities". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 1,
    "vulnerabilities": [
      {
        "id": 1,
        "title": "Synthetic Postman vulnerability",
        "description": "Lab scenario only",
        "cve_id": "CVE-2026-12345",
        "cvss_score": 9.8,
        "severity": "Critical",
        "asset_id": 8,
        "discovery_source": "Postman lab",
        "status": "Open",
        "created_at": "2026-10-20T12:09:11Z",
        "resolution_date": null
      }
    ]
  }
}
```


GET /vulnerabilities/{record_id} (Marc Bteich)

Purpose: get. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `GET {{base_vulnerabilities}}/vulnerabilities/{{vulnerability_id}}`.

Executed response example: HTTP 200; Newman request "Get vulnerability". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "id": 1,
    "title": "Synthetic Postman vulnerability",
    "description": "Lab scenario only",
    "cve_id": "CVE-2026-12345",
    "cvss_score": 9.8,
    "severity": "Critical",
    "asset_id": 8,
    "discovery_source": "Postman lab",
    "status": "Open",
    "created_at": "2026-10-20T12:09:11Z",
    "resolution_date": null
  }
}
```


PUT /vulnerabilities/{record_id} (Marc Bteich)

Purpose: update. Access: admin,analyst. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `PUT {{base_vulnerabilities}}/vulnerabilities/{{vulnerability_id}}`.

Request body example (template; credentials remain private):


```
{
  "status": "Closed"
}
```


Executed response example: HTTP 200; Newman request "Transition Under Investigation". Credentials are redacted.


```
{
  "status": "success",
  "message": "Vulnerability updated",
  "data": {
    "id": 1,
    "title": "Synthetic Postman vulnerability",
    "description": "Lab scenario only",
    "cve_id": "CVE-2026-12345",
    "cvss_score": 9.8,
```

```
    "severity": "Critical",
    "asset_id": 8,
    "discovery_source": "Postman lab",
    "status": "Under Investigation",
    "created_at": "2026-10-20T12:09:11Z",
    "resolution_date": null
  }
}
```


GET /vulnerabilities/search (Marc Bteich)

Purpose: search. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_vulnerabilities}}/vulnerabilities/search?cve=CVE-2026-12345`.

Search parameters demonstrated: asset, cve, severity, status; consult field rules below for allowed keys.

Executed response example: HTTP 200; Newman request "Search cve=CVE-2026-12345". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 1,
    "vulnerabilities": [
      {
        "id": 1,
        "title": "Synthetic Postman vulnerability",
        "description": "Lab scenario only",
        "cve_id": "CVE-2026-12345",
        "cvss_score": 9.8,
        "severity": "Critical",
        "asset_id": 8,
        "discovery_source": "Postman lab",
        "status": "Open",
        "created_at": "2026-10-20T12:09:11Z",
        "resolution_date": null
      }
    ]
  }
}
```


DELETE /vulnerabilities/{record_id} (Marc Bteich)

Purpose: delete. Access: admin. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `DELETE {{base_vulnerabilities}}/vulnerabilities/{{linked_vulnerability_id}}`.

Executed response example: HTTP 200; Newman request "Remove fixture vulnerability". Credentials are redacted.


```
{
  "status": "success",
  "message": "Vulnerability deleted",
  "data": null
}
```


GET /internal/stats (Marc Bteich)

Purpose: stats. Access: internal service only. Authentication: Service key.

Postman example: `GET {{base_assets}}/internal/stats`.

Executed response example: HTTP 200; Newman request "Internal statistics". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "total_assets": 6,
    "critical_assets": 1,
    "by_type": {
      "Server": 1,
      "Workstation": 1,
      "Database": 1,
      "Application": 1,
      "Network Device": 1,
      "Cloud Resource": 1
    }
  }
}
```


Notifications (Marc Bteich)

POST /notifications (Marc Bteich)

Purpose: create notification. Access: admin. Authentication: Bearer JWT.

Postman example: `POST {{base_notifications}}/notifications`.

Request body example (template; credentials remain private):


```
{
```

```
  "user_id": {{user_id}},
  "title": "Security awareness training",
  "message": "Mandatory training on Friday",
  "notification_type": "System Notification",
  "severity": "Low"
}
```

Executed response example: HTTP 201; Newman request "Create - system notification". Credentials are redacted.

```
{
  "status": "success",
  "message": "Notification created successfully",
  "data": {
    "id": 9,
    "user_id": 2,
    "title": "Security awareness training",
    "message": "Mandatory training on Friday",
    "notification_type": "System Notification",
    "severity": "Low",
    "status": "Unread",
    "created_at": "2026-10-20T12:09:15Z",
    "read_at": null
  }
}
```

GET /notifications/user/{user_id} (Marc Bteich)

Purpose: get user notifications. Access: admin,analyst,user,auditor. Authentication: Bearer JWT.

Path parameters: user_id. IDs must identify existing records.

Postman example: `GET {{base_notifications}}/notifications/user/{{user_id}}`.

Executed response example: HTTP 200; Newman request "User's own notifications". Credentials are redacted.

```
{
  "status": "success",
  "message": "6 notification(s) found for user 2",
  "data": {
    "count": 6,
    "notifications": [
      {
        "id": 10,
        "user_id": 2,
        "title": "Malicious IP blocked",
        "message": "185.220.101.4 tried to reach your laptop",
        "notification_type": "Threat Alert",
        "severity": "High",
        "status": "Unread",
```

```
      "created_at": "2026-10-20T12:09:15Z",
      "read_at": null
    },
    {
      "id": 9,
      "user_id": 2,
      "title": "Security awareness training",
      "message": "Mandatory training on Friday",
      "notification_type": "System Notification",
      "severity": "Low",
      "status": "Unread",
      "created_at": "2026-10-20T12:09:15Z",
      "read_at": null
    },
    {
      "id": 6,
      "user_id": 2,
      "title": "Incident #1 is now Closed",
      "message": "The status of your reported incident 'Phishing email received' changed
from Resolved to Closed.",
      "notification_type": "System Notification",
      "severity": "Medium",
      "status": "Unread",
      "created_at": "2026-10-20T12:09:01Z",
      "read_at": null
    },
    {
      "id": 5,
      "user_id": 2,
      "title": "Incident #1 is now Resolved",
      "message": "The status of your reported incident 'Phishing email received' changed
from Mitigated to Resolved.",
      "notification_type": "System Notification",
      "severity": "Medium",
      "status": "Unread",
      "created_at": "2026-10-20T12:09:01Z",
      "read_at": null
    },
    {
      "id": 4,
      "user_id": 2,
      "title": "Incident #1 is now Mitigated",
      "message": "The status of your reported incident 'Phishing email received' changed
from Under Investigation to Mitigated.",
      "notification_type": "System Notification",
      "severity": "Medium",
      "status": "Unread",
      "created_at": "2026-10-20T12:09:01Z",
      "read_at": null
    },
    {
      "id": 3,
      "user_id": 2,
      "title": "Incident #1 is now Under Investigation",
      "message": "The status of your reported incident 'Phishing email received' changed
from Open to Under Investigation.",
```

```
        "notification_type": "System Notification",
        "severity": "Medium",
        "status": "Unread",
        "created_at": "2026-10-20T12:09:01Z",
        "read_at": null
      }
    ]
  }
}
```


GET /notifications/search (Marc Bteich)

Purpose: search notifications. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_notifications}}/notifications/search?type=Threat%20Alert`.

Search parameters demonstrated: severity, status, type, user; consult field rules below for allowed keys.

Executed response example: HTTP 200; Newman request "Critical threat notification". Credentials are redacted.


```
{
  "status": "success",
  "message": "1 notification(s) match the search criteria",
  "data": {
    "count": 1,
    "notifications": [
      {
        "id": 7,
        "user_id": 1,
        "title": "Critical threat #1",
        "message": "postman.example.test",
        "notification_type": "Threat Alert",
        "severity": "Critical",
        "status": "Unread",
        "created_at": "2026-10-20T12:09:03Z",
        "read_at": null
      }
    ]
  }
}
```


GET /notifications/{notification_id} (Marc Bteich)

Purpose: get notification. Access: admin,analyst,user,auditor. Authentication: Bearer JWT.

Path parameters: notification_id. IDs must identify existing records.

Postman example: `GET {{base_notifications}}/notifications/search?type=Threat%20Alert`.

Executed response example: HTTP 200; Newman request "Get - owner". Credentials are redacted.


```
{
  "status": "success",
  "message": "Notification retrieved successfully",
  "data": {
    "id": 9,
    "user_id": 2,
    "title": "Security awareness training",
    "message": "Mandatory training on Friday",
    "notification_type": "System Notification",
    "severity": "Low",
    "status": "Unread",
    "created_at": "2026-10-20T12:09:15Z",
    "read_at": null
  }
}
```


PUT /notifications/{notification_id}/read (Marc Bteich)

Purpose: mark notification read. Access: admin,analyst,user. Authentication: Bearer JWT.

Path parameters: notification_id. IDs must identify existing records.

Postman example: `PUT {{base_notifications}}/notifications/{{notification_id}}/read`.

Executed response example: HTTP 200; Newman request "Mark read - owner". Credentials are redacted.


```
{
  "status": "success",
  "message": "Notification marked as read",
  "data": {
    "id": 9,
    "user_id": 2,
    "title": "Security awareness training",
    "message": "Mandatory training on Friday",
    "notification_type": "System Notification",
    "severity": "Low",
    "status": "Read",
    "created_at": "2026-10-20T12:09:15Z",
    "read_at": "2026-10-20T12:09:16Z"
  }
}
```

DELETE /notifications/{notification_id} (Marc Bteich)

Purpose: delete notification. Access: admin. Authentication: Bearer JWT.

Path parameters: notification_id. IDs must identify existing records.

Postman example: `DELETE {{base_notifications}}/notifications/{{notification_id}}`.

Executed response example: HTTP 200; Newman request "Delete - admin". Credentials are redacted.


```
{
  "status": "success",
  "message": "Notification 9 deleted successfully",
  "data": null
}
```


Playbooks (Marc Bteich)

POST /playbooks (Marc Bteich)

Purpose: create. Access: admin. Authentication: Bearer JWT.

Postman example: `POST {{base_playbooks}}/playbooks`.

Request body example (template; credentials remain private):


```
{
  "name": "Postman laboratory response",
  "description": "Lab procedure",
  "incident_category": "Malware",
  "severity": "High"
}
```


Executed response example: HTTP 201; Newman request "Create playbook". Credentials are redacted.


```
{
  "status": "success",
  "message": "Playbook created",
  "data": {
    "id": 1,
    "name": "Postman laboratory response",
    "description": "Lab procedure",
    "incident_category": "Malware",
    "severity": "High",
    "created_by": 1,
    "created_at": "2026-10-20T12:09:18Z"
  }
```

```
}
```

GET /playbooks (Marc Bteich)

Purpose: list all. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Postman example: `GET {{base_playbooks}}/playbooks`.

Executed response example: HTTP 200; Newman request "List playbooks". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "count": 1,
    "playbooks": [
      {
        "id": 1,
        "name": "Postman laboratory response",
        "description": "Lab procedure",
        "incident_category": "Malware",
        "severity": "High",
        "created_by": 1,
        "created_at": "2026-10-20T12:09:18Z"
      }
    ]
  }
}
```


GET /playbooks/{record_id} (Marc Bteich)

Purpose: get. Access: admin,analyst,auditor. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `GET {{base_playbooks}}/playbooks/{{playbook_id}}`.

Executed response example: HTTP 200; Newman request "Verify ordered actions". Credentials are redacted.


```
{
  "status": "success",
  "message": "OK",
  "data": {
    "id": 1,
    "name": "Postman laboratory response",
    "description": "Lab procedure",
    "incident_category": "Malware",
    "severity": "High",
    "created_by": 1,
```

```
    "created_at": "2026-10-20T12:09:18Z",
    "actions": [
      {
        "id": 2,
        "playbook_id": 1,
        "description": "Collect evidence",
        "step_number": 1,
        "status": "Pending",
        "assigned_user_id": null,
        "completion_date": null,
        "execution_history": [
          {
            "actor_id": 3,
            "at": "2026-10-20T12:09:18Z",
            "from_status": null,
            "to_status": "Pending",
            "assigned_user_id": null,
            "completion_date": null
          }
        ]
      },
      {
        "id": 1,
        "playbook_id": 1,
        "description": "Contain host",
        "step_number": 2,
        "status": "Pending",
        "assigned_user_id": null,
        "completion_date": null,
        "execution_history": [
          {
            "actor_id": 3,
            "at": "2026-10-20T12:09:18Z",
            "from_status": null,
            "to_status": "Pending",
            "assigned_user_id": null,
            "completion_date": null
          }
        ]
      }
    ]
  }
}
```


POST /playbooks/{record_id}/actions (Marc Bteich)

Purpose: add action. Access: admin,analyst. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `POST {{base_playbooks}}/playbooks/{{playbook_id}}/actions.`

Request body example (template; credentials remain private):

```
{
  "description": "Contain host",
  "step_number": 2
}
```

Executed response example: HTTP 201; Newman request "Add second step first". Credentials are redacted.


```
{
  "status": "success",
  "message": "Action created",
  "data": {
    "id": 1,
    "playbook_id": 1,
    "description": "Contain host",
    "step_number": 2,
    "status": "Pending",
    "assigned_user_id": null,
    "completion_date": null,
    "execution_history": [
      {
        "actor_id": 3,
        "at": "2026-10-20T12:09:18Z",
        "from_status": null,
        "to_status": "Pending",
        "assigned_user_id": null,
        "completion_date": null
      }
    ]
  }
}
```


PUT /actions/{record_id}/status (Marc Bteich)

Purpose: update action. Access: admin,analyst. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `PUT {{base_playbooks}}/actions/{{action_id}}/status`.

Request body example (template; credentials remain private):

```
{
  "status": "Completed"
}
```

Executed response example: HTTP 200; Newman request "Start assigned action". Credentials are redacted.

```
{
  "status": "success",
  "message": "Action updated",
  "data": {
    "id": 2,
    "playbook_id": 1,
    "description": "Collect evidence",
    "step_number": 1,
    "status": "In Progress",
    "assigned_user_id": 3,
    "completion_date": null,
    "execution_history": [
      {
        "actor_id": 3,
        "at": "2026-10-20T12:09:18Z",
        "from_status": null,
        "to_status": "Pending",
        "assigned_user_id": null,
        "completion_date": null
      },
      {
        "actor_id": 3,
        "at": "2026-10-20T12:09:19Z",
        "from_status": "Pending",
        "to_status": "In Progress",
        "assigned_user_id": 3,
        "completion_date": null
      }
    ]
  }
}
```


DELETE /playbooks/{record_id} (Marc Bteich)

Purpose: delete. Access: admin. Authentication: Bearer JWT.

Path parameters: record_id. IDs must identify existing records.

Postman example: `DELETE {{base_playbooks}}/playbooks/{{playbook_id}}`.

Executed response example: HTTP 200; Newman request "Delete playbook". Credentials are redacted.


```
{
  "status": "success",
  "message": "Playbook deleted",
  "data": null
}
```

Person B field rules and responses (Marc Bteich)

Service	Required create fields	Allowed updates / behavior
Assets	**asset_name, asset_type, ip_address, owner_id, department, criticality**	**name, owner, department, criticality, status; optional operating_system/location on create; status defaults Active**
Threats	**threat_type, threat_value, description, severity, source**	**description, severity, source; canonical IP/domain/hash; unique type/value; recent means latest five by date/id**
Vulnerabilities	**title, description, cve_id, cvss_score, severity, asset_id, discovery_source**	**description, severity, CVSS, next status only; Open initially; resolution_date automatic on Resolved**
Playbooks	**name, description, incident_category, severity**	**add actions with description/positive step_number and optional assigned_user_id; update action with next status and/or assigned_user_id; completion_date automatic**
Reports	**no body**	**HTTP source stats only; admin/auditor; unavailable source returns 503**

Assets types: Server, Workstation, Database, Application, Network Device, Cloud Resource. Threat types follow common/enums.py exactly: IP Address, Domain, File Hash, Malware Signature, Other. File hashes accept 32/40/64 hex characters. CVEs match CVE-[four digits]-[at least four digits]. CVSS is a finite JSON number 0–10 with at most one decimal place, matching NUMERIC(3,1). Severity/criticality: Low, Medium, High, Critical.

Vulnerability lifecycle: Open → Under Investigation → In Progress → Resolved → Closed. Action lifecycle: Pending → In Progress → Completed. Same-state, skipped and backward transitions are rejected. Assign an active user before action execution. Client-supplied history, completion dates and resolution dates are rejected.

Search: Assets name/type/ip/department/criticality; Threats type/value/severity/source; Vulnerabilities cve/severity/asset/status. Text fragments match literally and case-insensitively; other values are exact. Multiple filters combine with AND. Empty searches, duplicate

parameters, unknown fields and invalid values return 400. No matches return an empty list with 200.

List/search responses use data: {count, assets|threats|vulnerabilities|playbooks: [...]}. Single resources return data containing their fields and id. A playbook detail includes actions sorted by step_number; action responses include execution_history. Reports contain report_type/generated_at/generated_by plus named totals/groupings. See the actual redacted request/response evidence for executed examples.

9. Authentication and Authorization (Elie Medawar)

Every protected request carries a signed JWT that proves who the user is and what role they have; each service checks that token and the role before doing anything.

Authentication mechanism.

1. The user calls POST /users/login with a username (or email) and password.
2. The Users service finds the account and checks the password with bcrypt.checkpw against the stored hash. Plain-text passwords are never stored.
3. Inactive or Suspended accounts are refused with 403 even when the password is correct.
4. On success the service signs a JWT (HS256, secret JWT_SECRET from .env) with user_id, username, role and an expiry of 60 minutes, and returns it.
5. The client sends Authorization: Bearer <token> on every later request, to any service.

Because every service shares the same secret, each one can verify a token on its own, without calling the Users service on every request. Every success, failure and blocked attempt is written to the audit log.

Role	Code	Summary of access
Security Administrator	admin	Manages users, assets, incidents, threats, logs, vulnerabilities, notifications, playbooks; generates reports
Security Analyst	analyst	Investigates the incidents assigned to them; reads assets, threats, vulnerabilities, playbooks; no user management or deletions
Regular Employee/User	user	Registers, updates own profile, reports incidents and tracks their own reports, reads own notifications
Auditor	auditor	Read-only: users, assets, incidents, threats, logs, vulnerabilities, notifications, playbooks, reports

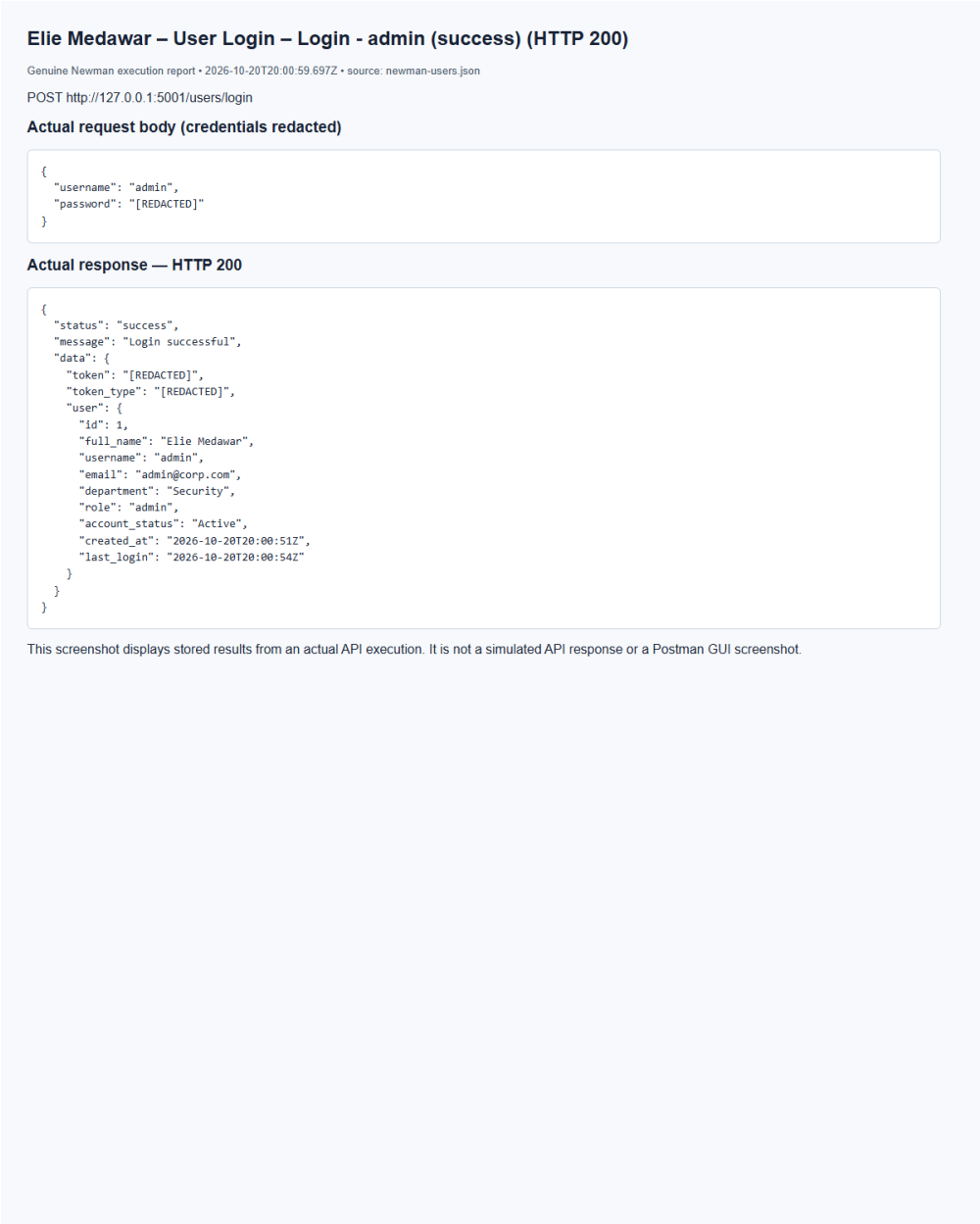
Permission management and access control rules. Each route declares its allowed roles with the shared decorator, for example @require_auth(roles=[ADMIN, AUDITOR]). For each request the decorator:

- answers 401 when the token is missing, malformed, wrongly signed or expired;
- answers 403 when the token's role is not in the route's list, and writes an UNAUTHORIZED_ACCESS record (severity Warning) to the audit log;
- otherwise puts the user in g.user and runs the route.

Some rules depend on the record itself, so the service checks them after loading it: a user may only view or update their own account and incidents, and an analyst may only view or work on incidents assigned to them. These denials are also logged. Calls between services use the X-Service-Key header instead of a user token, and /internal/* endpoints accept nothing else.

The screenshots show successful and failed authentication, then the same API (Retrieve All Users) called with insufficient permissions and with the correct permissions.

Elie Medawar – User Login – Login - admin (success) (HTTP 200)



Elie Medawar – User Login – Login - admin (success) (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:00:59.697Z • source: newman-users.json

POST http://127.0.0.1:5001/users/login

Actual request body (credentials redacted)


```
{
  "username": "admin",
  "password": "[REDACTED]"
}
```

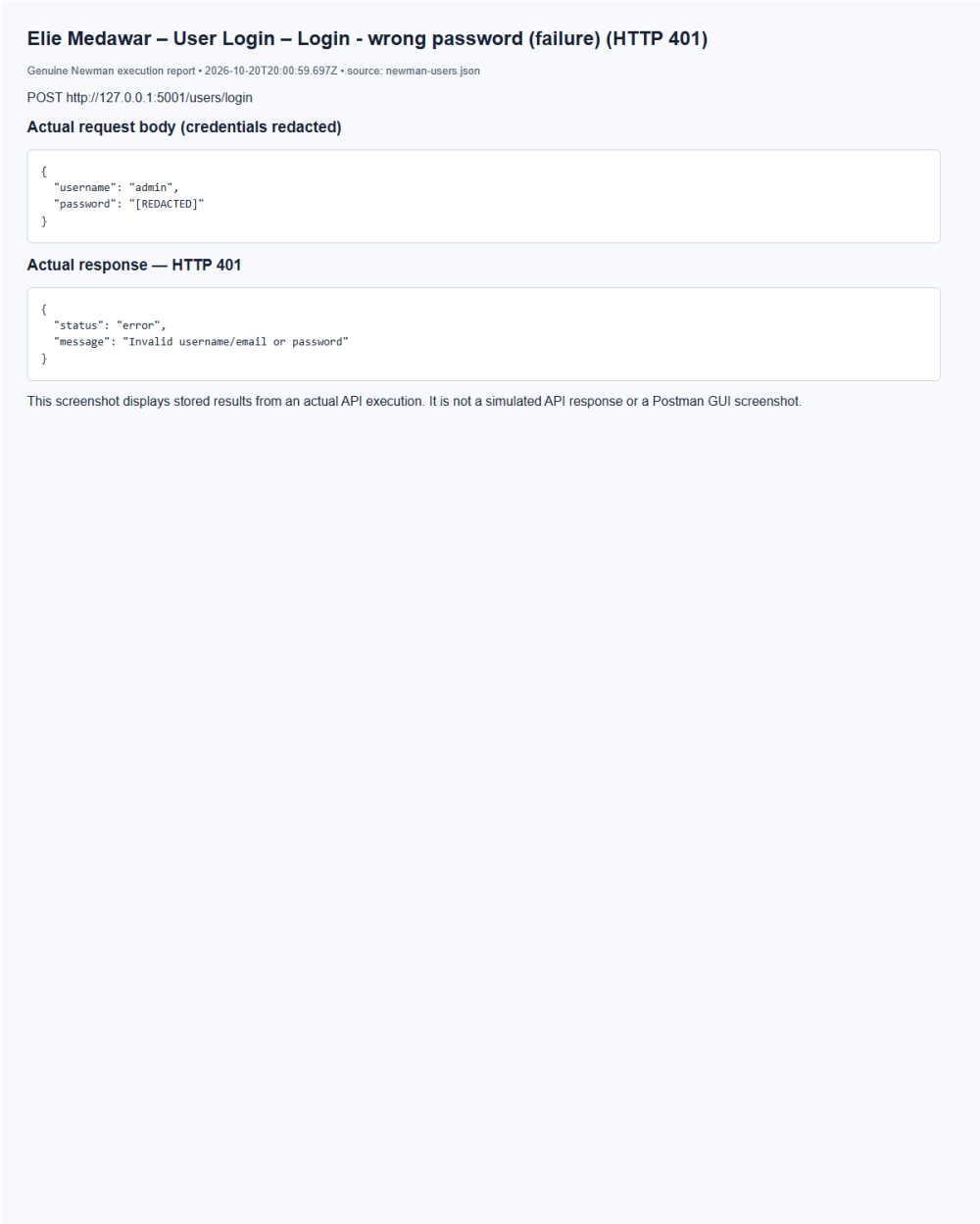

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Login successful",
  "data": {
    "token": "[REDACTED]",
    "token_type": "[REDACTED]",
    "user": {
      "id": 1,
      "full_name": "Elie Medawar",
      "username": "admin",
      "email": "admin@corp.com",
      "department": "Security",
      "role": "admin",
      "account_status": "Active",
      "created_at": "2026-10-20T20:00:51Z",
      "last_login": "2026-10-20T20:00:54Z"
    }
  }
}
```


This screenshot displays stored results from an actual API execution. It is not a simulated API response or a Postman GUI screenshot.

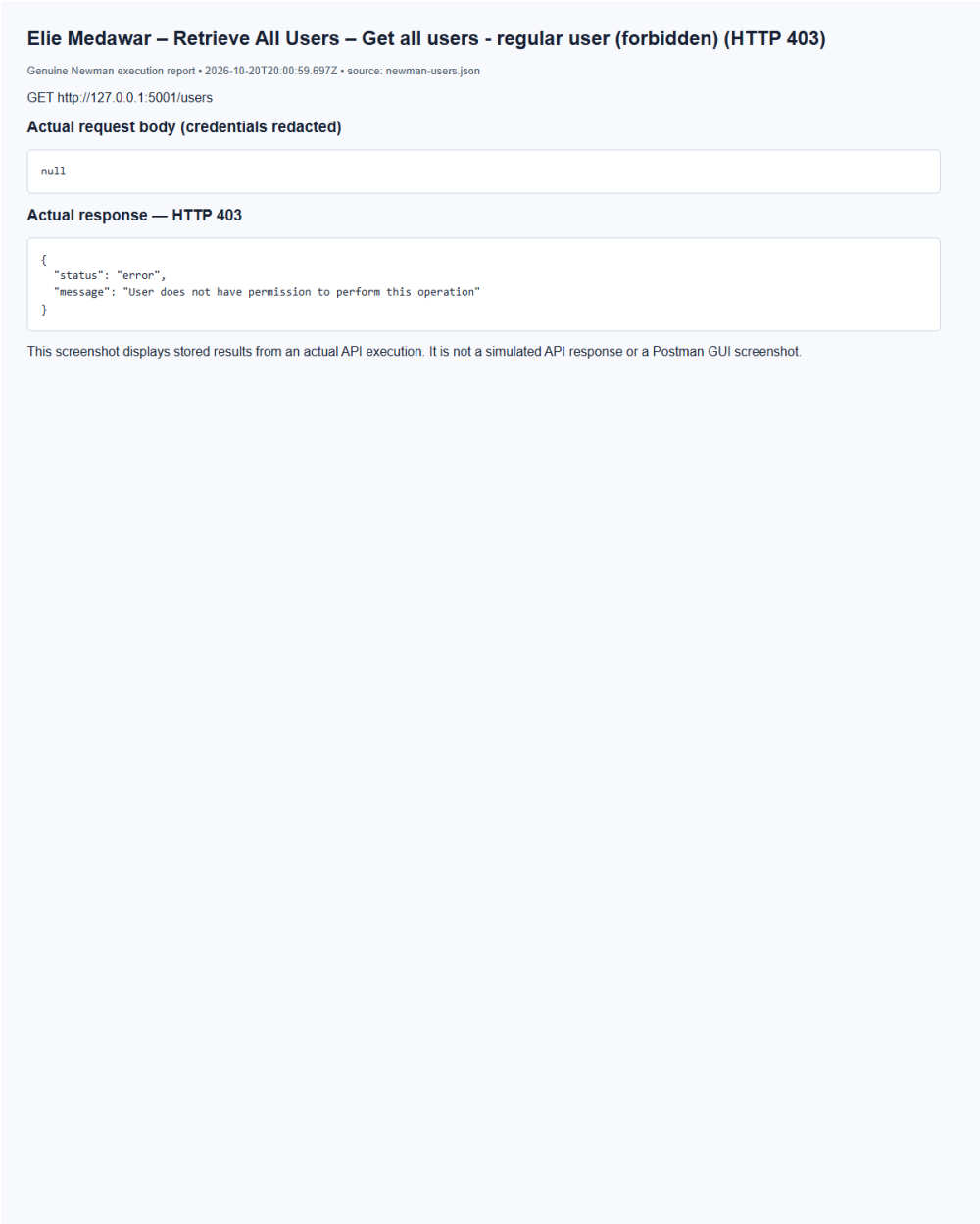
(Output shortened; the full capture is report/screenshots/elie/newman-users-01.png in the submission.)

Elie Medawar – User Login – Login - wrong password (failure) (HTTP 401)



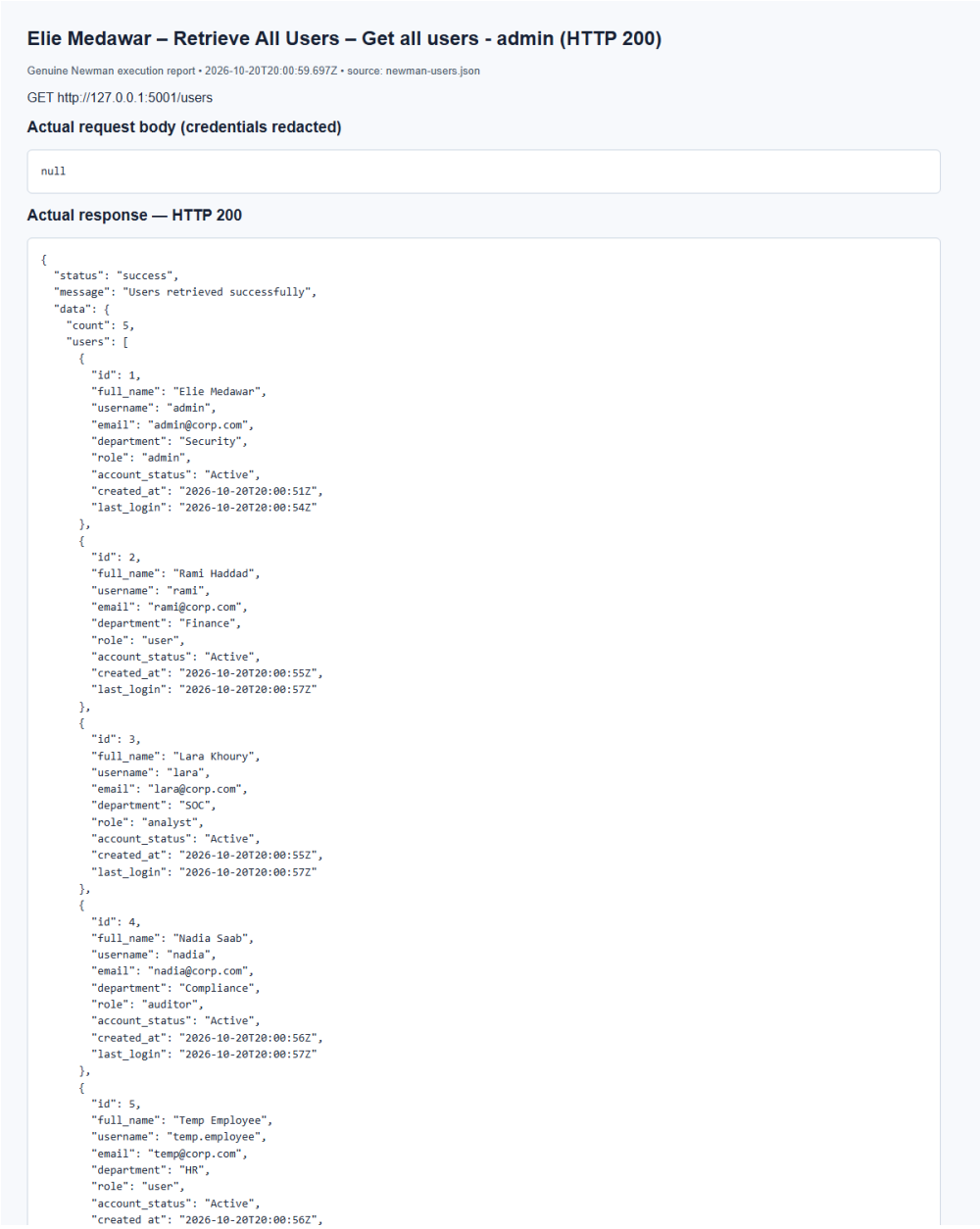
(Output shortened; the full capture is report/screenshots/elie/newman-users-02.png in the submission.)

Elie Medawar – Retrieve All Users – Get all users - regular user (forbidden) (HTTP 403)



(Output shortened; the full capture is report/screenshots/elie/newman-users-20.png in the submission.)

Elie Medawar – Retrieve All Users – Get all users - admin (HTTP 200)



Elie Medawar – Retrieve All Users – Get all users - admin (HTTP 200)

Genuine Newman execution report • 2026-10-20T20:00:59.697Z • source: newman-users.json

GET http://127.0.0.1:5001/users

Actual request body (credentials redacted)

```
null
```

Actual response — HTTP 200


```
{
  "status": "success",
  "message": "Users retrieved successfully",
  "data": {
    "count": 5,
    "users": [
      {
        "id": 1,
        "full_name": "Elie Medawar",
        "username": "admin",
        "email": "admin@corp.com",
        "department": "Security",
        "role": "admin",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:51Z",
        "last_login": "2026-10-20T20:00:54Z"
      },
      {
        "id": 2,
        "full_name": "Rami Haddad",
        "username": "rami",
        "email": "rami@corp.com",
        "department": "Finance",
        "role": "user",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:55Z",
        "last_login": "2026-10-20T20:00:57Z"
      },
      {
        "id": 3,
        "full_name": "Lara Khoury",
        "username": "lara",
        "email": "lara@corp.com",
        "department": "SOC",
        "role": "analyst",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:55Z",
        "last_login": "2026-10-20T20:00:57Z"
      },
      {
        "id": 4,
        "full_name": "Nadia Saab",
        "username": "nadia",
        "email": "nadia@corp.com",
        "department": "Compliance",
        "role": "auditor",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:56Z",
        "last_login": "2026-10-20T20:00:57Z"
      },
      {
        "id": 5,
        "full_name": "Temp Employee",
        "username": "temp.employee",
        "email": "temp@corp.com",
        "department": "HR",
        "role": "user",
        "account_status": "Active",
        "created_at": "2026-10-20T20:00:56Z",
```


(Output shortened; the full capture is report/screenshots/elie/newman-users-17.png in the submission.)

10. GitHub and Version Control (Elie Medawar)

The project lives in the GitHub repository https://github.com/eliemedawar/435L-Project. Each member worked on their own branches and pushed their own commits; the branches were merged into main through pull request #1 (Elie Medawar's services) and the merge of Marc Bteich's b/integration branch.

Workflow.

- Branches are named a/<service> (Elie Medawar) and b/<service> (Marc Bteich), one per service.
- Each member committed and pushed only their own work; neither edited the other's files. Shared code in common/ was changed only by its owner.
- Commits are small, with messages describing the change.
- .env (secrets), .local/ (private tool output), the local SQLite database and virtual environments are excluded by .gitignore.

Tasks completed by each member.

Member	Tasks
Elie Medawar	Shared common/ package (JWT auth, role checks, errors, validation, service clients, app factory, run_all.py); Users and Identity, Incident Management, Security Audit Logging and Notification services; summary, incident and activity reports; their Postman collections and execution evidence; report sections 1, 2, 5.1, 5.3, 5.5, 5.6 (summary/incidents/activity), 5.8, 7, 9, 10
Marc Bteich	Asset Management, Threat Intelligence, Vulnerability Management and Playbook services; asset, threat and vulnerability reports and the reports app; database schema, diagrams and seed data; merged Postman collection and execution evidence; report sections 3, 4, 5.2, 5.4, 5.6 (assets/threats/vulnerabilities), 5.7, 5.9, 6, 8
Zoulfikar Shokor	Joined the team after the Sprint I implementation was completed, so has no Sprint I commits.

GitHub – branches, contributors and full commit history of main

GitHub history – https://github.com/eliemedawar/435L-Project

Retrieved live from the GitHub API on 20 October 2026.

Branches

a/common, a/incidents, a/logs, a/notifications, a/reports, b/assets, b/database, b/integration, b/playbooks, b/reports, b/threats, b/vulnerabilities, main

Contributors (commits on the default branch)

GitHub account	Commits
eliemedawar	16
marcbteich1	12

Commit history on main

Commit	Author	Date (UTC)	Message
653eb1f	eliemedawar	2026-10-20 19:14	Ignore .local/ so private seed credentials and tool output are never committed
e3e80df	eliemedawar	2026-10-20 19:14	Merge Marc Bteich's services, database, reports and evidence (b/integration)
4aa6bf3	marcbteich1	2026-10-20 12:17	Prepare Person B report, API reference, diagrams and genuine execution evidence
7bf8135	marcbteich1	2026-10-20 12:13	Execute individual and merged Postman collections with redacted evidence tooling
2e1512d	marcbteich1	2026-10-20 12:09	Preserve response execution history in audit logs before cascade deletion
9e5c32e	marcbteich1	2026-10-20 11:56	Seed all ten tables with repeatable laboratory data and private accounts
974299a	marcbteich1	2026-10-20 11:56	Integrate all six HTTP-based reports and verify nine-service workflows
8cda771	marcbteich1	2026-10-20 11:55	Implement ordered response actions with transactional execution history
feacdec	marcbteich1	2026-10-20 11:53	Implement and verify vulnerability remediation lifecycle and notifications
0a39011	marcbteich1	2026-10-20 11:50	Implement and HTTP-test Threat Intelligence with critical admin alerts
631e39d	marcbteich1	2026-10-20 11:42	Document Assets setup and add attributed Postman test collection
409d176	marcbteich1	2026-10-20 11:41	Reject oversized asset identifiers before database queries
8339e45	marcbteich1	2026-10-20 11:38	Implement Assets APIs with authorization and live HTTP integration tests
c42453b	marcbteich1	2026-10-20 11:35	Add ten-table SQLite schema and verify database constraints
6a0160d	eliemedawar	2026-10-18 15:16	Merge pull request #1 from eliemedawar/a/reports
73d4f6b	eliemedawar	2026-10-18 14:18	Add Postman collection for the summary, incident and activity reports
ec2b90b	eliemedawar	2026-10-18 14:18	Implemented security summary, incident and activity reports from service statistics
6803b30	eliemedawar	2026-10-18 14:17	Add Postman collection for the Notification service
13b9341	eliemedawar	2026-10-18 14:17	Implemented Notification service: create, per-user list, retrieve, mark as read, search and delete
eccf64f	eliemedawar	2026-10-18 14:17	Add Postman collection for the Incident Management service
01b087c	eliemedawar	2026-10-18 14:17	Implemented Incident Management service: create, retrieve, assign, lifecycle status and investigation notes
0573854	eliemedawar	2026-10-18 14:17	Load all models up front, create tables once in run_all, add fetch_service helper
aea1d1b	eliemedawar	2026-10-18 13:59	Add Postman collection for the Audit Logging service
c4d1482	eliemedawar	2026-10-18 13:59	Implemented Audit Logging service: create, retrieve, search, delete and per-user log APIs
ea3660e	eliemedawar	2026-10-18 13:56	Add Postman collection for the Users service
a3eac84	eliemedawar	2026-10-18 13:56	Implemented Users service: registration, login, update, delete and retrieval APIs
96bc70f	eliemedawar	2026-10-18 13:56	Add shared common package: config, database, errors, validation, JWT auth and service clients
00b11ff	eliemedawar	2026-10-18 12:45	Add .gitignore

(Output shortened; the full capture is report/screenshots/github/github-history.png in the submission.)

11. References (Elie Medawar, Marc Bteich)

- Course brief: Final Project Fall 2026-2027 – Sprint I.
- Flask documentation – https://flask.palletsprojects.com/
- Flask-SQLAlchemy documentation – https://flask-sqlalchemy.readthedocs.io/
- SQLAlchemy 2.0 documentation – https://docs.sqlalchemy.org/en/20/
- PyJWT documentation – https://pyjwt.readthedocs.io/

- bcrypt for Python – https://pypi.org/project/bcrypt/
- SQLite foreign key support – https://www.sqlite.org/foreignkeys.html
- Python ipaddress – https://docs.python.org/3/library/ipaddress.html
- Postman collection format and Newman – https://learning.postman.com/docs/reference/newman-cli/command-line-integration-with-newman/
- OWASP Top 10 (injection, broken access control) – https://owasp.org/Top10/

Use of AI tools

Elie Medawar: Claude Code (Anthropic) was used to split the work between the members; to generate a first version of the shared common/ package, the Users, Incident Management, Audit Logging and Notification services, the summary/incident/activity reports and their Postman collections; to write test and evidence scripts; and to draft Elie Medawar's report sections and assemble this document. Purpose: to speed up repetitive code and documentation. Elie Medawar reviewed, ran and tested the work and can explain each design decision. All execution evidence comes from the team's own running system.

Marc Bteich: OpenAI Codex assisted with interpreting the team contract, implementing Person B services and database scripts, writing/running automated and Newman tests, debugging integration, and preparing diagrams/documentation/evidence reports. Generated code and prose require Marc’s personal review, adaptation and understanding. Execution results come from actual local runs; synthetic seed data is labelled. No screenshots are represented as a Postman GUI session when they are browser captures of Newman execution reports.